



RS6x_7x SDK

参考手册

文档页数：192
文档版本号：1.1
发布时间：2025-10-15

版本历史

版本	版本描述	日期
1.2	修复 3.3.3.4 错别字(其他教常用配置—>其他常用配置); 删除表 4-9 参数项多余的大括号	2025-10-29
1.1	更新 HIF 接口描述	2025-10-15
1.0	初始版本; 新建文档	2025-09-10

目录

版本历史.....	ii
目录.....	iii
图片目录.....	X
表格目录.....	XIII
1 概述.....	1
2 SDK 简介.....	2
2.1 架构简介.....	2
2.1.1 RS6130 硬件架构.....	2
2.1.2 RS6240 硬件架构.....	3
2.1.3 SDK 软件架构.....	4
2.2 目录结构.....	4
2.2.1 编译工具.....	4
2.2.2 软件源码.....	4
2.3 参考资料.....	6
3 新建工程.....	7
3.1 工程目录结构.....	7
3.2 工程创建.....	7
3.2.1 添加源文件.....	9
3.2.2 工程配置.....	14
3.2.3 板级选择.....	18
3.3 新建开发模板.....	20
3.3.1 基于 RS6130 新建.....	20
3.3.2 基于 RS6240 新建.....	22
3.3.3 配置项修改.....	24
3.3.3.1 时钟配置.....	24
3.3.3.2 Pinmux 配置.....	24
3.3.3.3 FLASH/XIP 运行频率.....	25
3.3.3.4 其他配置.....	26
4 子系统.....	28
4.1 CPU.....	28
4.1.1 Cache.....	30
4.1.2 浮点.....	31
4.1.3 双核.....	31
4.1.4 代码优化指引.....	31

4.1.4.1 SRAM 占用分析.....	31
4.1.4.2 HEAP 占用分析.....	33
4.2 DSP.....	34
4.3 FLASH 和 PSRAM.....	35
4.3.1 代码在 XIP 或 SRAM.....	35
4.4 Memory Map.....	37
4.4.1 Img 格式.....	37
4.4.2 Map.....	38
4.5 启动和配置.....	40
4.5.1 初始化.....	40
4.5.2 配置.....	40
4.6 Clock.....	41
4.7 Pinmux.....	42
4.8 LOG.....	43
4.9 控制台 Shell.....	45
4.10 电源管理.....	46
4.10.1 简介.....	46
4.10.2 接口.....	47
4.10.2.1 pm_device_register.....	47
4.10.2.2 pm_device_unregister.....	48
4.10.2.3 pm_notifier_register.....	48
4.10.2.4 pm_notifier_unregister.....	49
4.10.2.5 pm_policy_latency_update.....	49
4.10.2.6 pm_policy_wake_lock_is_active.....	49
4.10.2.7 pm_policy_wake_lock.....	50
4.10.2.8 pm_policy_wake_unlock.....	50
4.10.2.9 pm_wake_lock_timeout.....	51
4.10.2.10 pm_device_runtime_enable.....	51
4.10.2.11 pm_device_runtime_disable.....	51
4.10.2.12 pm_device_runtime_get.....	52
4.10.2.13 pm_device_runtime_put.....	52
4.11 KVR & KVF.....	53
4.11.1 简介.....	53
4.11.2 接口.....	53
4.11.2.1 KVR_Save.....	53
4.11.2.2 KVR_Load.....	53
4.11.2.3 KVR_Delete.....	54
4.11.2.4 KVF_Save.....	54
4.11.2.5 KVF_Load.....	55
4.11.2.6 KVF_Delete.....	55
4.12 Version.....	56
4.12.1 简介.....	56
4.12.2 接口.....	57
4.12.2.1 VER_GetSdkVersion.....	57
4.12.2.2 VER_GetSubimgVersion.....	58
4.12.2.3 VER_GetBootVersion.....	58

4.12.2.4 VER_GetBuildTime.....	59
4.13 HIF.....	59
4.13.1 功能简介.....	59
4.13.2 宏定义开关.....	59
4.13.3 接口说明.....	60
4.13.3.1 HIF_Init.....	60
4.13.3.2 HIF_DeInit.....	61
4.13.3.3 HIF_DefaultInitParam.....	62
4.13.3.4 HIF_ExtControl.....	62
4.13.3.5 HIF_ParamGet.....	63
4.13.3.6 HIF_MsgHdl_Regist.....	64
4.13.3.7 HIF_MsgResp.....	65
4.13.3.8 HIF_MsgReport.....	66
4.13.3.9 HIF_MsgReport_ListAddData.....	66
4.13.3.10 :HIF_MsgReport_ListStart.....	67
4.14 Other.....	68
4.14.1 接口.....	68
4.14.1.1 HAL_BOARD_GetFreq.....	68
4.14.1.2 HAL_BOARD_GetTime.....	69
4.14.1.3 HAL_BOARD_BlDelay.....	69
4.14.1.4 HAL_BOARD_GetRand.....	70
4.14.1.5 HAL_BOARD_Reset.....	70
5 外设驱动.....	71
5.1 GPIO.....	71
5.1.1 功能简介.....	71
5.1.1.1 上下拉电阻.....	71
5.1.1.2 输出驱动能力.....	72
5.1.1.3 功能模式.....	72
5.1.1.4 GPIO 与电源管理 WKIO.....	73
5.1.1.5 Debounce.....	73
5.1.2 接口.....	73
5.1.2.1 HAL_GPIO_Init.....	74
5.1.2.2 HAL_GPIO_DeInit.....	74
5.1.2.3 HAL_GPIO_SetPinParam.....	75
5.1.2.4 HAL_GPIO_GetPinParam.....	75
5.1.2.5 HAL_GPIO_DisablePin.....	76
5.1.2.6 HAL_GPIO_SetPinMode.....	77
5.1.2.7 HAL_GPIO_SetPinPull.....	77
5.1.2.8 HAL_GPIO_SetPinPush.....	78
5.1.2.9 HAL_GPIO_SetPinDriving.....	79
5.1.2.10 HAL_GPIO_ReadPort.....	80
5.1.2.11 HAL_GPIO_WritePort.....	80
5.1.2.12 HAL_GPIO_ReadPin.....	80
5.1.2.13 HAL_GPIO_WritePin.....	81
5.1.2.14 HAL_GPIO_TogglePin.....	82
5.1.2.15 HAL_GPIO_SetIRQParam.....	82
5.1.2.16 HAL_GPIO_SetIRQDebounce.....	83
5.1.2.17 HAL_GPIO_EnableIRQ.....	83

5.1.2.18 HAL_GPIO_DisableIRQ.....	84
5.1.2.19 HAL_WKIO_Init.....	84
5.1.2.20 HAL_WKIO_DeInit.....	85
5.1.2.21 HAL_WKIO_SetPinMode.....	85
5.1.2.22 HAL_WKIO_GetPinMode.....	86
5.1.2.23 HAL_WKIO_SetPinPull.....	87
5.1.2.24 HAL_WKIO_GetPinPull.....	88
5.1.2.25 HAL_WKIO_WritePin.....	88
5.1.2.26 HAL_WKIO_WritePort.....	89
5.1.2.27 HAL_WKIO_SetWakeParam.....	90
5.1.2.28 HAL_WKIO_EnableWake.....	90
5.1.2.29 HAL_WKIO_DisableWake.....	91
5.1.2.30 HAL_WKIO_SetWakeEvent.....	91
5.1.2.31 HAL_WKIO_GetWakeEvent.....	92
5.1.2.32 HAL_WKIO_ReadWakePin.....	92
5.1.2.33 HAL_WKIO_SetWakeDebounce.....	93
5.2 UART.....	94
5.2.1 功能简介.....	94
5.2.1.1 特性.....	94
5.2.1.2 使用.....	94
5.2.2 接口.....	95
5.2.2.1 HAL_UART_Init.....	95
5.2.2.2 HAL_UART_DeInit.....	96
5.2.2.3 HAL_UART_ExtControl.....	96
5.2.2.4 HAL_UART_Open.....	97
5.2.2.5 HAL_UART_Close.....	97
5.2.2.6 HAL_UART_SetTransferMode.....	98
5.2.2.7 HAL_UART_Receive.....	98
5.2.2.8 HAL_UART_Transmit.....	99
5.2.2.9 HAL_UART_GetRxData.....	100
5.2.2.10 HAL_UART_PutTxData.....	100
5.2.2.11 HAL_UART_IsTxReady.....	100
5.2.2.12 HAL_UART_IsTxEmpty.....	101
5.2.2.13 HAL_UART_IsRxReady.....	101
5.2.2.14 HAL_UART_RegisterIRQ.....	101
5.2.2.15 HAL_UART_UnregisterIRQ.....	102
5.2.2.16 HAL_UART_EnableTxReadyIRQ.....	103
5.2.2.17 HAL_UART_EnableRxReadyIRQ.....	103
5.2.2.18 HAL_UART_DisableTxReadyIRQ.....	104
5.2.2.19 HAL_UART_DisableRxReadyIRQ.....	104
5.2.2.20 HAL_UART_RegisterDMA.....	104
5.2.2.21 HAL_UART_UnregisterDMA.....	105
5.2.2.22 HAL_UART_StartTransmitDMA.....	106
5.2.2.23 HAL_UART_StopTransmitDMA.....	106
5.2.2.24 HAL_UART_StartReceiveDMA.....	106
5.2.2.25 HAL_UART_StopReceiveDMA.....	107
5.3 SPI.....	108
5.3.1 功能简介.....	108
5.3.1.1 特性.....	108
5.3.1.2 使用.....	110

5.3.2 模式介绍.....	112
5.3.2.1 SPI_MODE1.....	113
5.3.2.2 SPI_MODE2.....	114
5.3.3 接口.....	115
5.3.3.1 HAL_SPI_Init.....	115
5.3.3.2 HAL_SPI_DeInit.....	115
5.3.3.3 HAL_SPI_Open.....	116
5.3.3.4 HAL_SPI_Close.....	116
5.3.3.5 HAL_SPI_ExtControl.....	116
5.3.3.6 HAL_SPI_SetTransferMode.....	117
5.3.3.7 HAL_SPI_Receive.....	118
5.3.3.8 HAL_SPI_Transmit.....	119
5.3.3.9 HAL_SPI_TransmitReceive.....	119
5.3.3.10 HAL_SPI_GetRxData.....	120
5.3.3.11 HAL_SPI_PutTxData.....	121
5.3.3.12 HAL_SPI_StartTrans.....	121
5.3.3.13 HAL_SPI_StopTrans.....	121
5.3.3.14 HAL_SPI_IsTxReady.....	122
5.3.3.15 HAL_SPI_IsTxEmpty.....	122
5.3.3.16 HAL_SPI_IsRxReady.....	122
5.3.3.17 HAL_SPI_IsBusy.....	123
5.3.3.18 HAL_SPI_GetFifoLevel.....	123
5.3.3.19 HAL_SPI_IsCsInterrupt.....	124
5.3.3.20 HAL_SPI_EnableDMA.....	124
5.3.3.21 HAL_SPI_DisableDMA.....	124
5.3.3.22 HAL_SPI_EnableCS.....	125
5.3.3.23 HAL_SPI_DisableCS.....	125
5.3.3.24 HAL_SPI_RegisterIRQ.....	126
5.3.3.25 HAL_SPI_UnregisterIRQ.....	126
5.3.3.26 HAL_SPI_EnableIRQ.....	127
5.3.3.27 HAL_SPI_DisableIRQ.....	128
5.3.3.28 HAL_SPI_RegisterDMA.....	128
5.3.3.29 HAL_SPI_UnregisterDMA.....	129
5.3.3.30 HAL_SPI_StartDMA.....	129
5.3.3.31 HAL_SPI_StopDMA.....	130
5.3.3.32 HAL_SPI_PM_Suspend.....	130
5.3.3.33 HAL_SPI_PM_Resume.....	131
5.4 I2C.....	132
5.4.1 功能简介.....	132
5.4.1.1 特性.....	132
5.4.1.2 使用.....	132
5.4.2 接口.....	133
5.4.2.1 HAL_I2C_Init.....	133
5.4.2.2 HAL_I2C_DeInit.....	134
5.4.2.3 HAL_I2C_Open.....	134
5.4.2.4 HAL_I2C_Open.....	135
5.4.2.5 HAL_I2C_RecoverBus.....	135
5.4.2.6 HAL_I2C_SetTransferMode.....	136
5.4.2.7 HAL_I2C_Transmit.....	137
5.4.2.8 HAL_I2C_Receive.....	137
5.4.2.9 HAL_I2C_GetRxData.....	139

5.4.2.10 HAL_I2C_PutTxData.....	139
5.4.2.11 HAL_I2C_IsTxReady.....	140
5.4.2.12 HAL_I2C_IsRxReady.....	140
5.4.2.13 HAL_I2C_IsTxEmpty.....	140
5.4.2.14 HAL_I2C_RegisterIRQ.....	141
5.4.2.15 HAL_I2C_UnregisterIRQ.....	141
5.4.2.16 HAL_I2C_EnableIRQ.....	142
5.4.2.17 HAL_I2C_DisableIRQ.....	142
5.4.2.18 HAL_I2C_ClearIRQ.....	143
5.4.2.19 HAL_I2C_GetIRQStatus.....	144
5.5 DMA.....	145
5.5.1 功能简介.....	145
5.5.1.1 块传输.....	145
5.5.1.2 链表传输.....	148
5.5.2 接口.....	149
5.5.2.1 HAL_DMA_Init.....	149
5.5.2.2 HAL_DMA_DeInit.....	150
5.5.2.3 HAL_DMA_Open.....	150
5.5.2.4 HAL_DMA_Close.....	151
5.5.2.5 HAL_DMA_ConfigBlockTransfer.....	151
5.5.2.6 HAL_DMA_InitNodeCtrl.....	152
5.5.2.7 HAL_DMA_AddNextNode.....	153
5.5.2.8 HAL_DMA_ConfigListTransfer.....	153
5.5.2.9 HAL_DMA_ConfigScatterGatherTransfer.....	154
5.5.2.10 HAL_DMA_StartTransfer.....	155
5.5.2.11 HAL_DMA_AbortTransfer.....	155
5.5.2.12 HAL_DMA_RegisterIRQ.....	156
5.5.2.13 HAL_DMA_UnRegisterIRQ.....	157
5.5.2.14 HAL_DMA_EnableIRQ.....	157
5.5.2.15 HAL_DMA_DisableIRQ.....	158
5.5.2.16 HAL_DMA_GetChanStatus.....	158
5.6 FLASH.....	160
5.6.1 功能简介.....	160
5.6.2 接口.....	163
5.6.2.1 HAL_FLASH_Init.....	163
5.6.2.2 HAL_FLASH_DeInit.....	164
5.6.2.3 HAL_FLASH_Open.....	164
5.6.2.4 HAL_FLASH_Close.....	165
5.6.2.5 HAL_FLASH_Read.....	165
5.6.2.6 HAL_FLASH_Write.....	166
5.6.2.7 HAL_FLASH_Erase.....	166
5.6.2.8 HAL_FLASH_ExtControl.....	167
5.6.2.9 HAL_FLASH_Suspend.....	168
5.6.2.10 HAL_FLASH_Resume.....	169
5.7 WDG.....	169
5.7.1 功能简介.....	169
5.7.2 接口.....	170
5.7.2.1 HAL_WDG_Init.....	170
5.7.2.2 HAL_WDG_DeInit.....	170
5.7.2.3 HAL_WDG_Open.....	171

5.7.2.4 HAL_WDG_Close.....	172
5.7.2.5 HAL_WDG_Feed.....	172

图片目录

图 2-1	RS6130 系统框图.....	2
图 2-2	RS6130 应用子系统.....	3
图 2-3	RS6240 系统框图.....	3
图 2-4	RS6240 应用子系统.....	3
图 2-5	RS 系列 SDK 框架图.....	4
图 3-1	new_project 界面.....	8
图 3-2	修改工程名称菜单.....	8
图 3-3	输入工程名称.....	9
图 3-4	添加源文件目录.....	9
图 3-5	选择 kernel 文件夹.....	10
图 3-6	选择不复制文件夹方式.....	10
图 3-7	文件夹选择不编译.....	11
图 3-8	不编译的文件夹图标显示.....	11
图 3-9	新建虚拟文件夹.....	12
图 3-10	输入虚拟文件夹名称.....	12
图 3-11	为虚拟文件夹添加源文件.....	13
图 3-12	选择当前工程源文件.....	13
图 3-13	选择当前工程源文件.....	14
图 3-14	Taget 配置界面.....	14
图 3-15	Output 配置界面.....	15
图 3-16	User 配置界面.....	15
图 3-17	Compiler 配置界面.....	16
图 3-18	Assembler 配置界面.....	16
图 3-19	Linker 配置界面.....	17
图 3-20	Linker 文件选择.....	17
图 3-21	Device 配置界面.....	18
图 3-22	Options for myproject 选择 board.....	19
图 3-23	main 函数中添加应用.....	19
图 3-24	编译工程.....	20
图 3-25	CDK 加入新建 board.....	22
图 3-26	CDK 添加新 board 完成.....	22
图 3-27	myboard pinmux 配置.....	25
图 3-28	myboard pinmux 配置函数.....	25

图 3-29 myboard spi flash 配置.....	26
图 4-1 CDK CPU 指令集选择.....	29
图 4-2 CDK 链接脚本配置.....	29
图 4-3 CDK 雷达符号表配置.....	30
图 4-4 HEAP 占用情况.....	33
图 4-5 CDK 中 DSP lib 配置.....	34
图 4-6 启动过程.....	37
图 4-7 Img 组织形式.....	38
图 4-8 Img 格式.....	38
图 4-9 Img memory map.....	39
图 4-10 启动流程.....	40
图 4-11 配置调用关系.....	41
图 4-12 电源管理流程示意图.....	47
图 5-1 GPIO 框架示意图.....	71
图 5-2 UART 集成 API 使用流程.....	95
图 5-3 SPI Mode 示意图.....	109
图 5-4 SPI 集成 API 使用流程.....	110
图 5-5 SPI FIFO 使用示意图.....	112
图 5-6 SPI 模式选择.....	113
图 5-7 SPI 由于延时导致上升沿采样错误.....	113
图 5-8 SPI 采用 CLK_0 采数效果图.....	113
图 5-9 SPI 传输中 POP 示意图.....	114
图 5-10 I2C 集成 API 使用流程.....	132
图 5-11 DMA 块传输使用流程.....	146
图 5-12 DMA memory to memory 位宽 32bit 示意图.....	147
图 5-13 DMA memory to memory 位宽 8bit 示意图.....	147
图 5-14 DMA UART0-RX to memory 示意图.....	148
图 5-15 DMA UART0-RX to memory 示意图.....	148
图 5-16 DMA 链表传输使用流程.....	149
图 5-17 Flash Chip ID 示例.....	160
图 5-18 Dual SPI 传输示例.....	161
图 5-19 Quad SPI 传输示例.....	161
图 5-20 Spec SPI Format 支持示例.....	162
图 5-21 Spec QER 示例.....	162
图 5-22 Spec SPI max freq 示例.....	163
图 5-23 Spec sleep latency 示例.....	163

图 5-24 watchdog 使用示意图..... 170

表格目录

表 4-1	RS6130 CPU 规格.....	28
表 4-2	RS6240 CPU 规格.....	28
表 4-3	HEAP SHELL 返回值释义.....	33
表 4-4	FLASH XIP 地址分布.....	35
表 4-5	PSRAM XIP 地址分布.....	35
表 4-6	Address Remap.....	39
表 4-7	6130 时钟配置.....	42
表 4-8	6240 时钟配置.....	42
表 4-9	pm_device_register.....	47
表 4-10	pm_device_unregister.....	48
表 4-11	pm_notifier_register.....	48
表 4-12	pm_notifier_unregister.....	49
表 4-13	pm_policy_latency_update.....	49
表 4-14	pm_policy_wake_lock_is_active.....	49
表 4-15	pm_policy_wake_lock.....	50
表 4-16	pm_policy_wake_unlock.....	50
表 4-17	pm_wake_lock_timeout.....	51
表 4-18	pm_device_runtime_enable.....	51
表 4-19	pm_device_runtime_disable.....	51
表 4-20	pm_device_runtime_get.....	52
表 4-21	pm_device_runtime_put.....	52
表 4-22	KVR_Save.....	53
表 4-23	KVR_Load.....	53
表 4-24	KVR_Delete.....	54
表 4-25	KVF_Save.....	54
表 4-26	KVF_Load.....	55
表 4-27	KVF_Delete.....	55
表 4-28	VER_GetSubimgVersion.....	57
表 4-29	VER_GetSdkVersion.....	58
表 4-30	VER_GetBootVersion.....	58
表 4-31	VER_GetBuildTime.....	59
表 4-32	CONFIG_HIF.....	59
表 4-33	HIF_Init.....	60

表 4-34	HIF_DeInit.....	61
表 4-35	HIF_DefaultInitParam.....	62
表 4-36	HIF_ExtControl.....	62
表 4-37	HIF_ParamGet.....	63
表 4-38	HIF_MsgHdl_Regist.....	64
表 4-39	HIF_MsgResp.....	65
表 4-40	HIF_MsgReport.....	66
表 4-41	HIF_MsgReport_ListAddData.....	66
表 4-42	HIF_MsgReport_ListStart.....	67
表 4-43	HAL_BOARD_GetFreq.....	68
表 4-44	HAL_BOARD_GetTime.....	69
表 4-45	HAL_BOARD_BlkDelay.....	69
表 4-46	HAL_BOARD_GetRand.....	70
表 4-47	HAL_BOARD_Reset.....	70
表 5-1	HAL_GPIO_Init.....	74
表 5-2	HAL_GPIO_DeInit.....	74
表 5-3	HAL_GPIO_SetPinParam.....	75
表 5-4	HAL_GPIO_GetPinParam.....	75
表 5-5	HAL_GPIO_DisablePin.....	76
表 5-6	HAL_GPIO_SetPinMode.....	77
表 5-7	HAL_GPIO_SetPinPull.....	77
表 5-8	HAL_GPIO_SetPinPush.....	78
表 5-9	HAL_GPIO_SetPinDriving.....	79
表 5-10	HAL_GPIO_ReadPort.....	80
表 5-11	HAL_GPIO_WritePort.....	80
表 5-12	HAL_GPIO_ReadPin.....	80
表 5-13	HAL_GPIO_WritePin.....	81
表 5-14	HAL_GPIO_TogglePin.....	82
表 5-15	HAL_GPIO_SetIRQParam.....	82
表 5-16	HAL_GPIO_SetIRQDebounce.....	83
表 5-17	HAL_GPIO_EnableIRQ.....	83
表 5-18	HAL_GPIO_DisableIRQ.....	84
表 5-19	HAL_WKIO_Init.....	84
表 5-20	HAL_WKIO_DeInit.....	85
表 5-21	HAL_WKIO_SetPinMode.....	85
表 5-22	HAL_WKIO_GetPinMode.....	86

表 5-23	HAL_WKIO_SetPinPull.....	87
表 5-24	HAL_WKIO_GetPinPull.....	88
表 5-25	HAL_WKIO_WritePin.....	88
表 5-26	HAL_WKIO_WritePort.....	89
表 5-27	HAL_WKIO_SetWakeParam.....	90
表 5-28	HAL_WKIO_EnableWake.....	90
表 5-29	HAL_WKIO_DisableWake.....	91
表 5-30	HAL_WKIO_SetWakeEvent.....	91
表 5-31	HAL_WKIO_GetWakeEvent.....	92
表 5-32	HAL_WKIO_ReadWakePin.....	92
表 5-33	HAL_WKIO_SetWakeDebounce.....	93
表 5-34	HAL_UART_Init.....	95
表 5-35	HAL_UART_DeInit.....	96
表 5-36	HAL_UART_ExtControl.....	96
表 5-37	HAL_UART_Open.....	97
表 5-38	HAL_UART_Close.....	97
表 5-39	HAL_UART_SetTransferMode.....	98
表 5-40	HAL_UART_Receive.....	98
表 5-41	HAL_UART_Transmit.....	99
表 5-42	HAL_UART_GetRxData.....	100
表 5-43	HAL_UART_PutTxData.....	100
表 5-44	HAL_UART_IsTxReady.....	100
表 5-45	HAL_UART_IsTxEmpty.....	101
表 5-46	HAL_UART_IsRxReady.....	101
表 5-47	HAL_UART_RegisterIRQ.....	101
表 5-48	HAL_UART_UnregisterIRQ.....	102
表 5-49	HAL_UART_EnableTxReadyIRQ.....	103
表 5-50	HAL_UART_EnableRxReadyIRQ.....	103
表 5-51	HAL_UART_DisableTxReadyIRQ.....	104
表 5-52	HAL_UART_DisableRxReadyIRQ.....	104
表 5-53	HAL_UART_RegisterDMA.....	104
表 5-54	HAL_UART_UnregisterDMA.....	105
表 5-55	HAL_UART_StartTransmitDMA.....	106
表 5-56	HAL_UART_StopTransmitDMA.....	106
表 5-57	HAL_UART_StartReceiveDMA.....	106
表 5-58	HAL_UART_StopReceiveDMA.....	107

表 5-59	SPI pin 功能.....	108
表 5-60	HAL_SPI_Init.....	115
表 5-61	HAL_SPI_DeInit.....	115
表 5-62	HAL_SPI_Open.....	116
表 5-63	HAL_SPI_Close.....	116
表 5-64	HAL_SPI_ExtControl.....	116
表 5-65	HAL_SPI_SetTransferMode.....	117
表 5-66	HAL_SPI_Receive.....	118
表 5-67	HAL_SPI_Transmit.....	119
表 5-68	HAL_SPI_TransmitReceive.....	119
表 5-69	HAL_SPI_GetRxData.....	120
表 5-70	HAL_SPI_PutTxData.....	121
表 5-71	HAL_SPI_StartTrans.....	121
表 5-72	HAL_SPI_StopTrans.....	121
表 5-73	HAL_SPI_IsTxReady.....	122
表 5-74	HAL_SPI_IsTxEmpty.....	122
表 5-75	HAL_SPI_IsRxReady.....	122
表 5-76	HAL_SPI_IsBusy.....	123
表 5-77	HAL_SPI_GetFifoLevel.....	123
表 5-78	HAL_SPI_IsCsInterrupt.....	124
表 5-79	HAL_SPI_EnableDMA.....	124
表 5-80	HAL_SPI_DisableDMA.....	124
表 5-81	HAL_SPI_EnableCS.....	125
表 5-82	HAL_SPI_DisableCS.....	125
表 5-83	HAL_SPI_RegisterIRQ.....	126
表 5-84	HAL_SPI_UnregisterIRQ.....	126
表 5-85	HAL_SPI_EnableIRQ.....	127
表 5-86	HAL_SPI_DisableIRQ.....	128
表 5-87	HAL_SPI_RegisterDMA.....	128
表 5-88	HAL_SPI_UnregisterDMA.....	129
表 5-89	HAL_SPI_StartDMA.....	129
表 5-90	HAL_SPI_StopDMA.....	130
表 5-91	HAL_SPI_PM_Suspend.....	130
表 5-92	HAL_SPI_PM_Resume.....	131
表 5-93	HAL_I2C_init.....	133
表 5-94	HAL_I2C_Deinit.....	134

表 5-95	HAL_I2C_Open.....	134
表 5-96	HAL_I2C_Close.....	135
表 5-97	HAL_I2C_RecoverBus.....	135
表 5-98	HAL_I2C_SetTransferMode.....	136
表 5-99	HAL_I2C_Transmit.....	137
表 5-100	HAL_I2C_Receive.....	137
表 5-101	HAL_I2C_GetRxData.....	139
表 5-102	HAL_I2C_PutTxData.....	139
表 5-103	HAL_I2C_IsTxReady.....	140
表 5-104	HAL_I2C_IsRxReady.....	140
表 5-105	HAL_I2C_IsTxEmpty.....	140
表 5-106	HAL_I2C_RegisterIRQ.....	141
表 5-107	HAL_I2C_UnregisterIRQ.....	141
表 5-108	HAL_I2C_EnableIRQ.....	142
表 5-109	HAL_I2C_DisableIRQ.....	142
表 5-110	HAL_I2C_ClearIRQ.....	143
表 5-111	HAL_I2C_GetIRQStatus.....	144
表 5-112	HAL_DMA_Init.....	149
表 5-113	HAL_DMA_DeInit.....	150
表 5-114	HAL_DMA_Open.....	150
表 5-115	HAL_DMA_Close.....	151
表 5-116	HAL_DMA_ConfigBlockTransfer.....	151
表 5-117	HAL_DMA_InitNodeCtrl.....	152
表 5-118	HAL_DMA_AddNextNode.....	153
表 5-119	HAL_DMA_ConfigListTransfer.....	153
表 5-120	HAL_DMA_ConfigScatterGatherTransfer.....	154
表 5-121	HAL_DMA_StartTransfer.....	155
表 5-122	HAL_DMA_AbortTransfer.....	155
表 5-123	HAL_DMA_RegisterIRQ.....	156
表 5-124	HAL_DMA_UnRegisterIRQ.....	157
表 5-125	HAL_DMA_EnableIRQ.....	157
表 5-126	HAL_DMA_DisableIRQ.....	158
表 5-127	HAL_DMA_DisableIRQ.....	158
表 5-128	QER Type.....	162
表 5-129	HAL_FLASH_Init.....	163
表 5-130	HAL_FLASH_DeInit.....	164

表 5-131	HAL_FLASH_Open.....	164
表 5-132	HAL_FLASH_Close.....	165
表 5-133	HAL_FLASH_Read.....	165
表 5-134	HAL_FLASH_Write.....	166
表 5-135	HAL_FLASH_Erase.....	166
表 5-136	HAL_FLASH_ExtControl.....	167
表 5-137	HAL_FLASH_Suspend.....	168
表 5-138	HAL_FLASH_Resume.....	169
表 5-139	HAL_WDG_Init.....	170
表 5-140	HAL_WDG_Deinit.....	170
表 5-141	HAL_WDG_Open.....	171
表 5-142	HAL_WDG_Close.....	172
表 5-143	HAL_WDG_Feed.....	172

1 概述

RS6 SDK 是正和微芯 RS6 系列芯片所配套的软件开发套件，用于 RS6 系列芯片的产品开发。

本文档基于本 SDK 针对项目开发几乎必须经历的多个开发阶段如新建工程、移植模块、增加外设等进行详细说明，帮助开发者更快速地认识本 SDK，以便于开发者能更容易更高效地完成项目开发。

2 SDK 简介

正和微芯 RS6130 和 RS6240 雷达传感器芯片是本公司的 60GHz 系列超低功耗及高性能毫米波雷达传感器芯片，搭载了 32 位 RISC-V-CPU 以及丰富的外设资源，支持 XIP，是低功耗高性能感知智能应用的理想解决方案。

此系列 SDK 采用开源的实时嵌入式操作系统 freeRTOS，以提供更轻量、更灵活、更完整的应用服务。

2.1 架构简介

2.1.1 RS6130 硬件架构

RS6130 支持 1T3R 雷达和 2.4GHz 无线传输（BLE 和 802.15.4），搭载支持浮点 RISC-V CPU，最高运行速度 128MHz，支持 256KB system RAM 和 128KB Data Cube，支持 SPI FLASH XIP，支持 UART/SPI/I2C 等常用外设。

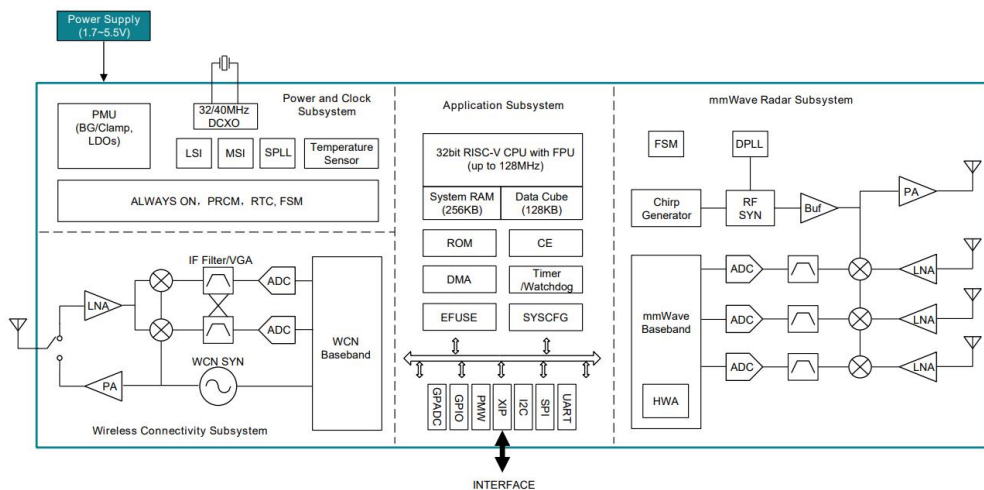


图 2-1 RS6130 系统框图

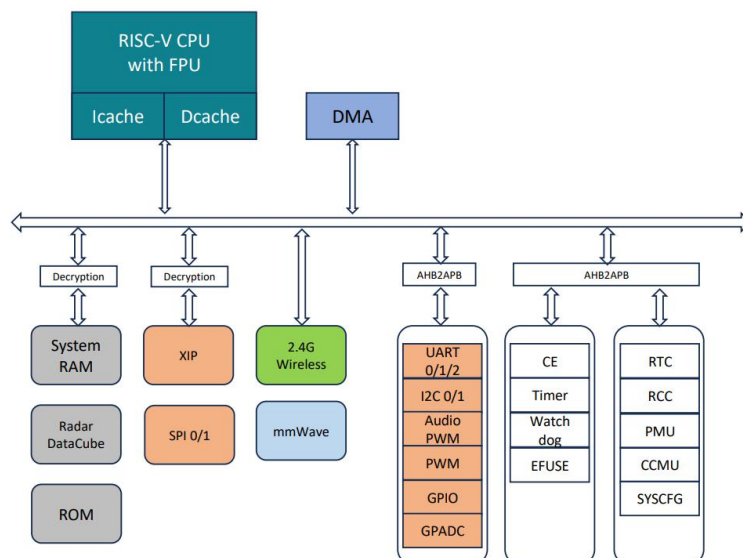


图 2-2 RS6130 应用子系统

2.1.2 RS6240 硬件架构

RS6240 支持 2T4R 雷达和 2.4GHz 无线传输（BLE 和 802.15.4），搭载双 RISC-V CPU，分别支持浮点和浮点加 DSP 指令集，最高运行速度 128MHz 和 256MHz，支持 295KB system RAM 和 256KB Data Cube，支持 SPI FLASH/PSRAM XIP 和数据存储，支持 UART/SPI/I2C 等常用外设。

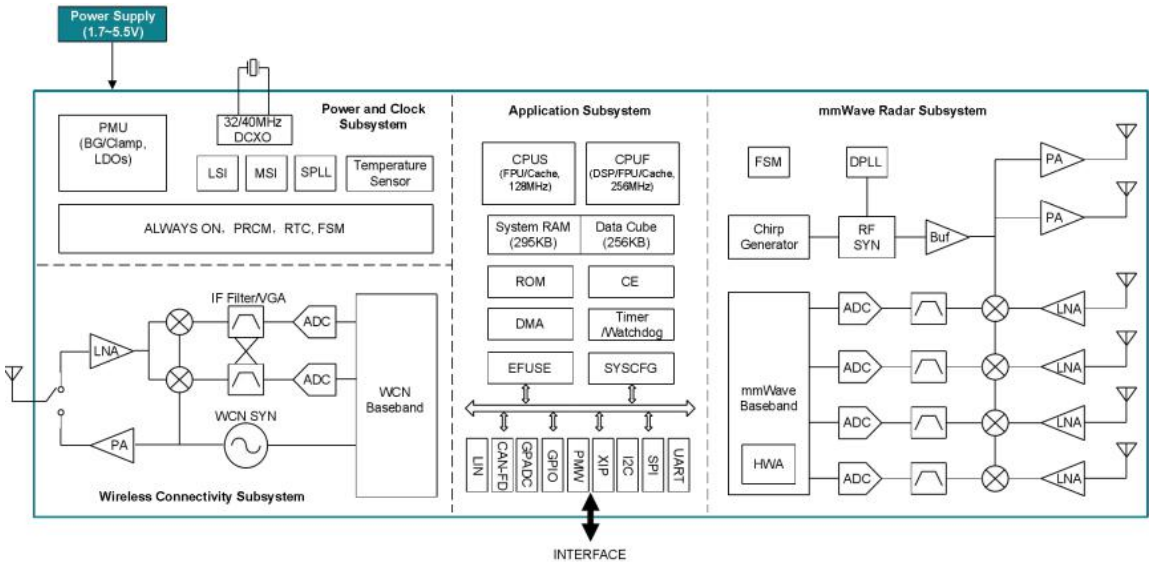


图 2-3 RS6240 系统框图

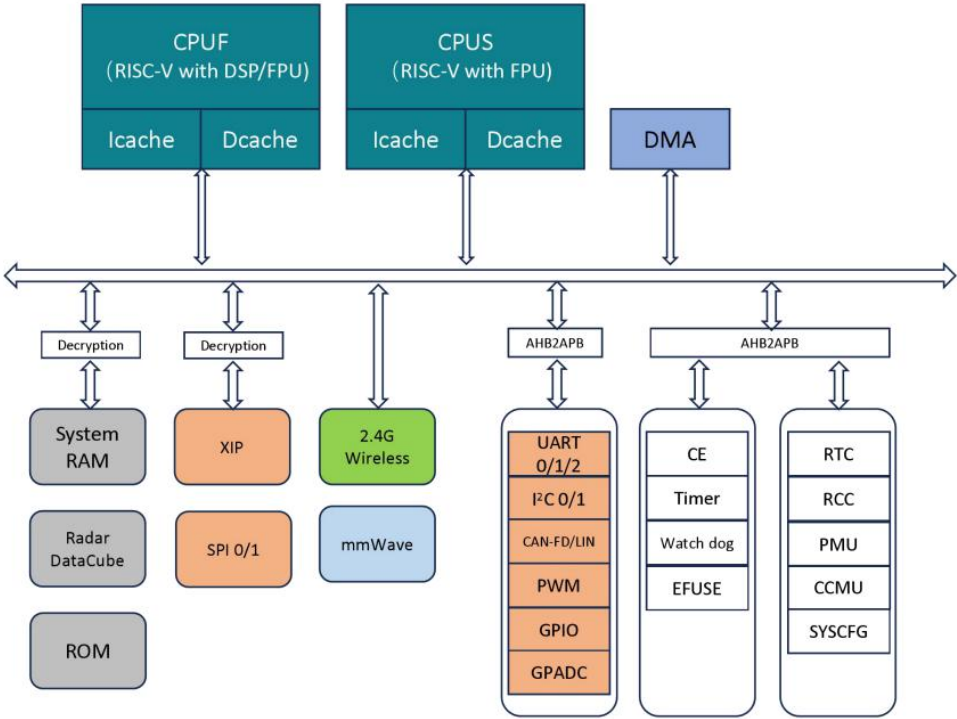


图 2-4 RS6240 应用子系统

2.1.3 SDK 软件架构

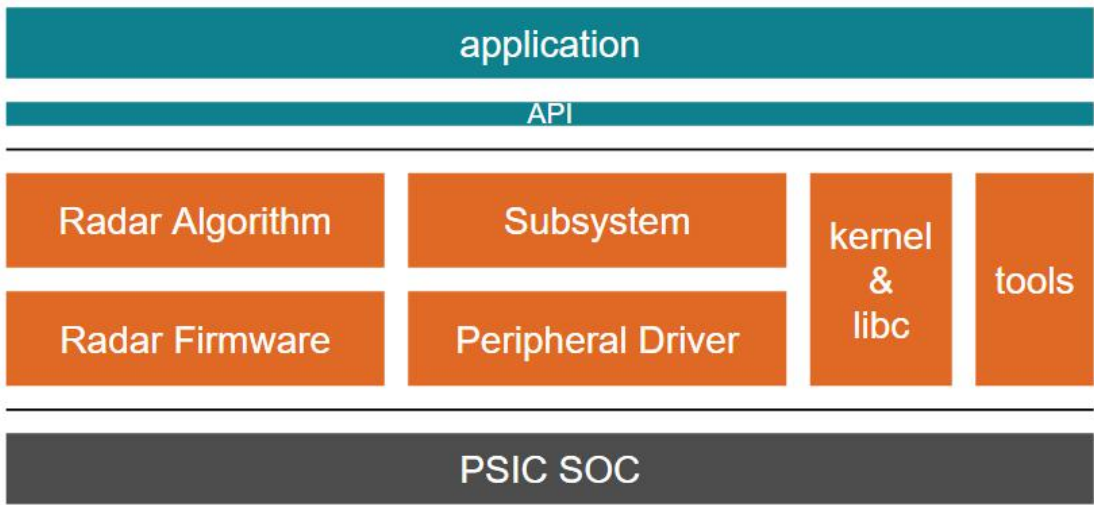


图 2-5 RS 系列 SDK 框架图

SDK 基于 PSIC 的 SOC 硬件平台，主要分 3 层：

- 1. 驱动层：包括雷达控制器驱动和各种外设驱动。雷达控制器驱动以独立二进制文件提供，外设驱动以源码提供。当前版本暂不支持无线通信驱动。
- 2. 中间件：中间件为应用提供组件，包括各种雷达算法，文件系统，测试框架，OTA 子系统等。
- 3. 应用层：基于中间件 API 层实现的各种雷达应用及 SOC 基础 sample。

在应用层下还有贯穿整个 SDK 的操作系统，C 库，以及各种编译辅助脚本。

2.2 目录结构

SDK 包括编译工具和源码两部分。

2.2.1 编译工具

编译由集成工具 CDK 实现。CDK 具有代码编辑，编译构建（编译，链接），调试分析等功能。

CDK 使用前需要进行安装，使用教程参考 [《RS6xxx_毫米波雷达开发_快速入门》](#)。

注：1.当前不支持通过 J-Link 在线调试。

2.2.2 软件源码

SDK 源码包括 C 代码和脚本，目录结构如下：

```
<home>/Software_Kit/02_SDK/
psdf_sdk                      /* 软件代码 */
|__ kernel                    /* RTOS 源码和 OS 抽象层 */
|   |__ freeRTOS              /* freeRTO 源码 */
|   |__ inc                   /* OS 抽象层 OSI 头文件 */
```

```

|   |__ src                                /* OS 抽象层 OSI 源文件 */
|
|__ libs                                  /* 存放 静态库(.a)，二进制文件(.img) */
|   |__ bootloader                        /* bootloader 二进制文件 */
|   |__ dsp                              /* dsp 静态库 */
|   |__ mmw                              /* radar control 二进制文件和符号表 */
|   |__ spiflash                          /* 外挂 spi flash/psram 芯片信息配置文件 */
|
|__ platform                             /* 芯片平台相关驱动和配置文件 */
|   |__ arch                             /* 芯片 CPU 架构相关文件，如：启动汇编，中断汇编处理 */
|   |__ boards                           /* 方案 Board 相关驱动和配置，如：pin-mux */
|   |__ drivers                           /* 芯片外设驱动，如：UART，GPIO */
|   |__ soc                              /* 芯片相关配置，如：地址段划分，中断编号 */
|   |__ utils                             /* 公共杂项，如：版本号，链接脚本，打包模板 */
|
|__ project                              /* 应用工程 */
|   |__ basic                            /* 基础工程，如：最小系统 */
|   |__ driver                           /* HAL 驱动，如：UART, FLASH */
|   |__ hello_word                       /* hello_word 工程 */
|   |__ mmwave                           /* radar 相关工程 */
|   |   |__ radar_analysis                /* 雷达分析工程，配合雷达分析工具使用 */
|   |   |
|   |__ subsys                           /* 子系统工程 */
|       |__ hif                          /* HIF 通信子系统 Demo 工程 */
|       |__ kvf                           /* Key-Value Flash 存储子系统 Demo 工程 */
|       |__ kvr                           /* Key-Value Ram 存储子系统 Demo 工程 */
|       |__ pm                           /* 电源管理子系统 Demo 工程 */
|       |__ shell                         /* shell 控制子系统 Demo 工程 */
|
|__ subsys                               /* 功能较独立的子系统 */
|   |__ cmd                              /* shell cmd 集合 */
|   |__ log                              /* 打印 LOG 输出 */
|   |__ dsp                              /* dsp 库 */
|   |__ hif                              /* HIF(Host Interface)通信协议 */
|   |__ kv                               /* Key-Value 存储 */
|   |   |__ kvr                           /* Key-Value Ram 存储，掉电丢失 */
|   |   |__ kvf                           /* Key-Value Flash 存储，掉电不丢失 */
|   |

```

```
| |__ libc          /* C 库适配 */
| |__ mmw          /* 雷达驱动和算法子系统 */
| |__ pm           /* PM 电源管理子系统 */
| |__ test         /* 测试工程 */
| |__ testsuite    /* 测试框架 */
| |__ shell        /* shell 子系统 */
|
|__ tools          /* 工具脚本 */
    |__ python     /* python 免安装包 */
    |__ scripts    /* 脚本，如：img 打包 */
```

2.3 参考资料

了解更多关于 FreeRTOS 的用法与规则，可以参考以下网址：

<https://www.freertos.org>

了解更多关于 CDK 工具的用法与规则，可以参考 CDK 工具的 Help：

打开 CDK 工具，在工具栏点击 Help，再选择“CDK help”，最后选择“xxx Doc”

也可以参考文档：

《RS6xxx_7x_毫米波雷达开发_快速入门》

3 新建工程

至此，开发者已经了解 SDK 架构和目录结构，本章节介绍如何新建一个 Project 以及一个 Board。
本节以在 project 目录下新建 “myproject” 为例，介绍怎么创建自己的应用工程。

3.1 工程目录结构

新建工程前，开发者需要对工程的目录结构有一定了解。示例工程文件夹路径如下：

```
<home>\MRS2xx_SDK\Software_Kit\02_SDK\project\basic\new_project"
```

打开该目录底下的 `project.cdkproj` 文件，编译成功后的工程目录结构如下：

```
new_project
|__ app                /* 工程源码目录 */
|  |__ inc             /* 工程 H 文件 */
|  |  |__ prj_config.h /* 工程配置文件 */
|  |__ src             /* 工程 C 文件 */
|  |  |__ main.c       /* 工程应用入口文件 */
|  |__ xxx             /* 可增加其他目录及文件 */
|
|__ cdk                /* CDK 配置和运行目录 */
|  |__ after_build.sh  /* 输出 img 打包引导脚本*/
|  |__ project.cdkproj /* CDK 配置文件 */
|  |__ Lst             /* 输出 img 打包引导脚本*/
|    |__ psdf.asm      /* 最终产生的汇编文件 */
|    |__ psdf.map      /* 最终产生的符号地址对文件 */
|  |__ Obj             /* 生成文件 */
|  |__ output          /* 输出 img 打包引导脚本*/
```

prj config 是工程配置文件，用于集中管理项目的硬件参数，软件功能开关，全局变量等配置信息。

3.2 工程创建

Step-A: 将 “project\basic\new_project” 文件夹复制到 “project” 目录下，并重命名为 “myproject”。

注：1.new_project 是一个空工程模版。

Step-B: 进入 “project\myproject\cdk” 目录，双击 “project.cdkproj” 文件打开 CDK;

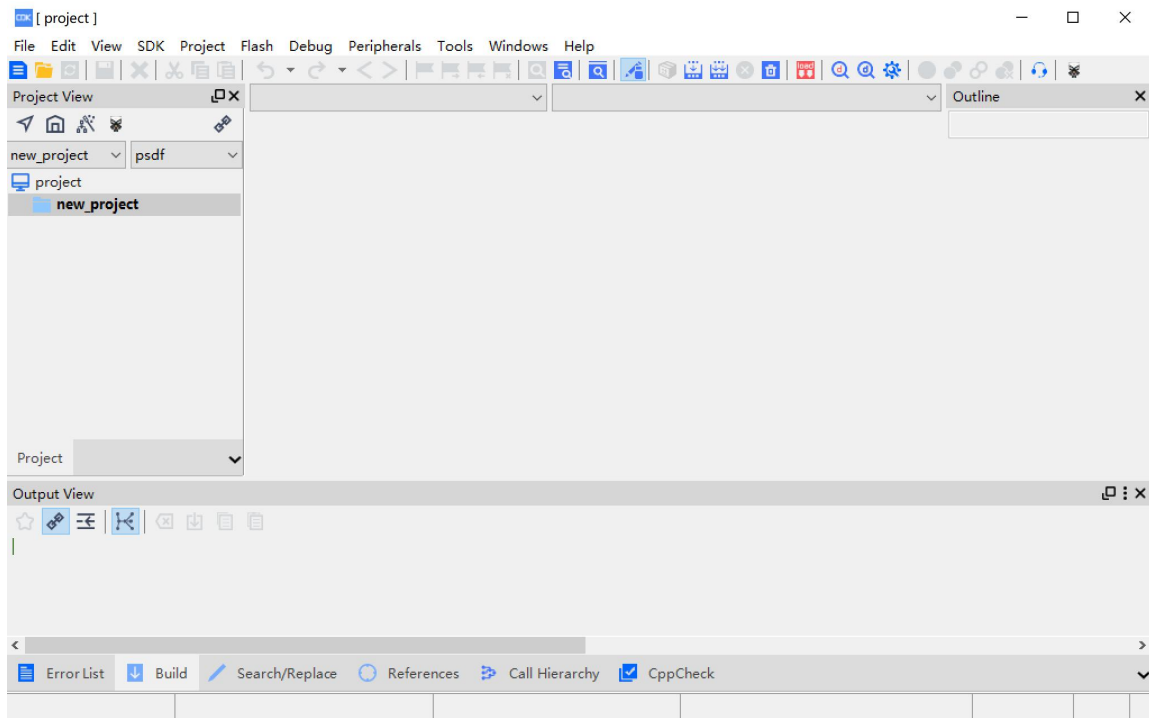


图 3- 1 new_project 界面

Step-C: 右键 “new_project” 下拉菜单中选择 “Rename Project”，弹出对话框输入 “myproject”，单击 “OK” ；

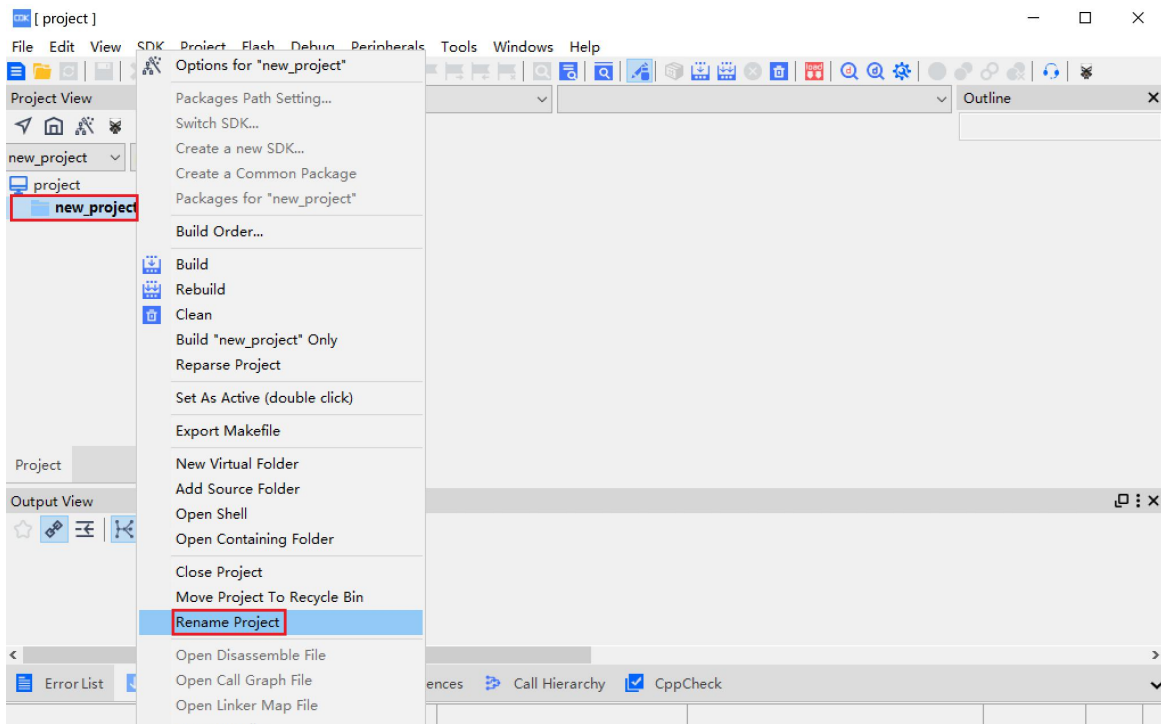


图 3- 2 修改工程名称菜单

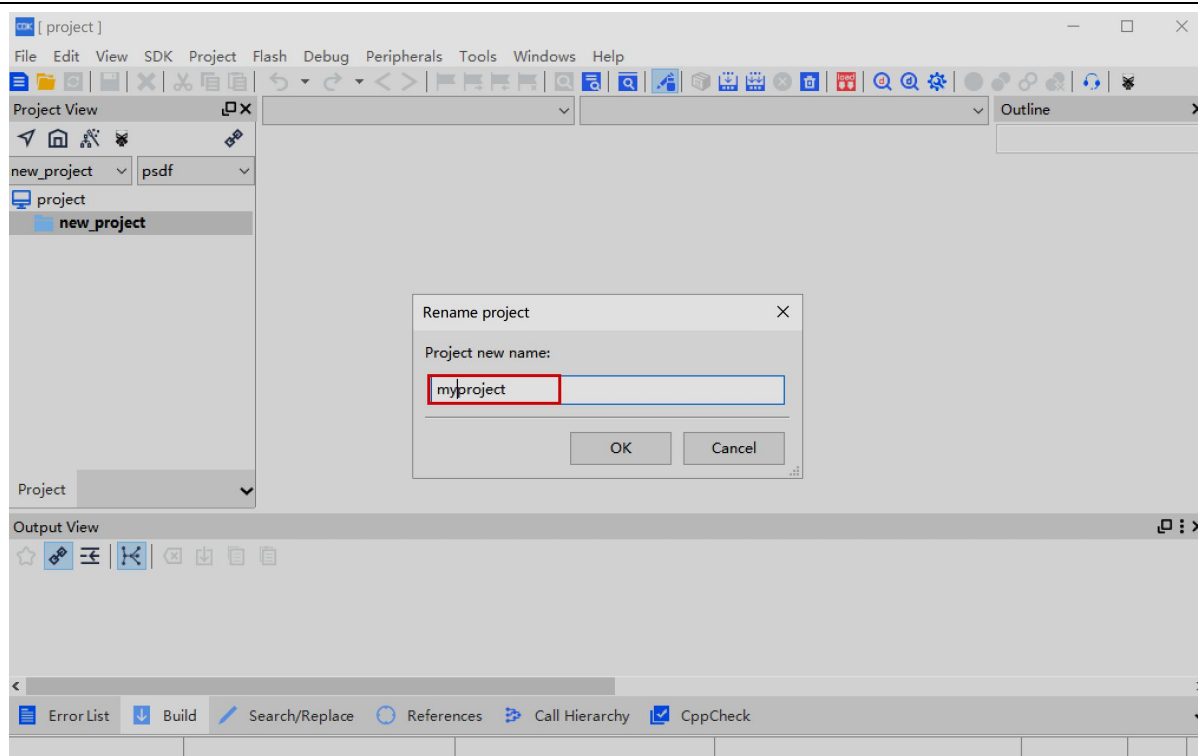


图 3-3 输入工程名称

3.2.1 添加源文件

在本工程中添加源文件和头文件路径。

Step-D-1: 右键“myproject”从弹出的下拉菜单中选择“Add Source Folder”；

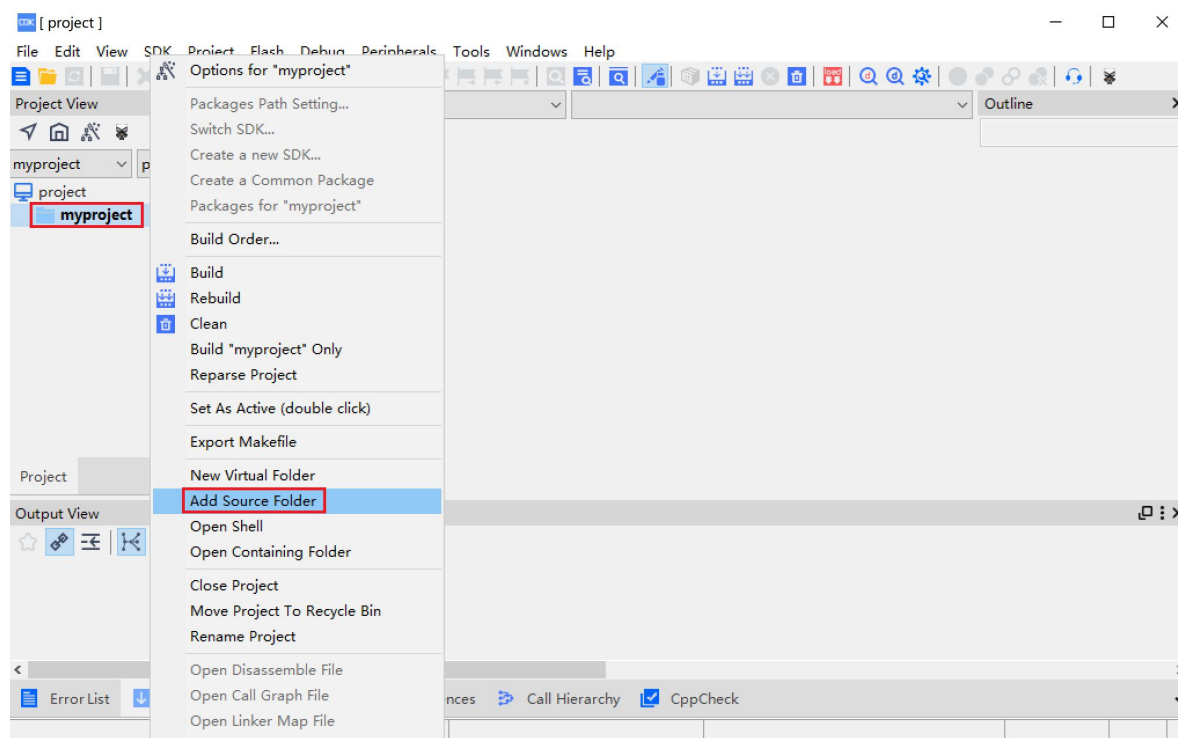


图 3-4 添加源文件目录

Step-D-2: 选择 SDK 中的“kernel”文件夹;

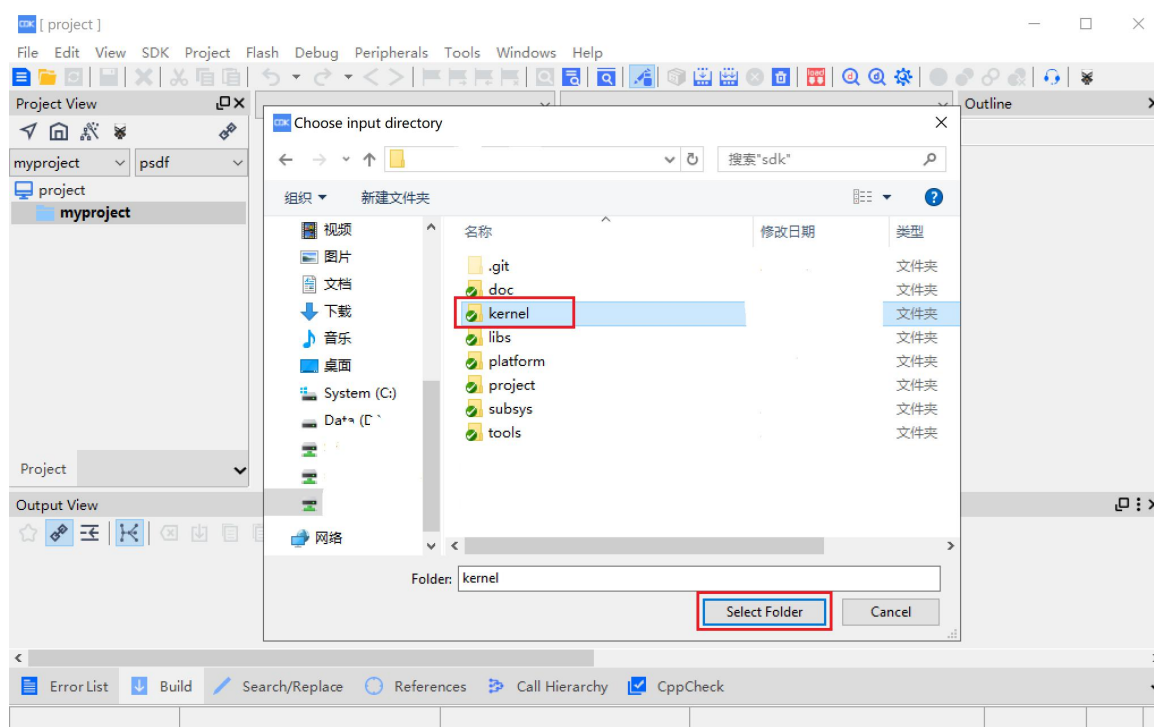


图 3-5 选择 kernel 文件夹

Step-D-3: 弹出的对话框选择“No”;

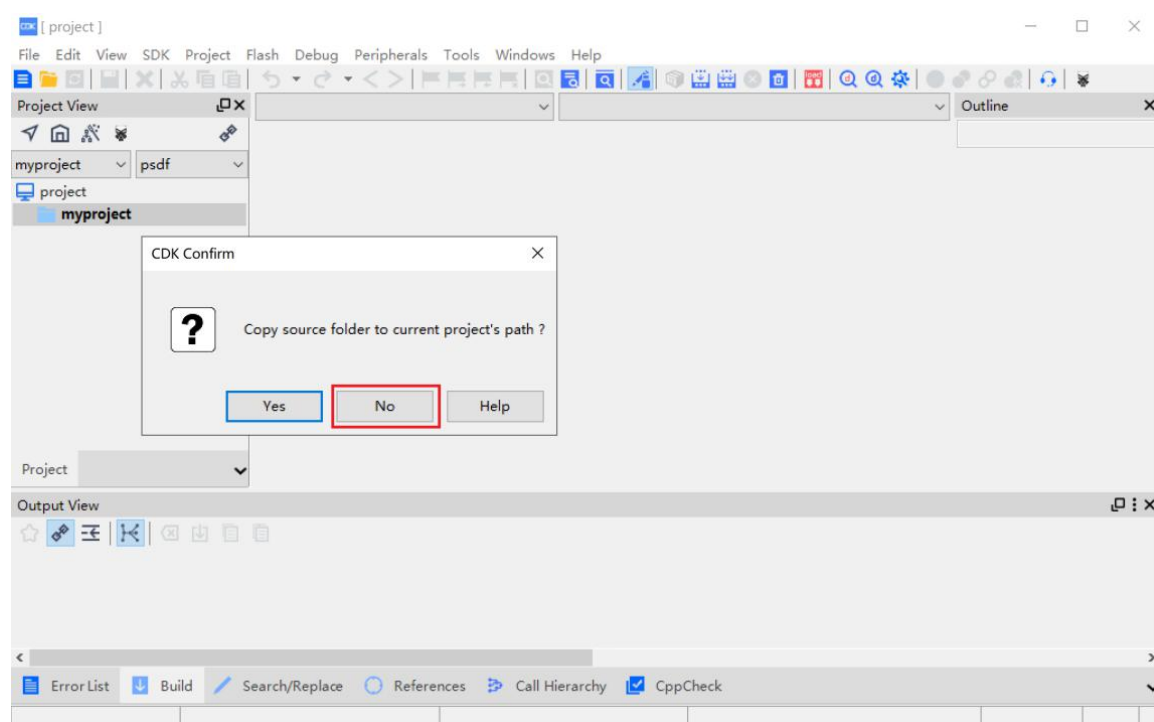


图 3-6 选择不复制文件夹方式

Step-D-4: 重复 Step-D-2 到 Step-D-3 的动作, 依次添加 “platform” 和 “subsys” 虚拟文件夹; 其中 “subsys” 文件夹可以屏蔽不需要的源码, 剔除不需要的源码或者文件夹编译, “kernel” 和 “platform” 文件夹则不需要屏蔽编译;

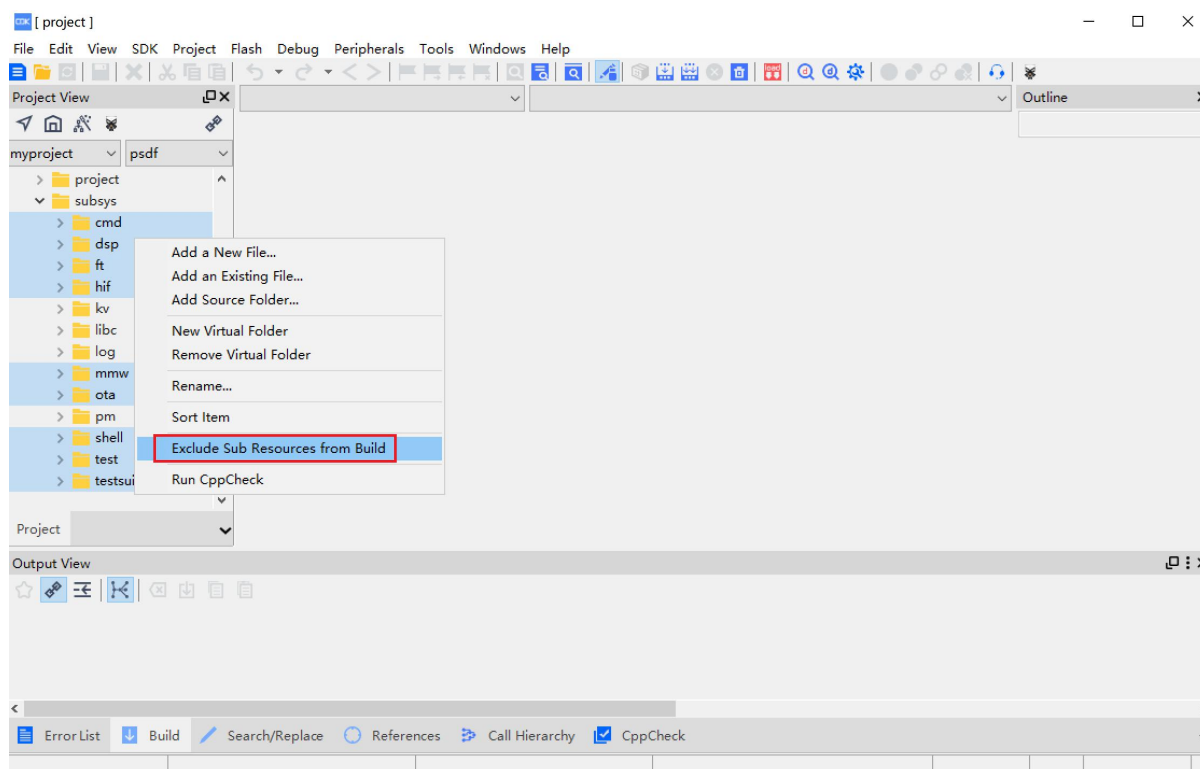


图 3-7 文件夹选择不编译

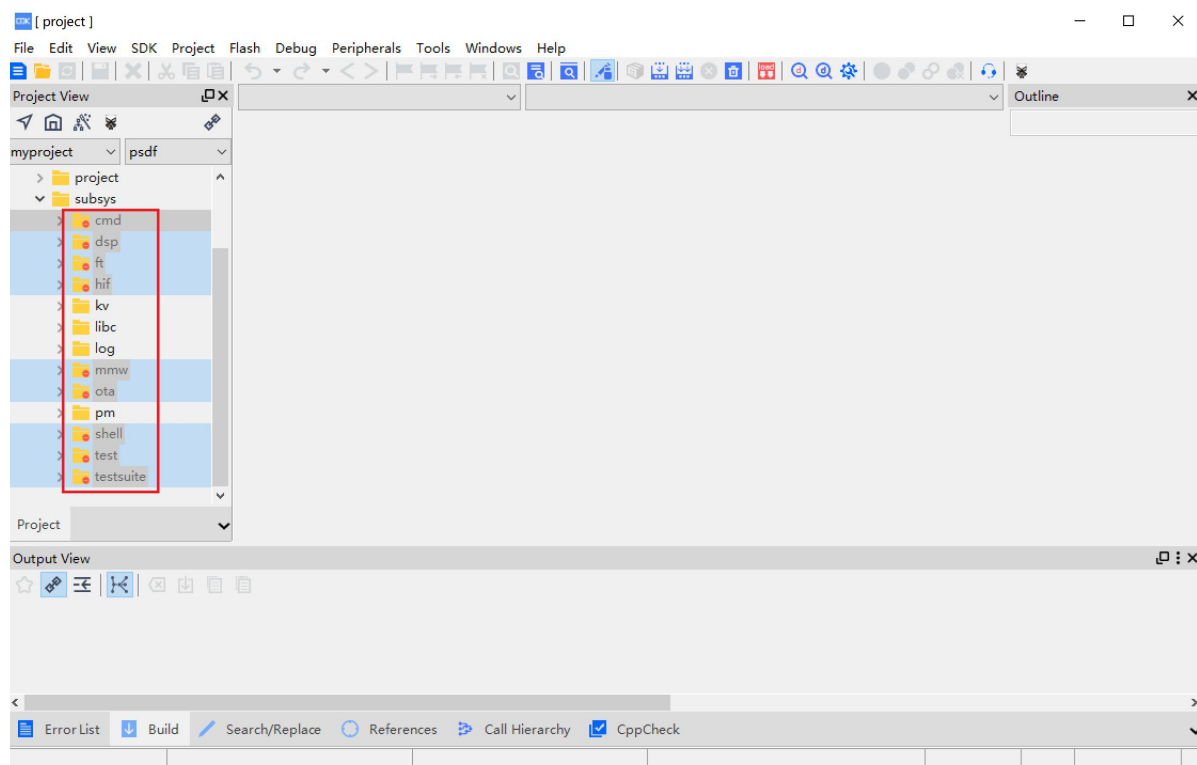


图 3-8 不编译的文件夹图标显示

Step-D-5: 添加工程应用文件；右键“myproject”弹出下拉框中选择“New Virtual Folder”并输入文件夹名称“project”；

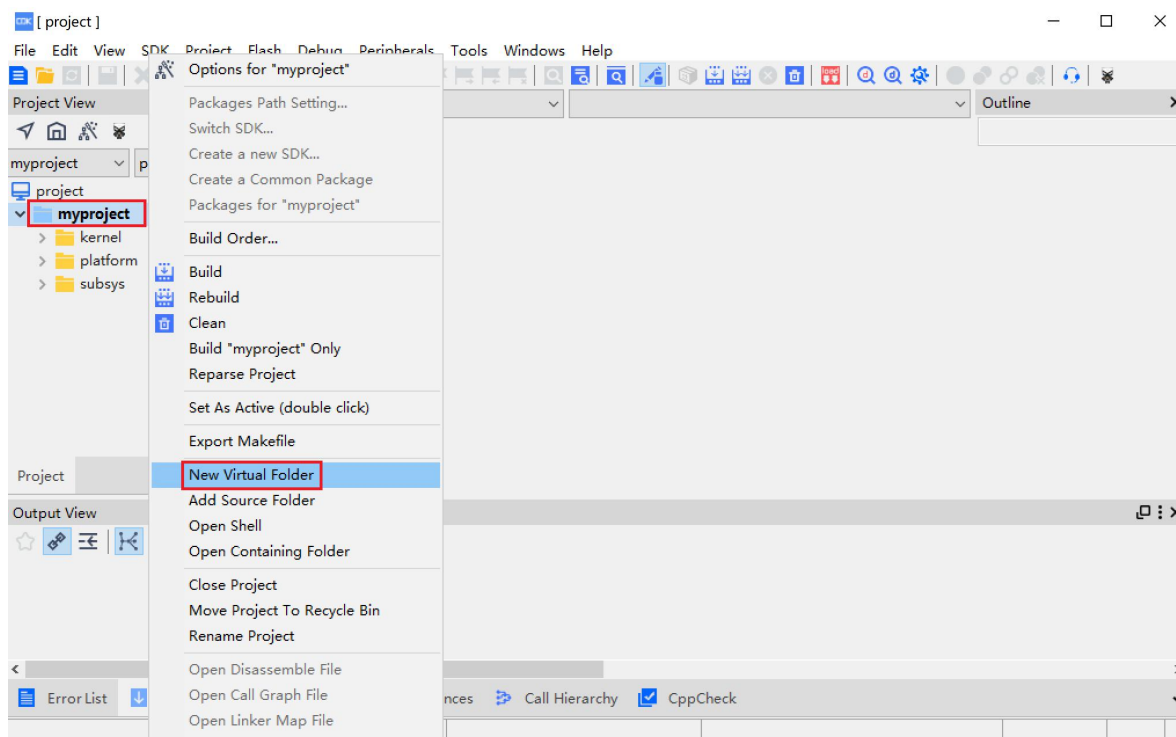


图 3-9 新建虚拟文件夹

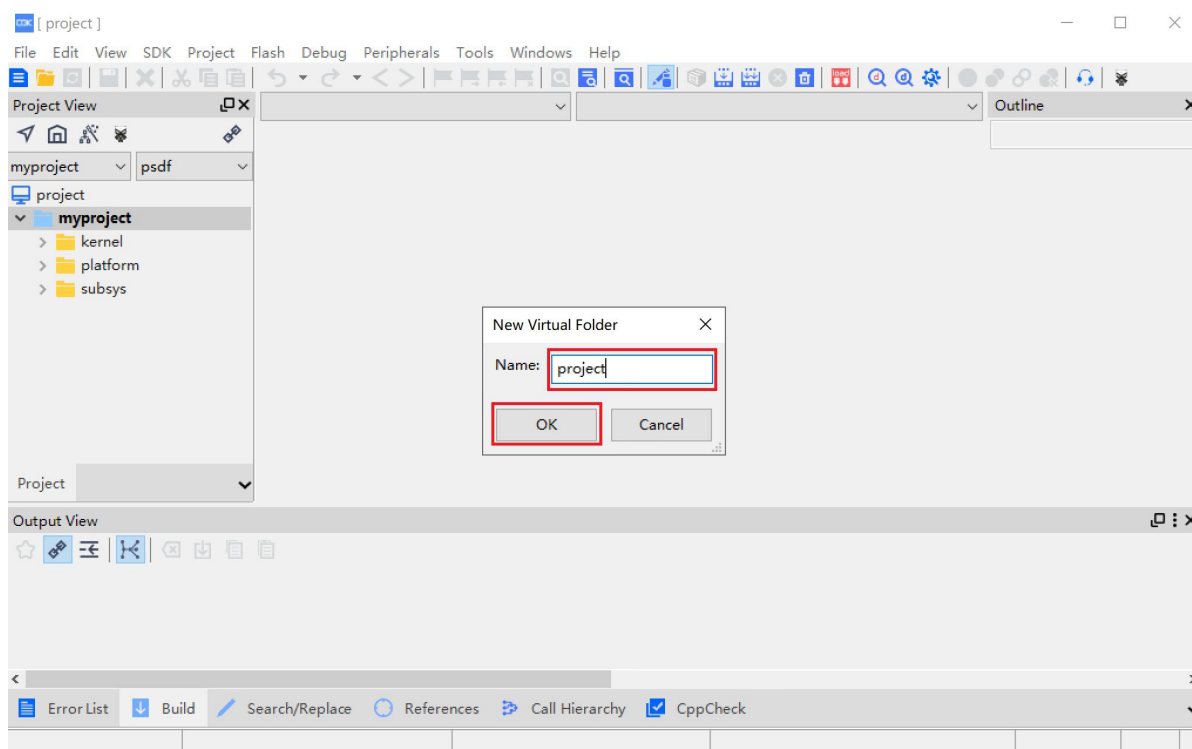


图 3-10 输入虚拟文件夹名称

Step-D-6: 右键“project”弹出下拉框中选择“Add Source Folder”并选择“app”文件夹；

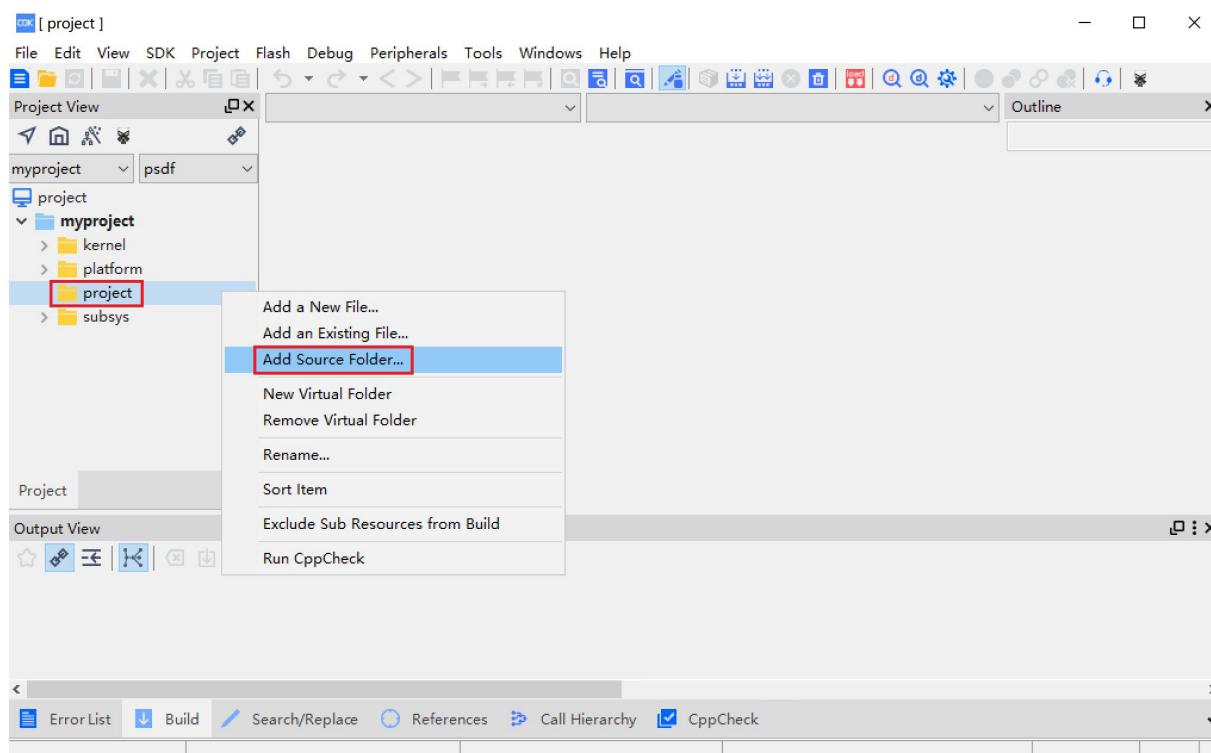


图 3-11 为虚拟文件夹添加源文件

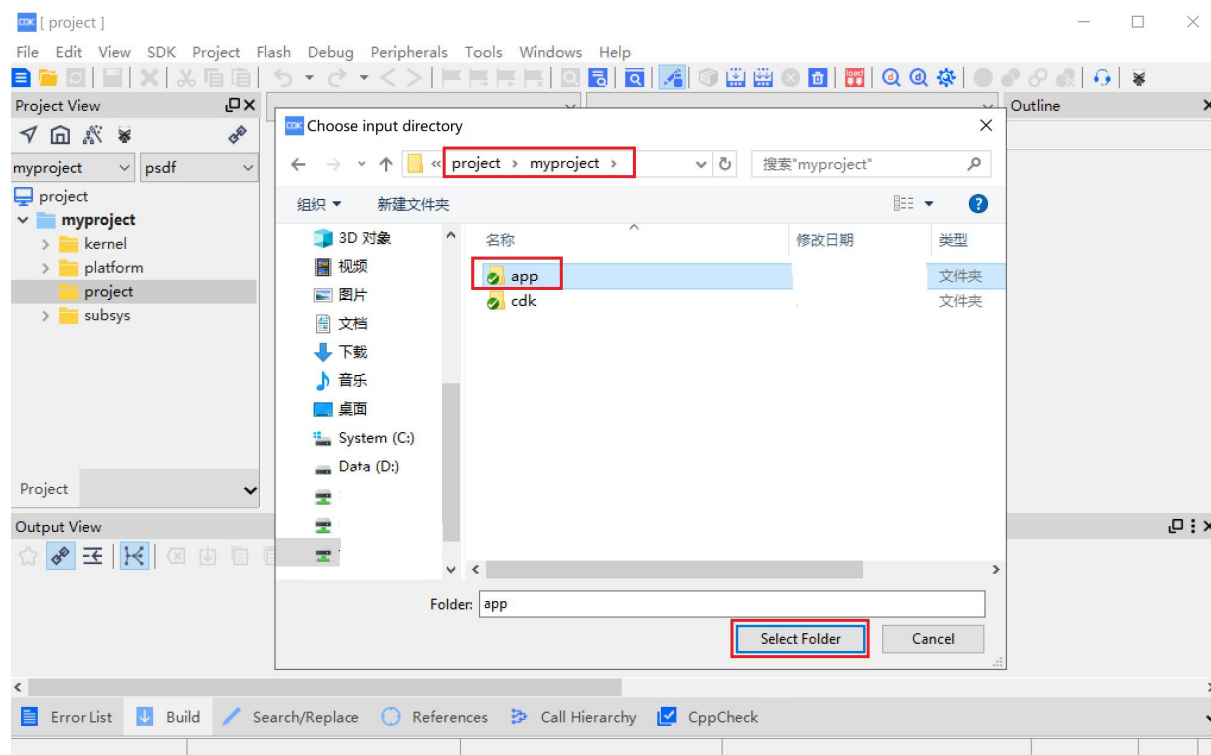


图 3-12 选择当前工程源文件

3.2.2 工程配置

Step-E: 修改工程配置。

Step-E-1: 右键“myproject”在下拉菜单中选择“Options for ‘myproject’”；project setting 是 CDK 环境提供的图形化配置界面，聚焦于编译构建过程和工程基础属性，包括芯片选择/工具链配置/链接基本配置等。

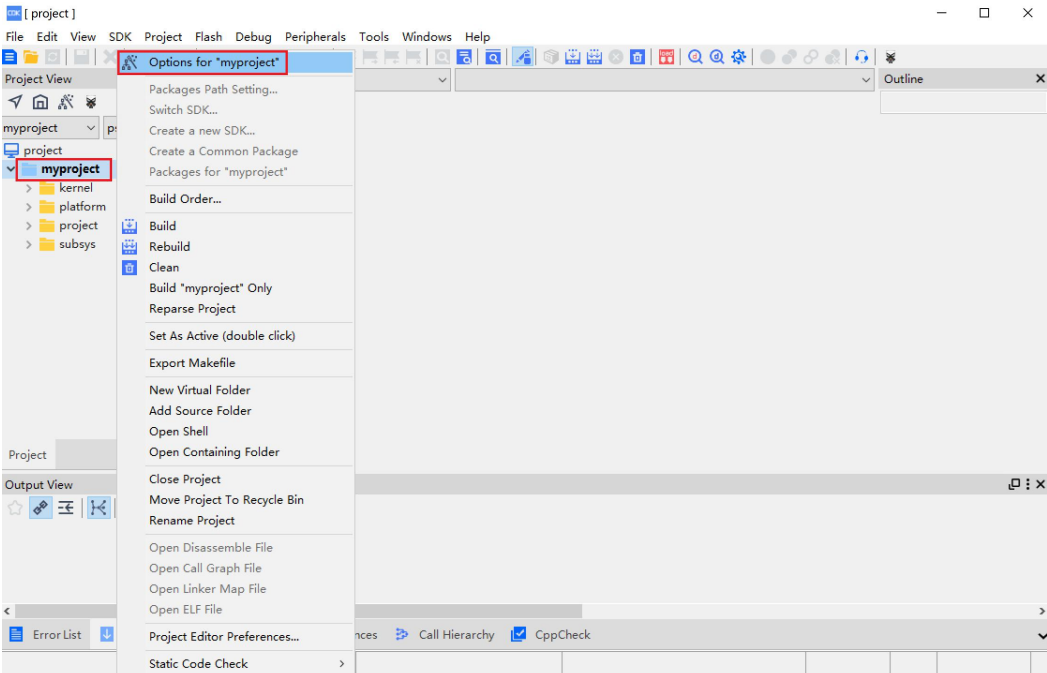


图 3-13 选择当前工程源文件

Step-E-2: Target 菜单中工具链按照下图配置，不能修改，选择“Use Stdlib”；

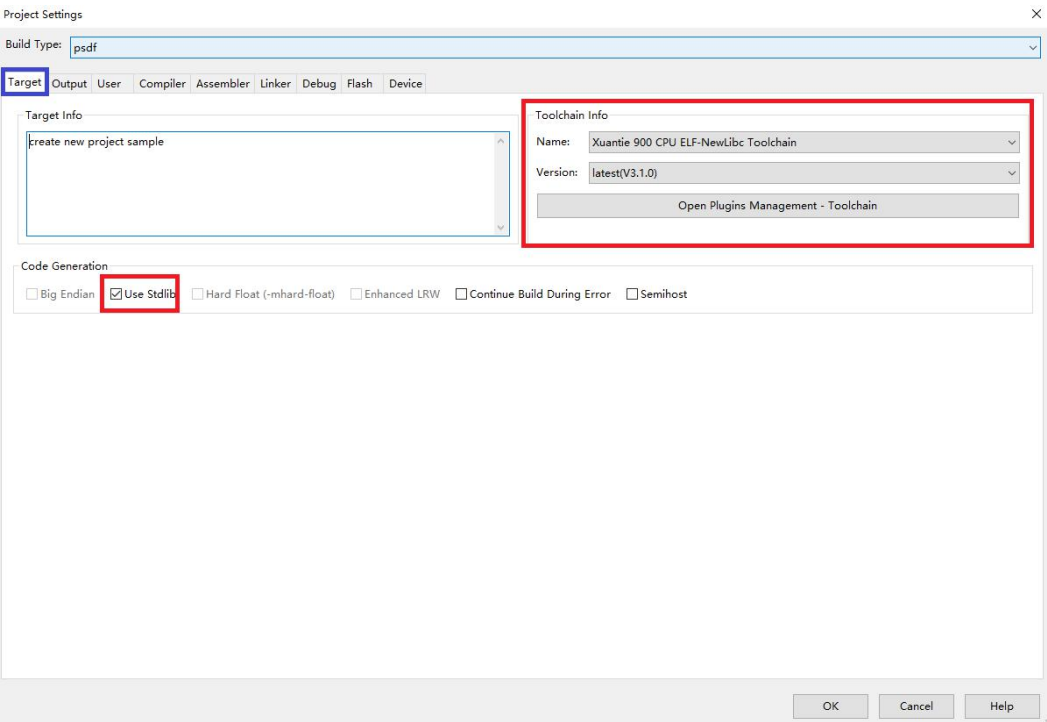


图 3-14 Target 配置界面

Step-E-3: Output 菜单中如下配置，输出名字“psdf”不能修改（否则 img 打包会异常），输出 elf 文件必须选中，asm 和 map 文件建议勾选方便 debug。

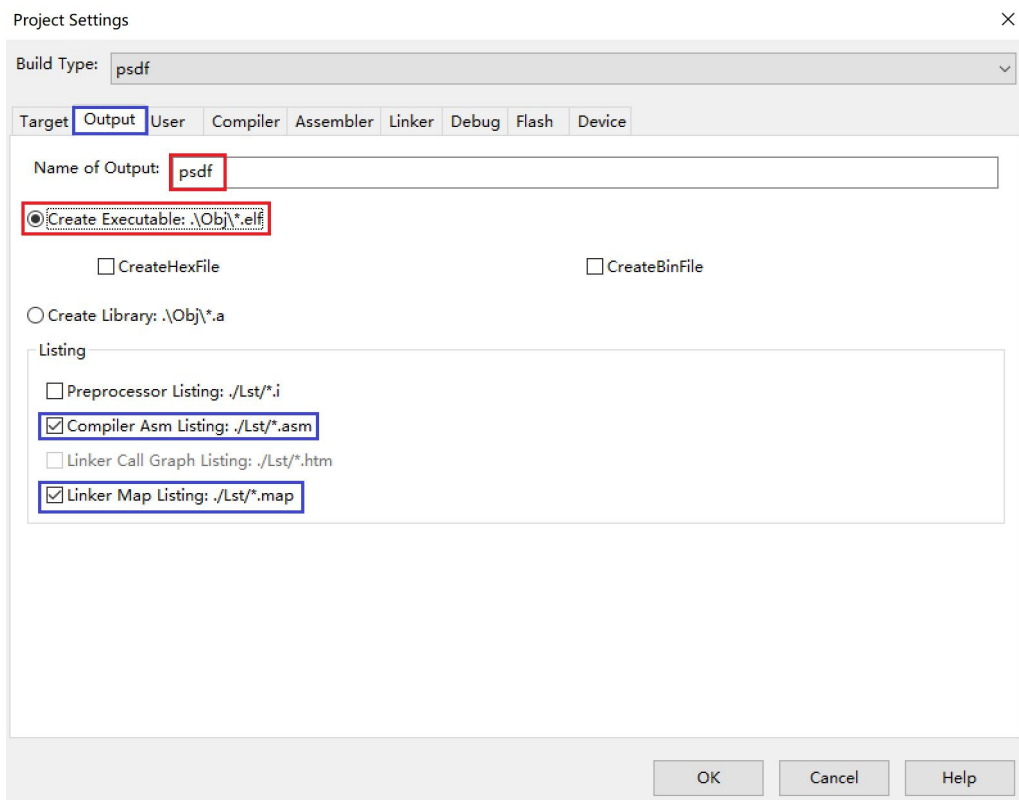


图 3-15 Output 配置界面

Step-E-4: User 菜单中如下配置，编译后脚本需勾选，用于启动 img 打包；

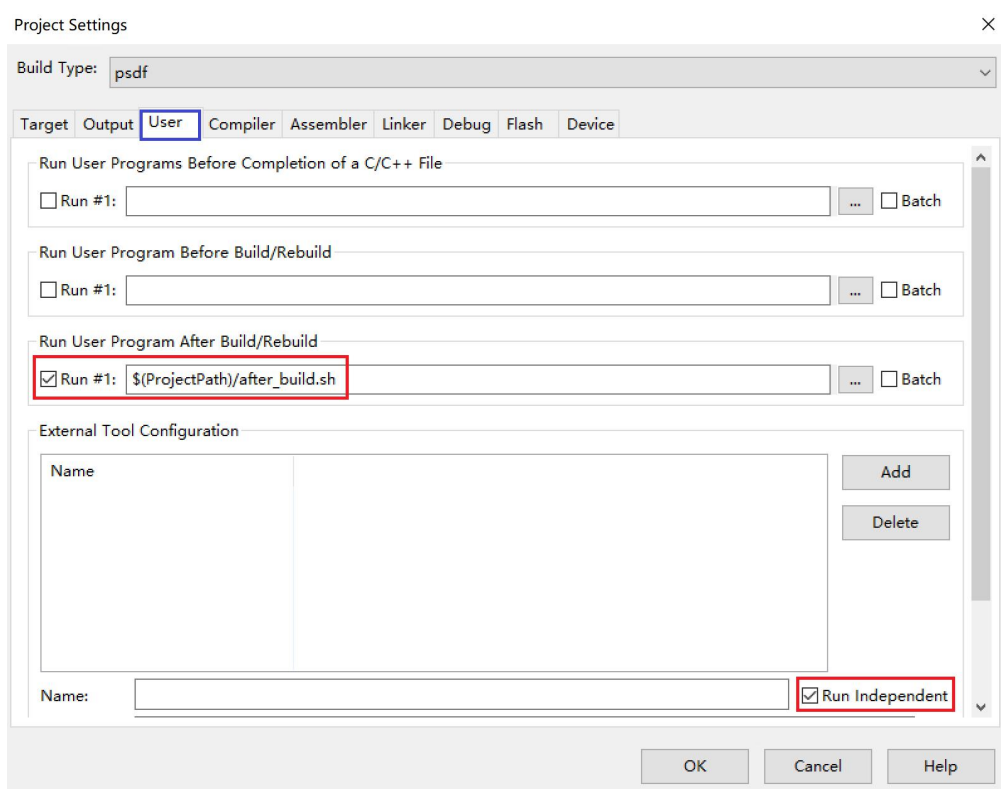


图 3-16 User 配置界面

Step-E-5：检查 Compiler 和 Assemble 菜单中如下配置，Flag 栏不能修改，头文件路径在前面添 Step-D 步骤时已自动添加；**Optimization 选择 -Os，选择-O3 需谨慎，可能会有异常。**

-MP -MMD -Wno-main -mcmodel=medlow -Wno-unused-function -Wpointer-arith -Wno-undef -Wall -ffunction-sections -fdata-sections -fno-inline-functions -fno-builtin -fno-strict-aliasing

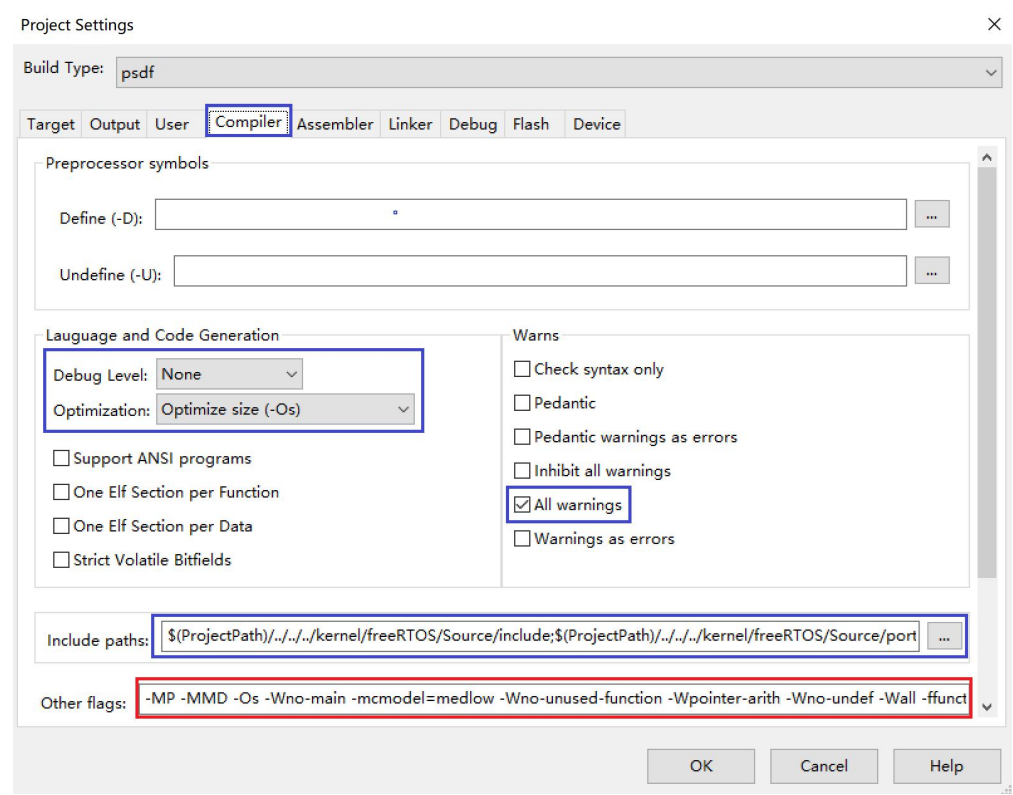


图 3-17 Compiler 配置界面

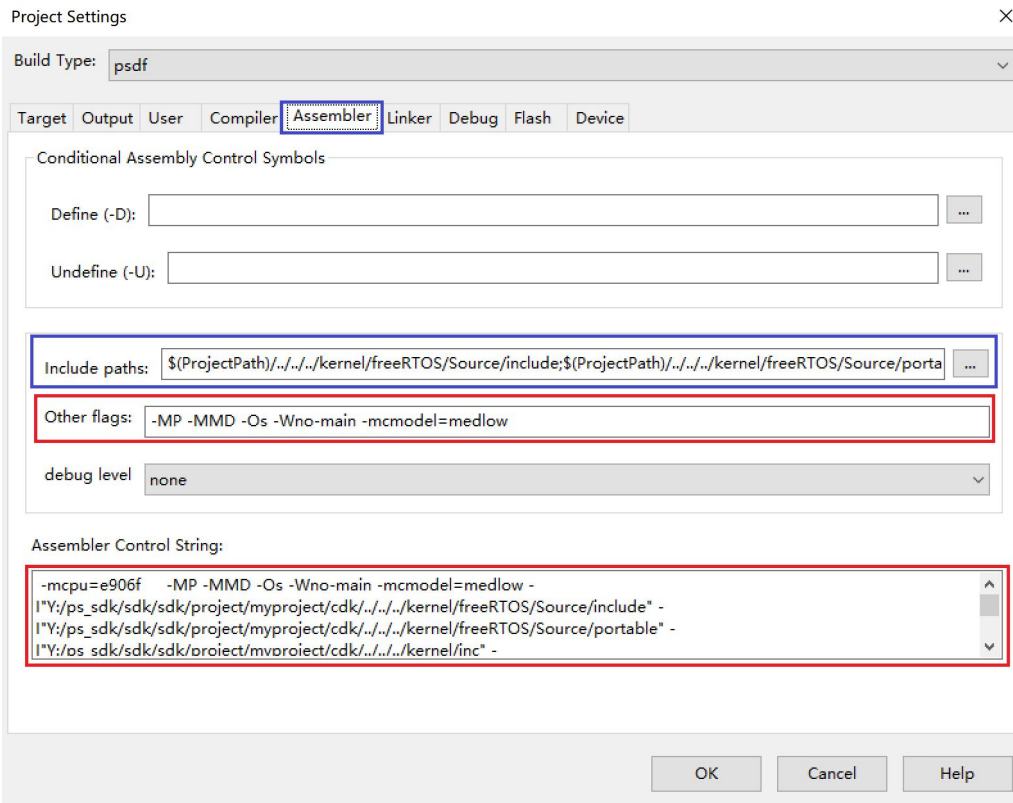


图 3-18 Assembler 配置界面

Step-E-7: Device 菜单中如下配置，若是 CPUS 选择“E906F”，若是 CPUF 选择“E906FP”；

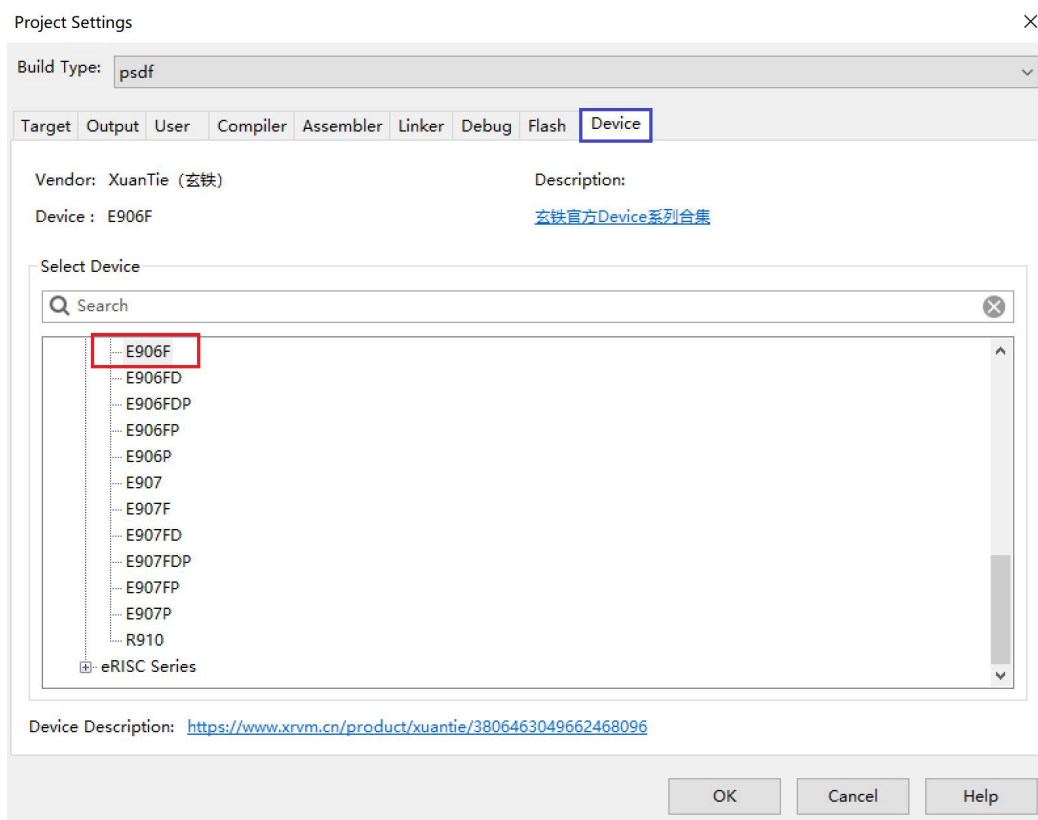


图 3-21 Device 配置界面

3.2.3 板级选择

Step-F: 在“Options for ‘myproject’”中选择对应的 board，分别在 Compiler 和 Assembler 页面下（见图 3-22）添加 board。可选择的 board 有：

CONFIG_BOARD_MRS6130_P1806

CONFIG_BOARD_MRS6130_P1812

CONFIG_BOARD_MRS6240_P2512_CPUS

CONFIG_BOARD_MRS6240_P2512_CPUF

然后在 main 函数（应用入口）中添加应用代码；

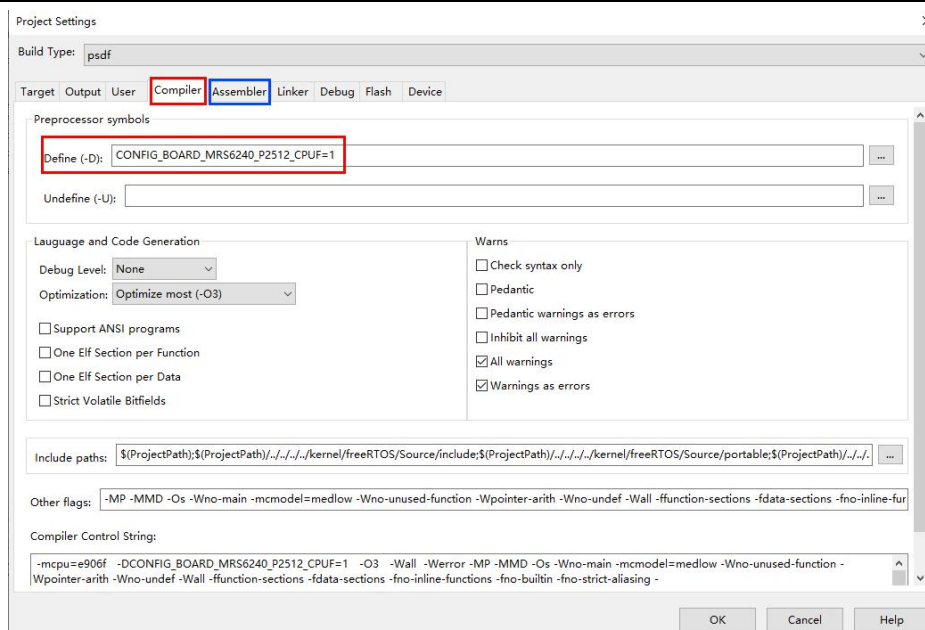


图 3-22 Options for myproject 选择 board

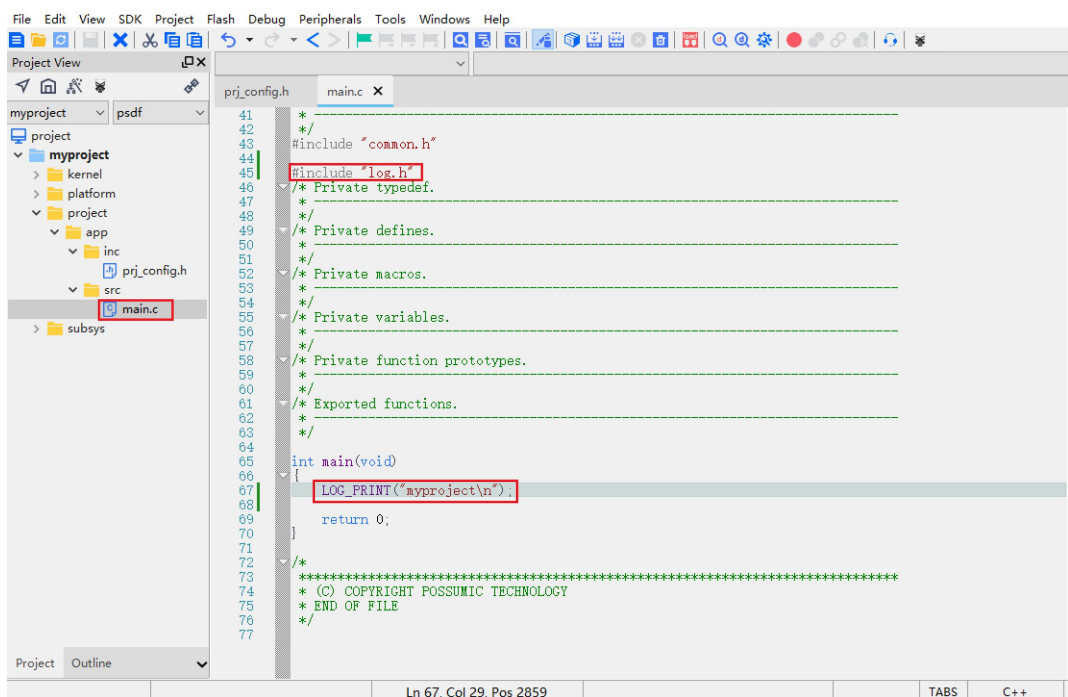


图 3-23 main 函数中添加应用

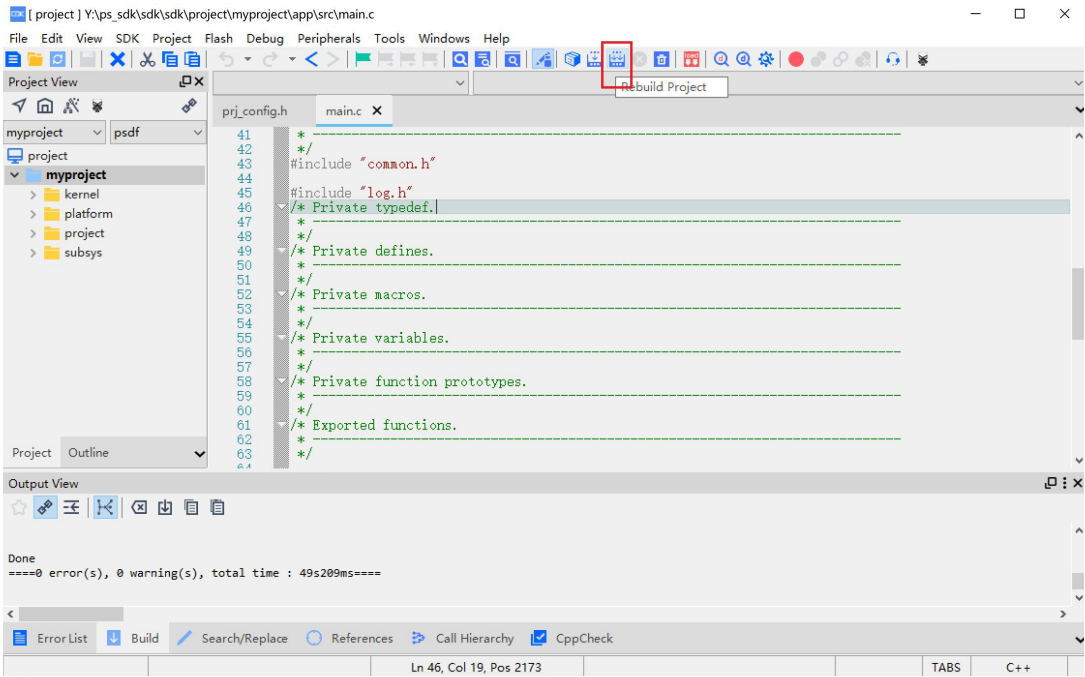


图 3-24 编译工程

3.3 新建开发模板

在项目实施过程中，可能需根据方案配置 pinmux，运行频率及各子系统。以下假设新建一个名为“myboard”的 board，soc 分别选择 rs6130 和 rs6240。

3.3.1 基于 RS6130 新建

Step A： 增加“myboard”文件夹及内部文件。

Step A-1： 进入“platform/boards”目录，复制“mrs6130_p1812”文件夹并重命名为“myboard”。

Step A-2： 进入“myboard”目录修改“mrs6130_p1812.c”和“mrs6130_p1812_config.h”为“myboard.c”和“myboard_config.h”。

Step A-3： 打开“myboard_config.h”，修改以下必须修改项。

```
#ifndef _MYBOARD_CONFIG_H    /* 头文件编译宏修改 */
#define _MYBOARD_CONFIG_H

#if CONFIG_BOARD_MYBOARD    /* BOARD 宏开关 */

#define CONFIG_BOARD_STRING    "myboard"    /* BOARD 名称 */
#define CONFIG_SOC_RS6130    1    /* 选择的 SOC */

#endif /* CONFIG_BOARD_MYBOARD */

#endif /* _BOARD_CONFIG_H */
```

Step A-4: 打开 “myboard.c” ，修改以下必须修改项。

```
#if (CONFIG_BOARD_MYBOARD)

#define LOG_MODULE                "myboard"
#define LOG_LEVEL                 CONFIG_BOARD_LOG_LEVEL
#include "log.h"

#endif /* CONFIG_BOARD_MYBOARD */
```

Step B: 修改 platform/boards/board_config.h。如下着色部分。

```
#ifndef CONFIG_BOARD_MRS6242_P2512_CPUF
#define CONFIG_BOARD_MRS6242_P2512_CPUF 0
#endif

#ifndef CONFIG_BOARD_MYBOARD
#define CONFIG_BOARD_MYBOARD 0
#endif

#if CONFIG_BOARD_MRS6130_P1806
#include "mrs6130_p1806/mrs6130_p1806_config.h"
#elif CONFIG_BOARD_MRS6130_P1812
#include "mrs6130_p1812/mrs6130_p1812_config.h"
#elif (CONFIG_BOARD_MRS6240_P2512_CPUS || CONFIG_BOARD_MRS6240_P2512_CPUF)
#include "mrs6240_p2512/mrs6240_p2512_config.h"
#elif ((CONFIG_BOARD_MRS6242_P2512_CPUS) || (CONFIG_BOARD_MRS6242_P2512_CPUF))
#include "mrs6242_p2512/mrs6242_p2512_config.h"
#elif (CONFIG_BOARD_MYBOARD == 1)
#include "myboard/myboard_config.h"
#else
#error "No board has been selected !!!"
#endif
```

Step C: 将 board 加入工程。

Step C-1: 打开一个基础工程，例如 hello word，将 “myboard” 加入 CDK。

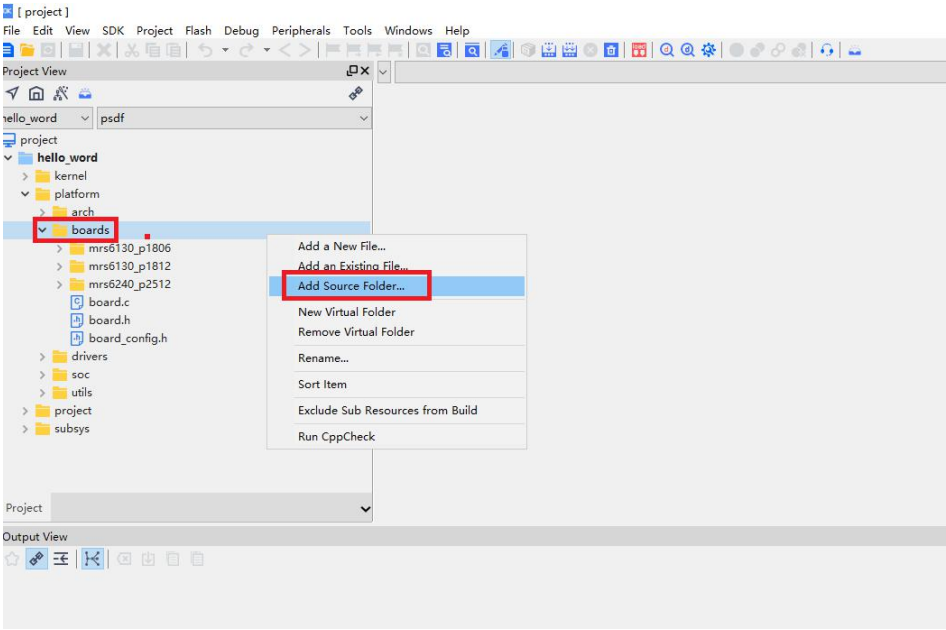


图 3-25 CDK 加入新建 board

加入完成后如下图，编译验证。

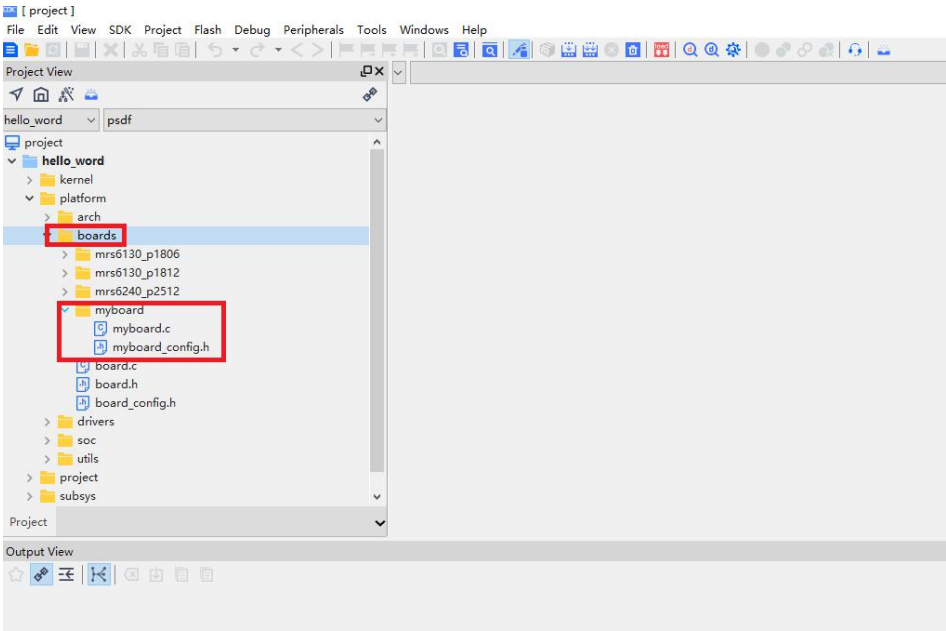


图 3-26 CDK 添加新 board 完成

Step C-2: 在目标工程按照 Step C-1 添加新的 board 到 CDK。

3.3.2 基于 RS6240 新建

若基于 RS6240 新建 board 与 6130 基本类似，但需注意有 CPUS 和 CPUF 两个配置。

在 myboard_config.h 需注意以下区分。

```
#ifndef _MYBOARD_CONFIG_H
#define _MYBOARD_CONFIG_H
```

```

#if (CONFIG_BOARD_MYBOARD_CPUS || CONFIG_BOARD_MYBOARD_CPUF)

#define CONFIG_BOARD_STRING                "myboard"

#if CONFIG_BOARD_MYBOARD_CPUS
#define CONFIG_SOC_RS6240_CPUS            1
#endif

#if CONFIG_BOARD_MYBOARD_CPUF
#define CONFIG_SOC_RS6240_CPUF            1
#endif

#endif /* CONFIG_BOARD_MYBOARD_CPUS || CONFIG_BOARD_MYBOARD_CPUF */

#endif /* _MYBOARD_CONFIG_H */

```

Myboard.c 需注意如下修改

```

#if (CONFIG_BOARD_MYBOARD_CPUS || CONFIG_BOARD_MYBOARD_CPUF)

#define LOG_MODULE                        "myboard"
#define LOG_LEVEL                        CONFIG_BOARD_LOG_LEVEL
#include "log.h"

#endif /* CONFIG_BOARD_MYBOARD_CPUS || CONFIG_BOARD_MYBOARD_CPUF */

```

board_config.h 需注意如下修改。

```

#ifndef CONFIG_BOARD_MRS6242_P2512_CPUF
#define CONFIG_BOARD_MRS6242_P2512_CPUF    0
#endif

#ifndef CONFIG_BOARD_MYBOARD_CPUS
#define CONFIG_BOARD_MYBOARD_CPUS          0
#endif

#ifndef CONFIG_BOARD_MYBOARD_CPUF
#define CONFIG_BOARD_MYBOARD_CPUF          0
#endif

#if CONFIG_BOARD_MRS6130_P1806
#include "mrs6130_p1806/mrs6130_p1806_config.h"
#elif CONFIG_BOARD_MRS6130_P1812
#include "mrs6130_p1812/mrs6130_p1812_config.h"
#elif (CONFIG_BOARD_MRS6240_P2512_CPUS || CONFIG_BOARD_MRS6240_P2512_CPUF)

```

```
#include "mrs6240_p2512/mrs6240_p2512_config.h"
#elif ((CONFIG_BOARD_MRS6242_P2512_CPUS) || (CONFIG_BOARD_MRS6242_P2512_CPUF))
#include "mrs6242_p2512/mrs6242_p2512_config.h"
#elif ((CONFIG_BOARD_MYBOARD_CPUS) || (CONFIG_BOARD_MYBOARD_CPUF))
#include "myboard/myboard_config.h"
#else
#error "No board has been selected !!!"
#endif
```

3.3.3 配置项修改

以下是常用配置项修改方式。

3.3.3.1 时钟配置

Myboard.c 分别可选择 32MHz, 96MHz, 112MHz 和 128MHz

```
#if (CONFIG_BOARD_CLOCK_CFG == 32)
#include "rs6130_config_cpus_32mhz.h"
#elif (CONFIG_BOARD_CLOCK_CFG == 96)
#include "rs6130_config_cpus_96mhz.h"
#elif (CONFIG_BOARD_CLOCK_CFG == 112)
#include "rs6130_config_cpus_112mhz.h"
#elif (CONFIG_BOARD_CLOCK_CFG == 128)
#include "rs6130_config_cpus_128mhz.h"
#endif
```

配置宏 CONFIG_BOARD_CLOCK_CFG 在 myboard_config.h 中定义。

```
/* 32: low
 * 96: medimu
 * 112: high
 * 128: highest
 */
#ifndef CONFIG_BOARD_CLOCK_CFG
#define CONFIG_BOARD_CLOCK_CFG 128
#endif
```

3.3.3.2 Pinmux配置

在 myboard.c 中定义。

```
/* Private constants.
 *
 */
#if CONFIG_DRIVER_UART_0
__attribute__((weak))const HAL_PinmuxConfig_t pinmux_uart0[] = {
    { GPIO_PORT_B, GPIO_PIN_0, {GPIOB_P0_F2_UART0_TX, GPIO_DRIVING_LEVEL_1, GPIO_PUSH_PULL, GPIO_PULL_UP } }, /* TX */
    { GPIO_PORT_B, GPIO_PIN_1, {GPIOB_P1_F2_UART0_RX, GPIO_DRIVING_LEVEL_1, GPIO_OPEN_DRAIN, GPIO_PULL_UP } }, /* RX */
};
#endif

#if CONFIG_DRIVER_UART_1
__attribute__((weak))const HAL_PinmuxConfig_t pinmux_uart1[] = {
    { GPIO_PORT_A, GPIO_PIN_2, {GPIOA_P2_F3_UART1_TX, GPIO_DRIVING_LEVEL_1, GPIO_PUSH_PULL, GPIO_PULL_UP } }, /* TX */
    { GPIO_PORT_A, GPIO_PIN_1, {GPIOA_P1_F3_UART1_RX, GPIO_DRIVING_LEVEL_1, GPIO_OPEN_DRAIN, GPIO_PULL_UP } }, /* RX */
};
#endif

#if CONFIG_DRIVER_UART_2
__attribute__((weak))const HAL_PinmuxConfig_t pinmux_uart2[] = {
    { GPIO_PORT_A, GPIO_PIN_2, {GPIOA_P2_F5_UART2_TX, GPIO_DRIVING_LEVEL_1, GPIO_PUSH_PULL, GPIO_PULL_UP } }, /* TX */
    { GPIO_PORT_A, GPIO_PIN_3, {GPIOA_P3_F5_UART2_RX, GPIO_DRIVING_LEVEL_1, GPIO_OPEN_DRAIN, GPIO_PULL_UP } }, /* RX */
};
#endif

#if CONFIG_DRIVER_SPI
__attribute__((weak))const HAL_PinmuxConfig_t pinmux_spi0[] = {
    { GPIO_PORT_A, GPIO_PIN_0, {GPIOA_P0_F2_SPI_CS, GPIO_DRIVING_LEVEL_3, GPIO_PUSH_PULL, GPIO_PULL_UP } }, /* CS */
};
```

图 3-27 myboard pinmux 配置

myboard.c 中定义的 board_GetPinmux 函数会被其他驱动引用。

```

HAL_Status_t board_GetPinmux(
    HAL_DevType_t type,
    uint8_t idx,
    HAL_PinMuxType_t cfgType,
    HAL_Pinmux_t *pPinMux)
{
    HAL_Status_t status = HAL_STATUS_INVALID_STATE;

    switch (type) {
        case HAL_DEV_TYPE_UART:
            if (idx == UART0_ID) {
                #if CONFIG_DRIVER_UART_0
                if (cfgType == HAL_PINMUX_TYPE_INIT) {
                    pPinMux->pinConfig = (HAL_PinmuxConfig_t *)pinmux_uart0;
                    pPinMux->pinCnt = HAL_ARRAY_SIZE(pinmux_uart0);
                    status = HAL_STATUS_OK;
                }
                #endif
            } else if (idx == UART1_ID) {
                #if CONFIG_DRIVER_UART_1
                if (cfgType == HAL_PINMUX_TYPE_INIT) {
                    pPinMux->pinConfig = (HAL_PinmuxConfig_t *)pinmux_uart1;
                    pPinMux->pinCnt = HAL_ARRAY_SIZE(pinmux_uart1);
                    status = HAL_STATUS_OK;
                }
                #endif
            } else if (idx == UART2_ID) {
                #if CONFIG_DRIVER_UART_2
                if (cfgType == HAL_PINMUX_TYPE_INIT) {
                    pPinMux->pinConfig = (HAL_PinmuxConfig_t *)pinmux_uart2;
                    pPinMux->pinCnt = HAL_ARRAY_SIZE(pinmux_uart2);
                    status = HAL_STATUS_OK;
                }
                #endif
            }
    }
}

```

图 3-28 myboard pinmux 配置函数

3.3.3.3 FLASH/XIP运行频率

修改 SPI Flash 运行频率（若有需要，详情参考外设驱动-FLASH 章节）；

```

HAL_Status_t BOARD_InitXIP(void)
{
    HAL_Status_t status = HAL_STATUS_OK;
    HAL_Dev_t *pFlashDev = NULL;

    pFlashDev = HAL_FLASH_Init(FLASH_DEFT_ID);
    if (pFlashDev == NULL) {
        LOG_DBG("init fail");
        status = HAL_STATUS_ERROR;
    }

    uint32_t listMode = FLASH_LIST_TYPE_DEFAULT | FLASH_LIST_TYPE_LIST;
    status = HAL_FLASH_ExtControl(pFlashDev, FLASH_EXT_SET_CHIP_INFO_MODE, &listMode,
    sizeof(listMode));
    if (status != HAL_STATUS_OK) {
        LOG_DBG("set list mode fail");
        return status;
    }

    uint32_t freq = HAL_CLOCK_GetFreq(pBoardDev_Clock, CLOCK_SYS);
    FLASH_InitParam_t flashInitParam = {

```

```

/* 时钟源和频率在类似 rs6130_config_cpus_96mhz.h 模板文件中配置*/
.clockSrc      = CONFIG_XIP_CLK_SRC,
.clockFreq     = freq / CONFIG_XIP_CLK_DIV,
.spiFormat     = FLASH_SPI_FORMAT_QUAD,
.psramEnable   = false,
};

status = HAL_FLASH_Open(pFlashDev, &flashInitParam);
if (status == HAL_STATUS_OK) {
    listMode = 0;
    status = HAL_FLASH_ExtControl(pFlashDev, FLASH_EXT_GET_USED_CHIP_INFO_MODE,
&listMode, sizeof(listMode));
    if (status != HAL_STATUS_OK) {
        LOG_DBG("get used list mode fail");
        return status;
    }

    if (listMode == FLASH_LIST_TYPE_DEFAULT) {
        status = HAL_STATUS_BUSY;
    }
}

return status;
}

```

图 3-29 myboard spi flash 配置

3.3.3.4 其他配置

其他常用配置

```

/* 系统总堆栈配置 */
#ifndef CONFIG_HEAP_SIZE
#define CONFIG_HEAP_SIZE                (1024 * 40)
#endif

/* 是否开启打印 */
#ifndef CONFIG_PRINTF
#define CONFIG_PRINTF                    1
#endif

/* 若开启打印，打印串口号和波特率 */
#if CONFIG_PRINTF
#ifndef CONFIG_PRINTF_UART_NUM
#define CONFIG_PRINTF_UART_NUM          0
#endif

#ifndef CONFIG_PRINTF_UART_BAUDRATE
#define CONFIG_PRINTF_UART_BAUDRATE      115200
#endif

```

```
/* 是否开启 shell */
#ifndef CONFIG_SHELL
#define CONFIG_SHELL 0
#endif

/* 若开启 shell，串口号和波特率 */
#if CONFIG_SHELL
#ifndef CONFIG_SHELL_UART_NUM
#define CONFIG_SHELL_UART_NUM 0
#endif

#ifndef CONFIG_SHELL_UART_BAUDRATE
#define CONFIG_SHELL_UART_BAUDRATE 115200
#endif
```

4 子系统

4.1 CPU

RS6130 支持 CPU 规格如下。

表 4-1 RS6130 CPU 规格

	run freq max	cache size	Instruction Set
CPUS	Max: 128MHz Default: 96MHz	i_cache : 4KB d_cache: 2KB	IMACF 整型指令、乘除法指令、原子操作指令、压缩指令、单精度浮点指令

RS6240 支持 CPUF 和 CPUS 双核规格如下。

表 4-2 RS6240 CPU 规格

	run freq max	cache size	Instruction Set
CPUS	Max:128MHz Default: 112MHz	i_cache : 4KB d_cache: 2KB	IMACF 整型指令、乘除法指令、原子操作指令、压缩指令、单精度浮点指令
CPUF	Max:256MHz Default: 224MHz	i_cache : 8KB d_cache: 16KB	IMACFP 整型指令、乘除法指令、原子操作指令、压缩指令、单精度浮点指令、DSP 指令

使用注意：

- 1. 使用不同 CPU 时需注意 CDK 的配置，以选择正确的指令集。

CDK 配置路径 Project Setting --> Device 页，当使用 CPUS 时选择 E906F，当使用 CPUF 时选择 E906FP。

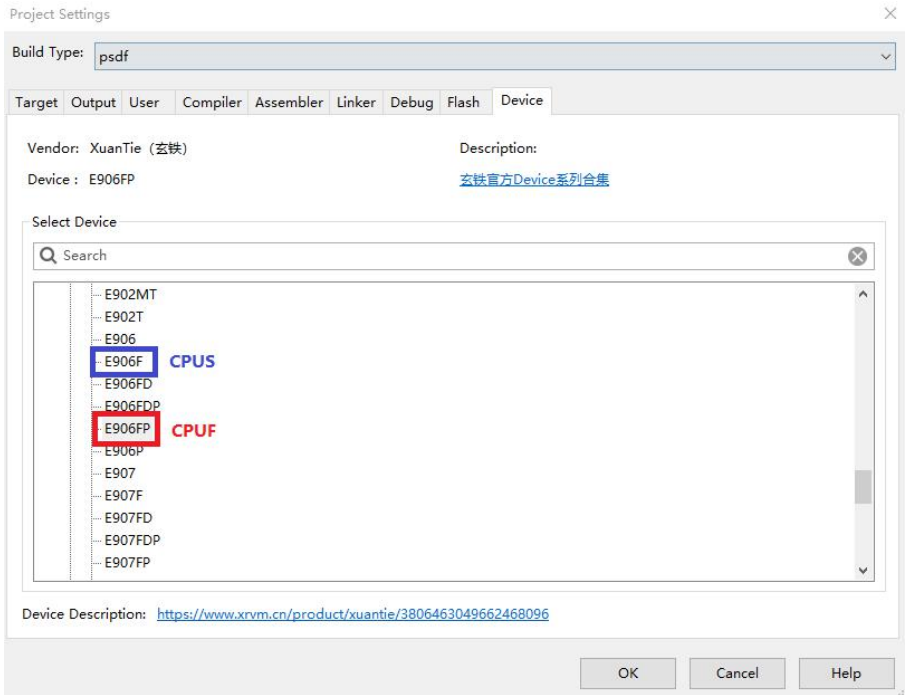


图 4-1 CDK CPU 指令集选择

2. CDK 配置中，选择对应的链接脚本。详情参考“Memory Map”章节的链接脚本描述。CPUS 和 CPUF 链接脚本的主要差异在 XIP 地址（CPUS 使用 0x0900 0000 地址，CPUF 使用 0x0940 0000 地址）。

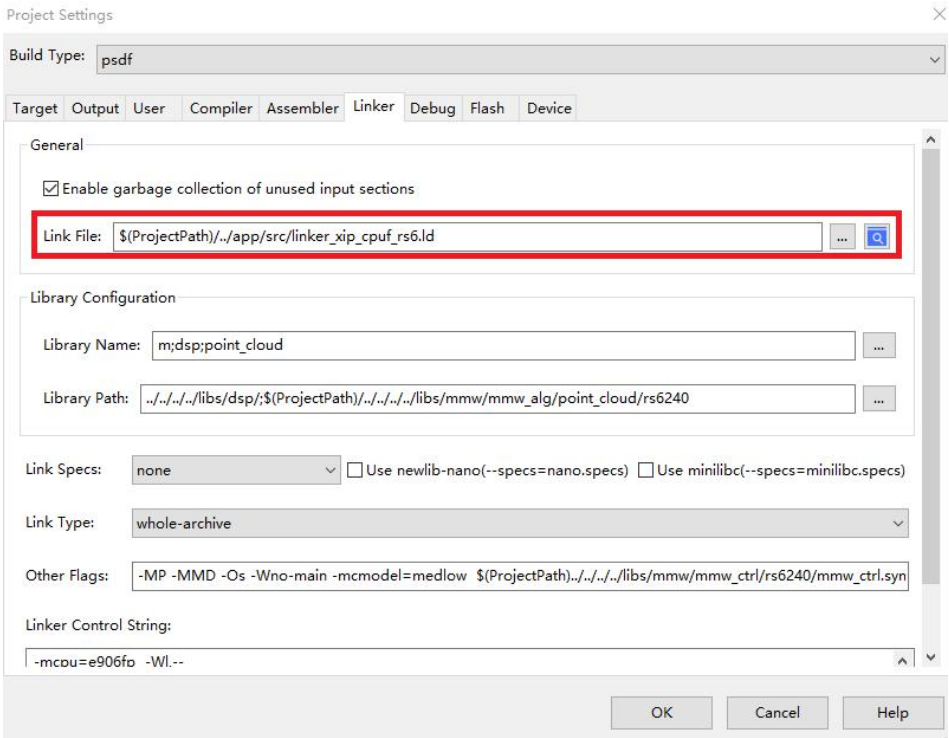


图 4-2 CDK 链接脚本配置

3. 若使用雷达，CDK 配置中选择正确的雷达固件符号表。

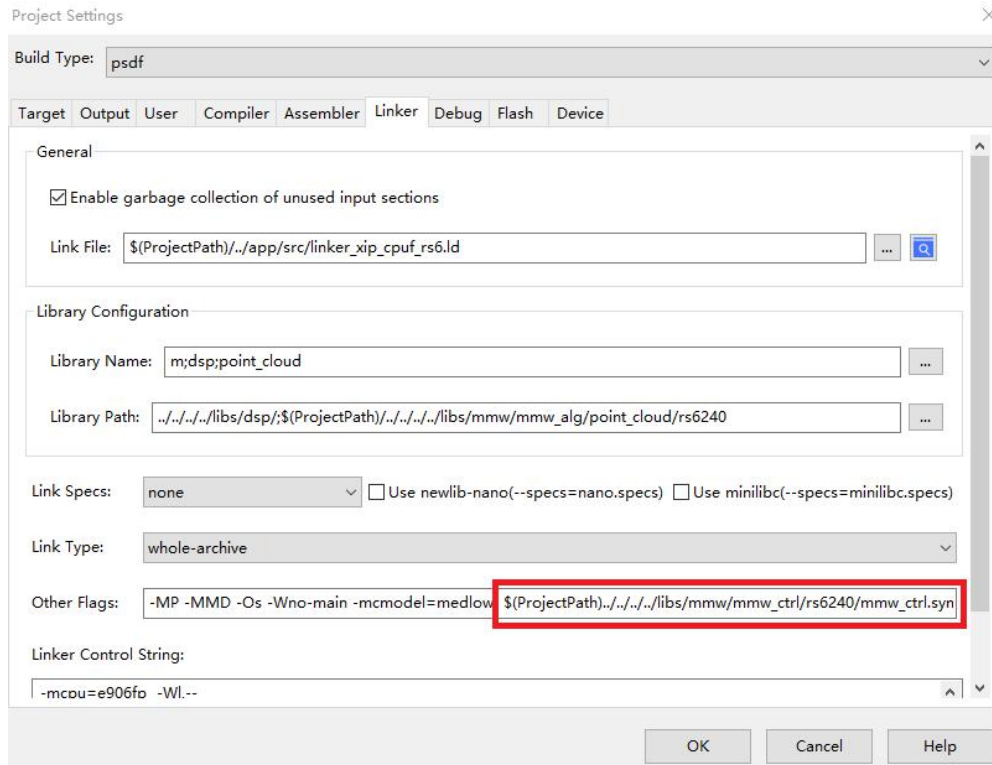


图 4-3 CDK 雷达符号表配置

4.1.1 Cache

CPU 中的 cache 可提高运行效率，尤其对运行在 XIP 中的代码和数据影响较大。Cache 的配置在 platform/soc/rs613x/src/rs613x.c 和 platform/soc/rs624x/src/rs624x.c 中，如下，cacheable 表示这段地址的读写会经过 cache，如 csi_sysmap_config_region(4, 0x18000000, SYSMAP_SYSMAPCFG_B_Msk | SYSMAP_SYSMAPCFG_C_Msk); 表示从前一个的 0x1040 0000 至 0x1800 0000 地址的访问都会经过 cache。这部分是数据，因此是经过 d-cache。

```
/* 0> 0x0000 0000 - 0x0050 0000 : rom/ram-code,      memory, non-cacheable, bufferable */
csi_sysmap_config_region(0, 0x00500000, SYSMAP_SYSMAPCFG_B_Msk);
/* 1> 0x0050 0000 - 0x0800 0000 : remap sram-code,    memory, cacheable,      bufferable */
csi_sysmap_config_region(1, 0x08000000, SYSMAP_SYSMAPCFG_B_Msk | SYSMAP_SYSMAPCFG_C_Msk);
/* 2> 0x0800 0000 - 0x1000 0000 : xip-code,          memory, cacheable,      bufferable */
csi_sysmap_config_region(2, 0x10000000, SYSMAP_SYSMAPCFG_B_Msk | SYSMAP_SYSMAPCFG_C_Msk);
/* 3> 0x1000 0000 - 0x1040 0000 : ram-data,          memory, non-cacheable, bufferable */
csi_sysmap_config_region(3, 0x10400000, SYSMAP_SYSMAPCFG_B_Msk);
/* 4> 0x1040 0000 - 0x1800 0000 : sram-data,         memory, non-cacheable, bufferable */
csi_sysmap_config_region(4, 0x18000000, SYSMAP_SYSMAPCFG_B_Msk | SYSMAP_SYSMAPCFG_C_Msk);
/* 5> 0x1800 0000 - 0x2000 0000 : xip-data,          memory, cacheable,      bufferable */
csi_sysmap_config_region(5, 0x20000000, SYSMAP_SYSMAPCFG_B_Msk | SYSMAP_SYSMAPCFG_C_Msk);
/* 6> 0x2000 0000 - 0x4000 0000 : sbus-sram,         memory, non-cacheable, bufferable */
csi_sysmap_config_region(6, 0x40000000, SYSMAP_SYSMAPCFG_B_Msk);
/* 7> 0x4000 0000 - 0xFFFF FFFF : periph,           device, non-cacheable, non-bufferable */
csi_sysmap_config_region(7, 0xFFFFFFF, SYSMAP_SYSMAPCFG_SO_Msk);
```

csi_icache_invalid API 表示无效全部 i-cache，后续的指令需要重新加载至 cache。

csi_dcache_clean_invalid_range API 表示无效和清除指定地址段的 d-cache，对应范围内的数据会写回和重新加载至 cache。

4.1.2 浮点

RS6130 和 RS6240 的 CPU 都支持硬件浮点指令，默认开启硬件浮点。软件在使用时若使用浮点可提高运算精度，但相应的会带来运行功耗的提升及代码 size 的增加。

注：硬件只支持单精度浮点，若软件使用双精度浮点会使用软件浮点库而增加代码空间。

4.1.3 双核

6240 可支持双核运行。建议雷达固件及算法相关功能运行在 CPUF，应用逻辑和通信可运行在 CPUS。

Prj_config.h 中使用 CONFIG_BOARD_MYBOARD6240_CPUF 则代码编译在 COUF 运行（另外注意链接文件和 CPU 指令集选择，见 4.1 CPU 注意事项），若定义 CONFIG_BOARD_MYBOARD6240_CPUS 则代码编译在 CPUS 运行。若需要双核则在最后一个核编译时使能 CONFIG_SOC_DUAL_CORE 即可一起打包。

当前版本暂不支持双核同时使用。

4.1.4 代码优化指引

4.1.4.1 SRAM占用分析

以 Radar Analysis 工程为例，介绍怎么分析 sram code 占有情况。

从编译输出可以获得 SRAM 占用总体情况，如下，总 SRAM 294KB（Data Sheet 描述为 295KB，另 1KB 用于文件系统 KVR（见 KVR & KVF 章节描述）），当前 img 占用 188.88KB，占比 64.24%。

（注：以下数据仅供案例分析，非最终实际工程 SRAM 占用情况。）

```
output image : mrs6240_p2512.img
version : R0.0-M0.0.0-V2.0.0
image size (flash): 252876 (246.95 KB)
sram size: 188.88k (64.24%). total size: 294k
```

打开当前工程目录下 output/img_cfg.yml 文件，找到 merge-img/remap-address。

```
merge-img:
  remap-address:
    - offset: 0x400400
      run-address: 0x500000 /* 0x500000 表示 app 应用(包括雷达算法静态库)指令运行地址 */
      size: 0x12000 /* app 应用代码占用 SRAM 大小 */
      type: sram-ibus-cpuf
    - offset: 0x419780
      run-address: 0x10500000 /* 0x10500000 表示 app 应用数据运行地址 */
      size: 0x16000 /* app 应用数据占用 SRAM 大小 */
      type: sram-dbus-cpuf
    - offset: 0x1000
```

```

run-address: 0x9400000
size: 0x20000          /* app 应用指令和数据占用 FLASH 大小 */
type: xip
- offset: 0x412400
  run-address: 0x580000 /* 0x580000 表示雷达固件指令运行地址 */
  size: 0x6a80          /* 雷达固件指令占用 SRAM 大小 */
  type: sram-ibus-cpusf
- offset: 0x418e80
  run-address: 0x10580000 /* 0x10580000 表示雷达固件数据运行地址 */
  size: 0x900           /* 雷达固件数据数据占用 SRAM 大小 */
  type: sram-dbus-cpusf
- offset: 0x21000
  run-address: 0x9c00000
  size: 0x19000         /* 雷达固件指令和数据占用 FLASH 大小 */
  type: xip

```

打开当前工程目录下 Lst/psdf.map 文件，和 app/src/linker_xip_cpuf_rs6.ld 文件（根据 CDK 配置打开对应文件）。例如要获取算法代码占用 SRAM 大小，则在对应前后增加标志即可，再在编译输出 map 文件获取标志地址相减即可获得大小。

app/src/linker_xip_cpuf_rs6.ld 文件

```

isram_text : {
    . = ALIGN(0x4) ;
    __s_isram_text = . ;

.....

    /* libc and libgcc */
    __s_libs_text = . ;
    *libc.a(*.text*)
    *libgcc.a(*.text*)
    *libdsp.a(*.text*)
    *libm.a(*.text*)
    *libc_newlib_stub.o(*.text*)
    __e_libs_text = . ;

    /* mmw */
    __s_mmw_algo_text = . ;          /* 雷达算法起始标志 */
    *mmw_process.o(*.text*)
    *mmw_psic_lib.o(*.text*)
    *mmw_algorithm.o(*.text*)
    *src_mmw.o(*.text*)
    *mmw_mdsp.o(*.text*)
    *mmw_mimo.o(*.text*)
    *common_lib.o(*.text*)
    __e_mmw_algo_text = . ;          /* 雷达算法结束标志 */

```

```
.....  
    . = ALIGN(0x4) ;  
    __e_isram_text = . ;  
} > ISRAM
```

Psdof.map 分别搜索__e_mmw_algo_text 和 __s_mmw_algo_text 获得地址，计算大小：0x0050d4e2 - 0x00508a14 = 0x4ACE = 18.7KB

Lst/psdf.map 文件

```
0x00508a14          __s_mmw_algo_text = .  
  
0x0050d4e2          __e_mmw_algo_text = .
```

要获得更详细函数大小，则在 psdf.map 搜索函数即可，例如 fast_db_trans_f32 函数大小为 0x3E.

Lst/psdf.map 文件

```
*mmw_process*.o(*.text*)  
.text.fast_db_trans_f32  
          0x00508a14      0x3e Obj/mmw_process_mmw_point_cloud_process.o  
          0x00508a14      fast_db_trans_f32
```

4.1.4.2 HEAP占用分析

以 Radar Analysis 工程为例，介绍怎么分析 heap 占有情况。
在 prj_config.h 中可以通过修改 CONFIG_HEAP_SIZE 宏定义来配置 heap 的大小（默认最大为 130KB）
在串口中输入“heap” shell 命令 获得 HEAP 占用总体情况，如下

```
psic:/$ heap  
start addr      : 10502000  
total size      : 130 KB (133120 B)  
current free size : 115 KB (118136 B)  
largest free size : 115 KB (118136 B)  
smallest free size : 115 KB (118136 B)  
number free blocks : 1  
min free size in history : 114 KB (117688 B)  
number call alloc : 29  
number call free  : 1  
  
Command Success
```

图 4-4 HEAP 占用情况

表 4-3 HEAP SHELL 返回值释义

Heap shell	Description
start addr	heap 起始地址为 10501E30
total size	heap 大小为 130KB

Heap shell	Description
current free size	当前剩余的 heap 大小为 115KB
largest free size	当前最大 heap 空闲块大小为 115KB
smallest free size	当前最小 heap 空闲块大小为 115KB
number free blocks	当前存在的 heap 空闲块数量为 1
min free size in history	历史最小空闲 heap 为 114KB
number call alloc	程序启动后分配内存的次数为 29
number call free	程序启动后释放内存的次数为 1

4.2 DSP

Soc 提供了软件 DSP 静态库，使用时需配置 CDK，如下图，Library Name 需加上 dsp，Library Path 需加上路径，指向 libs/dsp 目录。

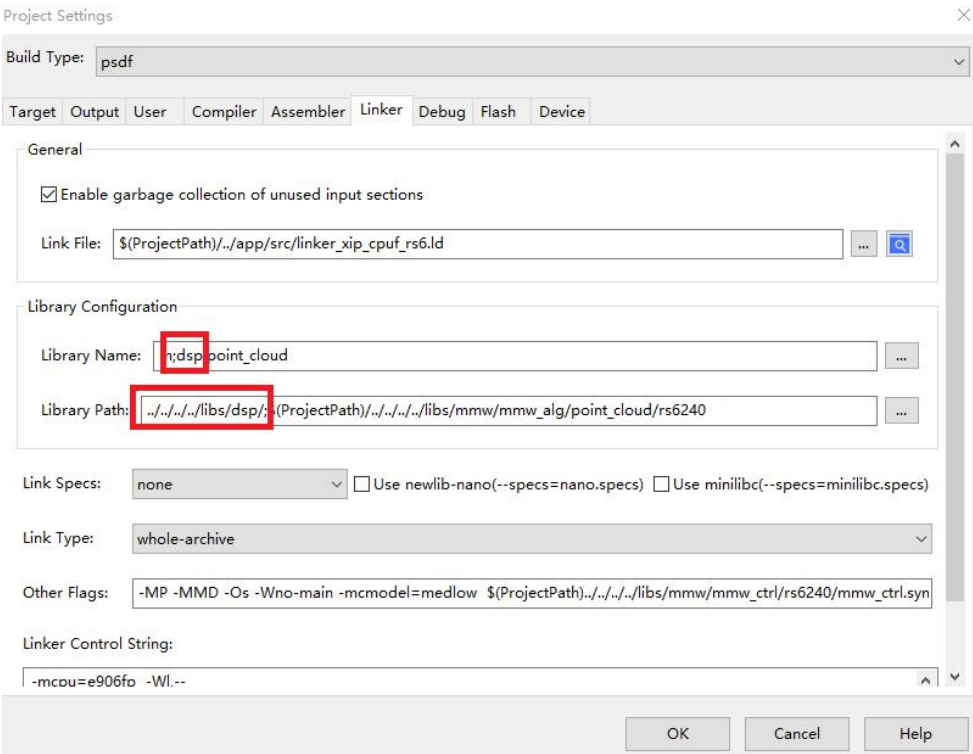


图 4-5 CDK 中 DSP lib 配置

头文件路径：subsys/dsp/inc/csi_math.h

API 说明参考《rs6xxx_DSP 接口_说明手册》。

4.3 FLASH 和 PSRAM

SOC 支持外挂 SPI FLASH 和 SPI PSRAM。通过 API HAL_FLASH_Open 可开启 SPI FLASH 及 SPI PSRAM，详情参考外设驱动 FLASH 章节。

FLASH 中可运行代码和只读数据。

FLASH 可通过 XIP 地址直接读取，也可通过 FLASH Read API 读取，但写入只能通过 API 先 Erase 再 Write。

通过 XIP 读取的地址如下：

表 4-4 FLASH XIP 地址分布

Address		Description
SYS 0	0x0900 0000	运行在 CPUS 上应用的 XIP 地址
SYS 1	0x0940 0000	运行在 CPUF 上的应用 XIP 地址
MMW CTRL	0x09C0 0000	雷达 BB 固件 XIP 运行地址，可以运行在 CPUS 或 CPUF 上
MMW ALG	0x0980 0000	雷达算法 XIP 运行地址，可运行在 CPUS 或 CPUF 上

PSRAM 可用于雷达算法大量中间数据和 LOG 的存储，理论上可执行 XIP 代码但不推荐使用。PSRAM 可运行代码和读写数据，可通过 XIP 地址读写，也可通过 API 进行读写。当前版本暂不支持。

表 4-5 PSRAM XIP 地址分布

Address		Description
SYS 0 Code	0x0D08 0000	运行在 CPUS 上应用的 PSRAM Code 地址段
SYS 0 Data	0x1D08 0000	运行在 CPUS 上的应用 PSRAM Data 地址段
SYS 1 Code	0x1D10 0000	运行在 CPUF 上应用的 PSRAM Code 地址段
SYS 1 Data	0x1D10 0000	运行在 CPUF 上的应用 PSRAM Data 地址段

4.3.1 代码在 XIP 或 SRAM

在默认代码运行在 XIP 的方案中，将代码移动至 SRAM，有以下两种方法：

方法一：修改对应的 linker_cpu.ld 文件。

例如文件名为 add_new_file.c，则在 isram_text 段中按照如下方法添加，那么 add_new_file.c 中定义的全部函数都将链接进入 isram_text 段。

```
isram_text : {
    . = ALIGN(0x4) ;
    __s_isram_text = . ;

    /* add_new_file 前后的*为文件通配符，建议前面一定要加，后面的*可以不加 */
```

```

        *add_new_file*.o(*.text*)

        . = ALIGN(0x4);
        __e_isram_text = .;
    } > ISRAM

```

方法二：通过在函数前添加修饰宏。

例如在函数前添加__sram_text 或__keep_sram_text 修饰符将此函数链接至 sram。

这两个宏定义在 platform/utils/compiler.h，使用时需包含此头文件。注意，__sram_text 的使用需确认 CONFIG_SECTION_ATTRIBUTE_SRAM 宏有开启。__sram_text 和__keep_sram_text 差异是__keep_sram_text 修饰函数无论是否孤立函数都会被链接至 sram，__sram_text 修饰的函数只有不是孤立的函数才会被链接至 sram。孤立函数是指不被其他函数调用，尤其是主流程，编译时会被优化节省 code size。

platform/utils/compiler.h 中宏定义

```

#define __keep_sram_text    __attribute__((section (".keep_sram_text")))
#define __keep_sram_rodata __attribute__((section (".keep_sram_rodata")))
#define __keep_sram_data   __attribute__((section (".keep_sram_data")))
#define __keep_sram_bss    __attribute__((section (".keep_sram_bss")))

#if (CONFIG_SECTION_ATTRIBUTE_SRAM == 1)
#define __sram_text        __attribute__((section (".sram_text")))
#define __sram_rodata     __attribute__((section (".sram_rodata")))
#define __sram_data       __attribute__((section (".sram_data")))
#define __sram_bss        __attribute__((section (".sram_bss")))
#else
#define __sram_text
#define __sram_rodata
#define __sram_data
#define __sram_bss
#endif

```

使用示例

```

__keep_sram_text int32_t HAL_CLOCK_GetFreq(HAL_Dev_t *clock, CLOCK_Type_t clkType)
{
    if ((clock == NULL) || (clkType >= CLOCK_MAX)) {
        return (int32_t)0;
    }

    Clock_Device_t *pClockDevice = (Clock_Device_t *)clock;

    return pClockDevice->freq[clkType];
}

```

链接文件链接.sram_text 和.keep_sram_text 段

```
isram_text : {  
    . = ALIGN(0x4) ;  
    __s_isram_text = . ;  
  
    KEEP(*(.(keep_sram_text)));  
    *(.sram_text);  
  
    . = ALIGN(0x4) ;  
    __e_isram_text = . ;  
} > ISRAM
```

4.4 Memory Map

软件开发的主要流程如下：

1. 通过 CDK 编译输出 bin，同时通过打包脚本将相关 bin 合成一个 img；
2. 通过烧录工具（BURN Tools）与预留在 soc 内部的 ROM 程序通信将 img 写至外部 spi Flash；
3. 复位 soc，CPU 首先运行 ROM 程序，ROM 程序从 spi Flash 加载 bootloader 并运行，bootloader 加载应用程序并运行。



图 4-6 启动过程

4.4.1 Img 格式

编译输出及烧录在 spi Flash 中的 img（例如 mrs6240_p2512.img）组织形式如下，主要由 bootloader.img，merge.img，flash_list.img 和 custom_cfg.img 组成。

1. bootloader.img：二级 boot 程序，负责 img 校验，程序加载及 OTA 等功能，SDK 以 img 方式提供。
2. merge.img：应用主体，包括 sys.img，mmw_ctrl.img，mmw_alg.img 等子 img。
 - (1) sys0.img/sys1.img：应用 img，例如 radar anlysis sample 编译输出的 sys1.img，包括 OS，外设驱动，子系统，雷达应用等。SDK 提供源码，编译时输出 sys0.img/sys1.img。cpus 对应 sys0，cpuf 对应 sys1 需注意各系列 soc 支持情况。
 - (2) mmw_ctrl.img：雷达 BB 驱动 img，SDK 以 img 方式提供。
 - (3) mmw_alg.img：雷达算法 img，SDK 对部分闭源算法以 img 方式提供。
2. Flash_list.img：用于存储 spi flash 支持列表参数。
3. custon_cfg.img：用于存储应用配置，可选。

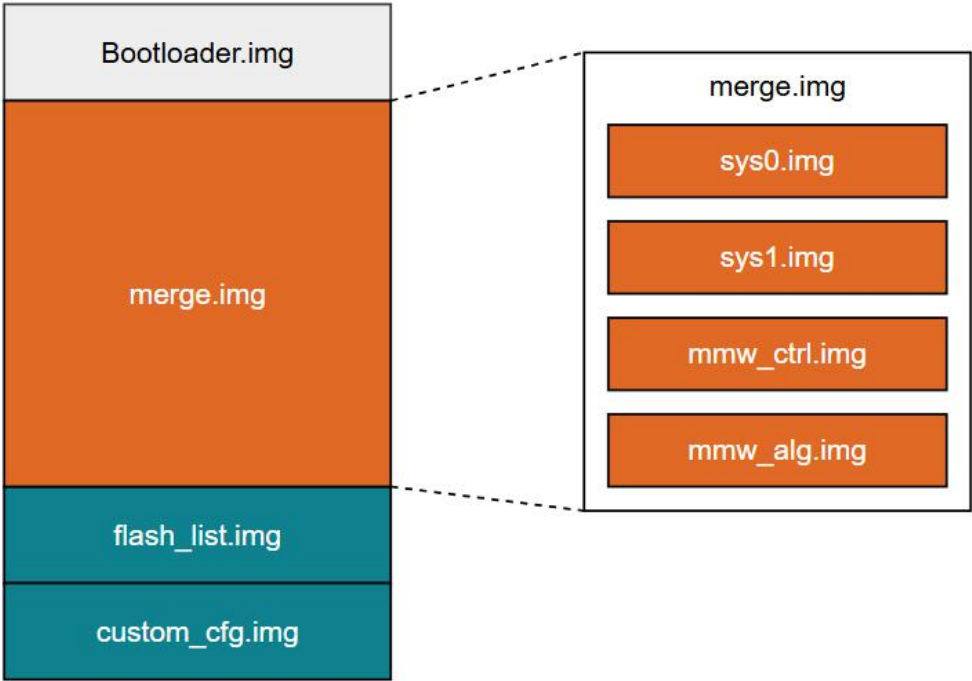


图 4-7 Img 组织形式

每个 img 及子 img 格式如下，img-head 包含版本，img 大小等信息，img-tail 为 hash 校验，img-body 为运行代码或配置主体。



图 4-8 Img 格式

4.4.2 Map

每个 img 的内存排布如下，包括运行在 XIP 的代码和 RO-data，运行在 SRAM 的代码，RO-data，data 及堆栈等段。整个 img 按此排布烧录在 spi flash 的中。bootloader 会将对应的 isram_text，isram_rodata 和 dsram_data 复制到对应的 sram，并设置好 ixip，isram，dsram 的虚拟运行地址。

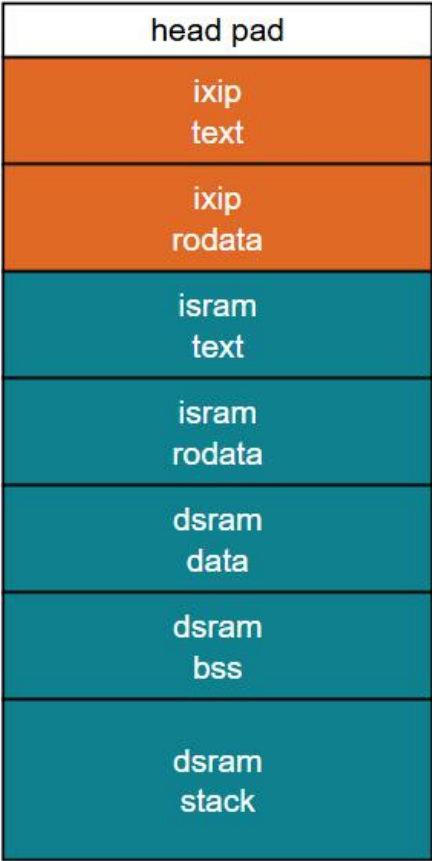


图 4-9 Img memory map

各 img 的 memory remap address 如下表

表 4- 6 Address Remap

	SYS0	SYS1	MMW_CTRL	MMW_ALG
iXIP	0x0900 0000	0x0940 0000	0x09C0 0000	0x0980 0000
iSRAM	0x0050 0000	0x0050 0000	0x0058 0000	0x0054 0000
dSRAM	0x1050 0000	0x1050 0000	0x1058 0000	0x1054 0000

bootloader 会将运行地址映射到不同的物理地址，比如 XIP 映射到 spi flash 不同的偏移位置，sram 映射到物理 sram 的不同位置。

在 SDK 源码的 platform/utils 目录下存在如下链接脚本，各段对应上面的描述。

- 1. linker_cpus.ld: cpus 链接脚本，默认代码运行在 XIP，实时性要求和休眠时 XIP 起来前的代码放在 sram
- 2. linker_cpuf.ld: cpuf 链接脚本，默认代码运行在 XIP，实时性要求和休眠时 XIP 起来前的代码放在 sram
- 3. linker_sram_cpus.ld: cpus 链接脚本，所有代码运行在 SRAM
- 4. Linker_sram_cpuf.ld: cpuf 链接脚本，所有代码运行在 SRAM

4.5 启动和配置

4.5.1 初始化

如前所述，bootloader 加载 merge.img 后跳转至 sys0.img 或 sys1.img，cpu 控制权交给应用。

应用启动流程如下（详情如图 4-5）：

- 1. reset_handler：初始化了中断栈，中断向量表及 CPU 属性配置，platform/arch/riscv/startup.s 实现。
- 2. soc_init：bss clear（data 段初始化在 bootloader 中实现），memory feature 配置，cache 开启。总入口在 platform/soc/common/src/soc.c，具体在各 soc 文件实现，例如 platform/soc/rs6130/rs6130.c。
- 3. board_init：具体方案配置初始化，包括 OS 启动，系统驱动初始化。总入口在 platform/boards/board.c，具体在各方案 board 中实现，例如 platform/boards/mrs6240_p2512/mrs6240_p2512.c。
- 4. main：应用入口，在各 project 的 main.c 中实现。

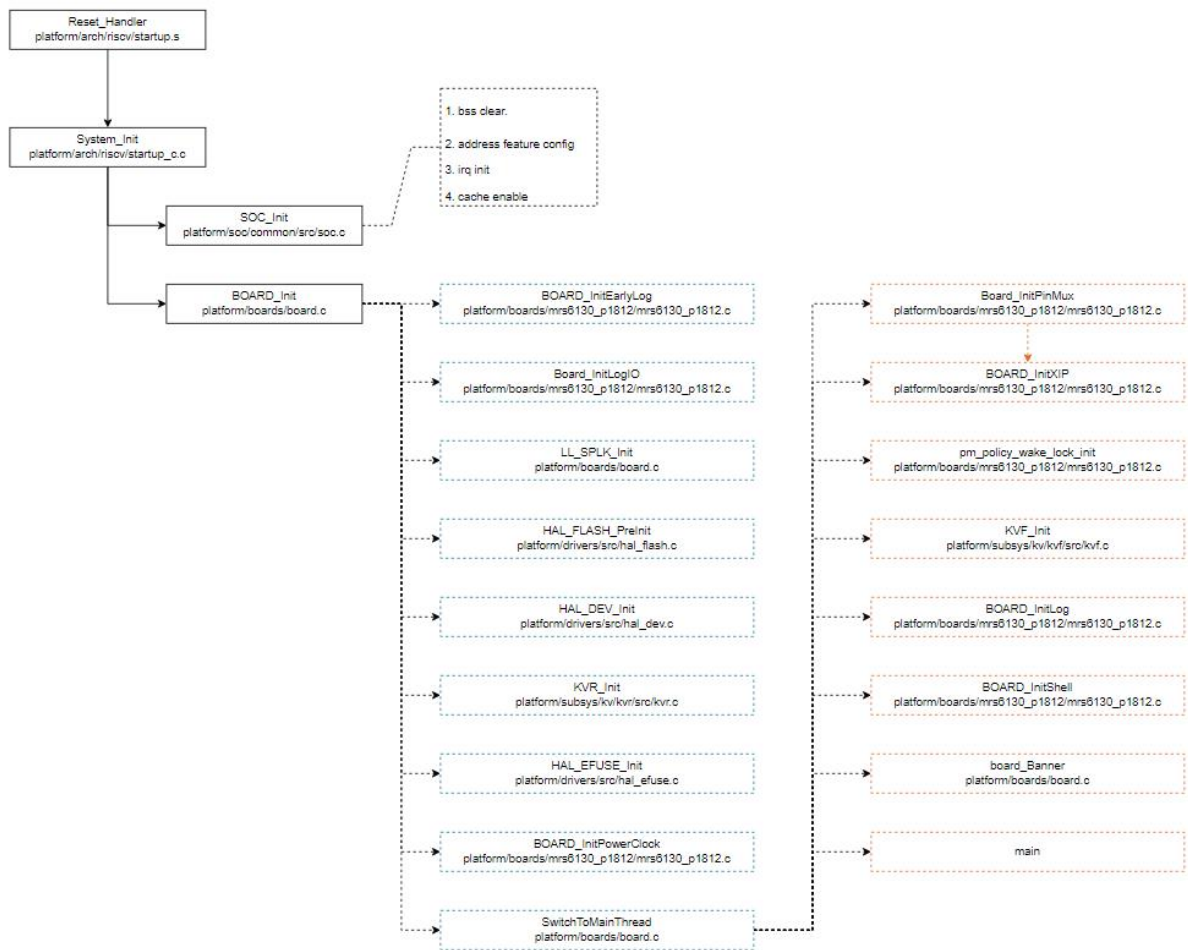


图 4-10 启动流程

4.5.2 配置

系统可通过宏配置相关功能和参数，其配置包含关系如下图。prj_config.h 为各工程配置，处于最顶层，可覆盖 board_config.h 和 soc_config.h 中的默认配置。

驱动及应用编码中包含“config.h”文件即可，勿只含 prj_config.h 等头文件导致配置不全问题。

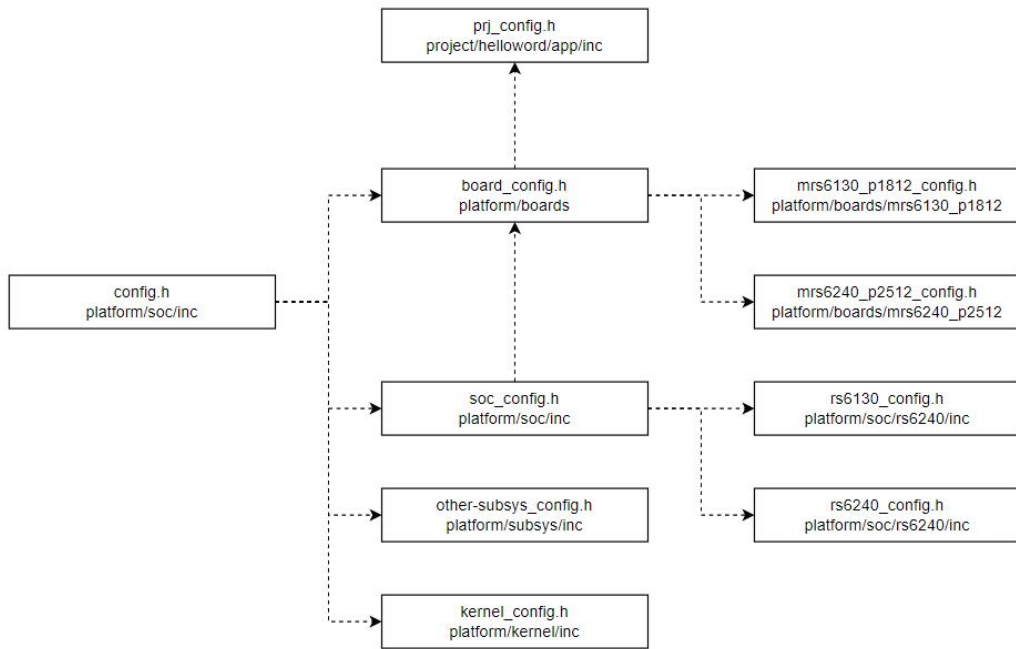


图 4-11 配置调用关系

4.6 Clock

时钟配置具体在各 board 文件中初始化，例如 mrs6240_p2512 这款 board 在 platform/boards/mrs6240_p2512/mrs6240_p2512.c 中的 BOARD_InitPowerClock 函数实现。

具体配置与 soc 的电源和时钟架构相关，为了简化使用，默认提供了 3 档配置，通过宏 CONFIG_BOARD_CLOCK_CFG 选择，不建议用户随意调整各参数，除非已掌握当前 soc 的电源和时钟细节。

```

#if (CONFIG_BOARD_CLOCK_CFG == 32)
#include "rs624x_config_cpuf_32mhz.h"
#elif (CONFIG_BOARD_CLOCK_CFG == 96)
#include "rs624x_config_cpuf_96mhz.h"
#elif (CONFIG_BOARD_CLOCK_CFG == 112)
#include "rs624x_config_cpuf_112mhz.h"
#elif (CONFIG_BOARD_CLOCK_CFG == 160)
#include "rs624x_config_cpuf_160mhz.h"
#elif (CONFIG_BOARD_CLOCK_CFG == 192)
#include "rs624x_config_cpuf_192mhz.h"
#elif (CONFIG_BOARD_CLOCK_CFG == 224)
#include "rs624x_config_cpuf_224mhz.h"
#elif (CONFIG_BOARD_CLOCK_CFG == 240)
#include "rs624x_config_cpuf_240mhz.h"
#elif (CONFIG_BOARD_CLOCK_CFG == 256)
#include "rs624x_config_cpuf_256mhz.h"
#endif
  
```

以上配置选择对应各时钟频率如下。

表 4- 7 6130 时钟配置

CPUF MHz	Msi MHz	PLL MHz	SYS_CLK MHz	AHB_CLK MHz	APB_CLK MHz	XIP MHz	SPI MHz
32	32	OFF	32	32	32	16	8
96	32	192	96	96	48	48	24
112	32	224	112	112	56	56	28
128	32	256	128	128	64	64	32

表 4- 8 6240 时钟配置

CPUF MHz	Msi MHz	PLL MHz	SYS_CLK MHz	AHB_CLK MHz	APB_CLK MHz	XIP MHz	SPI MHz
32	32	OFF	32	32	32	16	8
96	32	192	96	96	48	48	24
112	32	224	112	112	56	56	28
160	32	320	160	80	40	80	40
192	32	384	192	96	48	96	48
224	32	224	224	112	56	56	56
240	32	240	240	120	60	60	60
256	32	256	256	128	64	64	64

4.7 Pinmux

在 board 初始化实现中，例如 platform/boards/mrs6240_p2512/mrs6240_p2512.c 中定义了各 pinmux 的配置。使用宏 DEFINE_PINMUX_WEAK 按照格式编写。

```
#if CONFIG_DRIVER_UART
DEFINE_PINMUX_WEAK(uart0, 2, // 名称 uart0, pinmux 列表为 2 个，即下面初始化了 2 个 pin
    { GPIO_PORT_B, GPIO_PIN_0, {GPIOB_P0_F2_UART0_TX, GPIO_DRIVING_LEVEL_1,
    GPIO_PUSH_PULL, GPIO_PULL_UP }}, /* TX */
    // GPIO 端口号, pin 号, pin function 名称, 输出驱动能力, 输出类型, 上下拉电阻
    { GPIO_PORT_B, GPIO_PIN_1, {GPIOB_P1_F2_UART0_RX, GPIO_DRIVING_LEVEL_1,
    GPIO_OPEN_DRAIN, GPIO_PULL_UP }}, /* RX */
);

DEFINE_PINMUX_WEAK(uart0_deinit, 2,
    { GPIO_PORT_B, GPIO_PIN_0, {GPIOx_Pn_F15_DIS, GPIO_DRIVING_LEVEL_1, GPIO_PUSH_PULL,
    GPIO_PULL_UP }}, /* TX */
    { GPIO_PORT_B, GPIO_PIN_1, {GPIOx_Pn_F15_DIS, GPIO_DRIVING_LEVEL_1,
```

```
GPIO_OPEN_DRAIN, GPIO_PULL_UP }}, /* RX */
);
#endif
```

若添加其他用途 pinmux，需更改函数 board_GetPinMux。

```
HAL_Status_t board_GetPinMux(
    HAL_DevType_t type,
    uint8_t idx,
    HAL_PinMuxType_t cfgType,
    HAL_Pinmux_t * pPinMux)
{
    HAL_Status_t status = HAL_STATUS_INVALID_STATE;

    switch (type) {
    case HAL_DEV_TYPE_UART:    // 驱动类型
        if (idx == UART0_ID) { // 驱动 ID
            if (cfgType == HAL_PINMUX_TYPE_INIT) { // 配置类似，初始化配置
                pPinMux->pinConfig = (HAL_PinmuxConfig_t *)pinmux_uart0.pinConfig;
                pPinMux->pinCnt = pinmux_uart0.pinCnt;
                status = HAL_STATUS_OK;
            } else if (cfgType == HAL_PINMUX_TYPE_DEINIT) { // 配置类似，反初始化配置
                pPinMux->pinConfig = (HAL_PinmuxConfig_t *)pinmux_uart0_deinit.pinConfig;
                pPinMux->pinCnt = pinmux_uart0_deinit.pinCnt;
                status = HAL_STATUS_OK;
            }
        } else {
            break;
        }
        break;

    default:
        break;
    }

    return status;
}
```

4.8 LOG

Log 子系统包括两部分：打印和 IO。

开启 log 子系统，首先需将 subsys/log 目录的源文件和头文件路径添加在编译中，其次需要开启对应的宏，subsys/log/inc/log_config.h 文件定义了配置宏。

```
/* 是否开启打印，1 为开启，0 为关闭 */
```

```

#ifndef CONFIG_PRINTF
#define CONFIG_PRINTF                                0
#endif

/* 是否通过 UART 打印，默认为 1 */
#ifndef CONFIG_PRINTF_UART
#define CONFIG_PRINTF_UART                          1
#endif

/* 打印串口号，0 为串口 0， 1 为串口 1 */
#ifndef CONFIG_PRINTF_UART_NUM
#define CONFIG_PRINTF_UART_NUM                      0
#endif

/* 串口打印波特率 */
#ifndef CONFIG_PRINTF_UART_BAUDRATE
#define CONFIG_PRINTF_UART_BAUDRATE                115200
#endif

```

在各个 board 的配置文件中，如 “mrs6130_p1812_config.h” 中 “CONFIG_PRINTF_UART_NUM” 会关联使能对应的串口。

```

/** Printf **/
#if CONFIG_PRINTF
#ifndef CONFIG_PRINTF_UART_NUM
#define CONFIG_PRINTF_UART_NUM                    0
#endif

#ifndef CONFIG_PRINTF_UART_BAUDRATE
#define CONFIG_PRINTF_UART_BAUDRATE              115200
#endif

/* depend on UART_DRIVER */
#if ((CONFIG_PRINTF >= 1) && (CONFIG_PRINTF_UART_NUM == 0))
#undef CONFIG_DRIVER_UART_0
#define CONFIG_DRIVER_UART_0                      1

#elif ((CONFIG_PRINTF >= 1) && (CONFIG_PRINTF_UART_NUM == 1))
#undef CONFIG_DRIVER_UART_1
#define CONFIG_DRIVER_UART_1                      1

#elif ((CONFIG_PRINTF >= 1) && (CONFIG_PRINTF_UART_NUM == 2))
#undef CONFIG_DRIVER_UART_2
#define CONFIG_DRIVER_UART_2                      1

#else
#error "CONFIG_PRINTF enable, but CONFIG_PRINTF_UART_NUM is error value!"
#endif
#endif /* CONFIG_PRINTF */

```

注：串口的 pinmux 在各 board.c 文件中定义，如 “mrs6130_p1812.c”。

使用中需按照如下方式包含文件：

```
/* 在 log.h 前包含 common.h 或 hal_types.h */
#include "common.h"

#define LOG_MODULE          "name"      /* log 输出的模块前缀 */
#define LOG_LEVEL           LEVEL_ERR   /* log 级别 */
#include "log.h"
```

可使用的打印宏定义在 log.h 中，包括“LOG_ERR”，“LOG_ERR_HEX”等分级别宏以及“LOG_PRINT”“LOG_HEX”不受“LOG_LEVEL”限制的宏。参考样例工程：project/basic/shell_log

```
LOG_PRINT("Current 'LOG_LEVEL' is '%s'\n", strLogLevel[logLevel]);
LOG_PRINT("Current 'LOG_MODULE' is '%s'\n\n", LOG_MODULE);

LOG_PRINT("1> This is 'LOG_PRINT' print\n");
LOG_HEX(sampleData, SAMPLE_DATA_SIZE, "2> This is 'LOG_HEX' print");
LOG_DBG("3> This is 'LOG_DBG' print\n");
LOG_DBG_HEX(sampleData, SAMPLE_DATA_SIZE, "2> This is 'LOG_DBG' print");
LOG_INF("1> This is 'LOG_INF' print\n");
LOG_INF_HEX(sampleData, SAMPLE_DATA_SIZE, "2> This is 'LOG_INF_HEX' print");
LOG_WRN("1> This is 'LOG_WRN' print\n");
LOG_WRN_HEX(sampleData, SAMPLE_DATA_SIZE, "2> This is 'LOG_WRN_HEX' print");
LOG_ERR("1> This is 'LOG_ERR' print\n");
LOG_ERR_HEX(sampleData, SAMPLE_DATA_SIZE, "2> This is 'LOG_ERR_HEX' print");
```

LOG_IO 是一种通过拉高拉低 IO 来调试的手段，需在 prj_config.h 中配置“CONFIG_LOG_IO_TAG_MASK”“CONFIG_LOG_IO_MASK”宏。参考样例工程：project/basic/run_time_measure。

```
/* 定义 TAG，需要注意不能与系统已有 TAG 重叠(定义在 log.h) */
#define CONFIG_LOG_IO_TAG_MASK          0x00040000

/* 定义 IO mask， 0x3 表示 PA0 和 PA1 受控制 */
#define CONFIG_LOG_IO_MASK              0x3

LOG_IO(0x00040000, 1, 1, 0);    /* PA1 拉高 */
OSI_MSleep(500);
LOG_IO(0x00040000, 1, 0, 0);    /* PA1 拉低 */

LOG_IO(0x00040000, 1, 0, 3);    /* PA1 连续 3 个正脉冲 */
```

4.9 控制台 Shell

Shell 的使用，需将“platform/subsys/shell”中源码和路径加入编译，并开启宏“CONFIG_SHELL”。

```

/* 使能 shell */
#ifndef CONFIG_SHELL
#define CONFIG_SHELL 0
#endif

/* 配置 shell 串口号 */
#if CONFIG_SHELL
#ifndef CONFIG_SHELL_UART_NUM
#define CONFIG_SHELL_UART_NUM 0
#endif

/* 配置 shell 串口波特率 */
#ifndef CONFIG_SHELL_UART_BAUDRATE
#define CONFIG_SHELL_UART_BAUDRATE 115200
#endif

```

“platform/cmd”中定义了一些默认的 shell，可参考使用。

```

/* shell 实现函数 */
int set_config_cmd(Shell *shell, int argc, char *argv[])
{
    /* ..... */
    return 0;
}

/* shell 子命令 */
ShellCommand gpio_cmds[] = {
/* set_config：子命令提示符
 * set_config_cmd：命令实现函数
 * “<port>”：命令帮助提示符
 */
    SHELL_CMD_ITEM(set_config, set_config_cmd, "<port> <pull>"),
    SHELL_CMD_ITEM_END()
};

/* shell 命令组 */
SHELL_CMD_GROUP(gpio, gpio_cmds, "gpio command");

```

4.10 电源管理

4.10.1 简介

电源管理是通过 OS 的 idle 线程实现的，当系统中无其他应用线程运行时进入 idle 线程。idle 线程获取下一个任务解除阻塞的时间，用此时间与系统预设的进入休眠耗时（enter_latency）和唤醒延迟（exit_latency）比较获得其休眠策略（浅休眠或深休眠）。若系统可深度休眠，则会配置低功耗定时器唤醒时间，并根据设备注册的顺序依次调用设备的休眠接口进行休眠，最后系统休眠处理保存系统线程

并配置硬件进行下电处理。低功耗定时器或其他唤醒源会唤醒系统，系统硬件上电完成后系统恢复函数首先会恢复系统现场，进行唤醒源处理，再根据设备休眠逆序调用设备恢复接口恢复设备，最后开中断和进行调度切换。

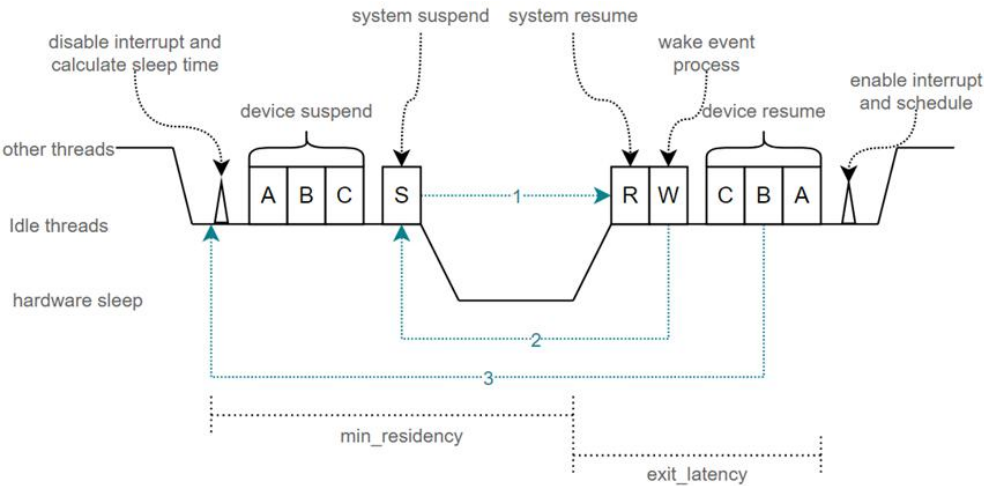


图 4-12 电源管理流程示意图

使用电源管理 API，需要先在工程配置文件打开以下宏：

```
#define CONFIG_PM 1
#define CONFIG_PM_DEVICE 1

/* 若需要使用设备 runtime 功能，还需配置该宏 */
#define CONFIG_PM_DEVICE_RUNTIME 1
```

4.10.2 接口

4.10.2.1 pm_device_register

表 4-9 pm_device_register

接口	说明
函数	int pm_device_register(struct pm_device *dev, pm_device_action_cb_t action);
功能	设备休眠唤醒注册。
参数	struct pm_device *dev 指向待注册的设备对象，用户只需要定义 struct pm_device 类型，不需要对成员进行赋值。 pm_device_action_cb_t action 设备休眠唤醒的处理回调，一旦系统进入/退出休眠该回调将被执行。 pm_device_action_cb_t 类型声明为： typedef int (*pm_device_action_cb_t)(void *dev, enum pm_device_action action),

接口	说明
	pm_device_action 声明为： PM_DEVICE_ACTION_SUSPEND, /* 设备挂起 */ PM_DEVICE_ACTION_RESUME, /* 设备恢复 */ PM_DEVICE_ACTION_TURN_OFF, /* 目前未用到 */ PM_DEVICE_ACTION_TURN_ON, /* 目前已用到 */
注意事项	无。
返回值	返回 0 表示注册成功，返回 1 表示设备已被注册过，其他值表示注册失败。

4.10.2.2 pm_device_unregister

表 4- 10 pm_device_unregister

接口	说明
函数	int pm_device_unregister(struct pm_device *dev)
功能	设备休眠唤醒注销。
参数	struct pm_device *dev 指向已注册过的设备对象。
注意事项	无。
返回值	返回 0 表示注销成功，其他值表示失败。

4.10.2.3 pm_notifier_register

表 4- 11 pm_notifier_register

接口	说明
函数	void pm_notifier_register(struct pm_notifier *notifier);
功能	设备休眠通知注册。
参数	struct pm_notifier *notifier 指向待通知注册的对象，struct pm_notifie 类型声明如下： struct pm_notifier { sys_snode_t _node; void (*state_entry)(enum pm_state state); void (*state_exit)(enum pm_state state); }; 一旦注册成功后，当系统进入/退出休眠时 state_entry/state_exit 所指向的函数分别将被执行。

接口	说明
注意事项	无。
返回值	无。

4.10.2.4 pm_notifier_unregister

表 4- 12 pm_notifier_unregister

接口	说明
函数	int pm_notifier_unregister(struct pm_notifier *notifier);
功能	设备休眠通知注销。
参数	struct pm_notifier *notifier 指向已注册过通知的对象。
注意事项	无。
返回值	0 表示注销成功，其它值为不成功。

4.10.2.5 pm_policy_latency_update

表 4- 13 pm_policy_latency_update

接口	说明
函数	void pm_policy_latency_update(uint32_t entry_latency, uint32_t exit_latency);
功能	休眠唤醒延迟更新。
参数	uint32_t entry_latency 系统进入休眠的耗时。 uint32_t exit_latency 系统退出休眠的耗时。 耗时的更新要根据实际应用的执行耗时。
注意事项	无。
返回值	无。

4.10.2.6 pm_policy_wake_lock_is_active

表 4- 14 pm_policy_wake_lock_is_active

接口	说明
函数	bool pm_policy_wake_lock_is_active(void);

接口	说明
功能	判断唤醒锁是否存在。
参数	无。
注意事项	无。
返回值	返回 true 表示存在唤醒锁，返回 false 不存在唤醒锁。

4.10.2.7 pm_policy_wake_lock

表 4- 15 pm_policy_wake_lock

接口	说明
函数	void pm_policy_wake_lock(struct wakelock *lock);
功能	获取一个唤醒锁，只要系统持有唤醒锁便不会进入休眠，与 pm_policy_wake_unlock 成对使用。
参数	struct wakelock *lock 表示待获取的唤醒锁，其类型声明如下： struct wakelock { sys_snode_t _node; /* 唤醒锁节点 */ char *name; /* 唤醒锁名称，用户可配 */ uint16_t ref; /* 唤醒锁计数值，用户不需要配置 */ uint32_t expires; /* 唤醒锁超时时间，用户不需要配置 */ };
注意事项	无。
返回值	无。

4.10.2.8 pm_policy_wake_unlock

表 4- 16 pm_policy_wake_unlock

接口	说明
函数	void pm_policy_wake_unlock(struct wakelock *lock);
功能	释放一个唤醒锁，与 pm_policy_wake_lock 成对使用。
参数	struct wakelock *lock 表示待释放的唤醒锁。
注意事项	无。

接口	说明
返回值	无。

4.10.2.9 pm_wake_lock_timeout

表 4- 17 pm_wake_lock_timeout

接口	说明
函数	void pm_wake_lock_timeout(struct wakelock *lock, uint32_t timeout);
功能	获取一个超时唤醒锁。
参数	struct wakelock *lock 表示待获取的唤醒锁。 uint32_t timeout 唤醒锁持有的时间，单位为 ms。一旦获取该唤醒锁，系统将经过 timeout 时间后自动释放该唤醒锁。若想提前释放该唤醒锁，则将 timeout 设置为 0。
注意事项	无。
返回值	无。

4.10.2.10 pm_device_runtime_enable

表 4- 18 pm_device_runtime_enable

接口	说明
函数	int pm_device_runtime_enable(struct pm_device *dev);
功能	设备 runtime 功能使能，runtime 功能是应用需要用时设备时才打开。
参数	struct pm_device *dev 表示待使能 runtime 功能的设备。 一旦使能该功能，若设备已经是处于打开状态，会关闭设备，且设备的休眠唤醒将不受系统 PM 影响。
注意事项	无。
返回值	返回 0 表示使能成功，其它值表示使能失败。

4.10.2.11 pm_device_runtime_disable

表 4- 19 pm_device_runtime_disable

接口	说明
----	----

接口	说明
函数	int pm_device_runtime_disable(struct pm_device *dev);
功能	设备 runtime 功能失能，runtime 功能是应用需要用时设备时才打开。
参数	struct pm_device *dev 表示待关闭 runtime 功能的设备。
注意事项	无。
返回值	返回 0 表示关闭成功，其它值表示关闭失败。

4.10.2.12 pm_device_runtime_get

表 4- 20 pm_device_runtime_get

接口	说明
函数	int pm_device_runtime_get(struct pm_device *dev);
功能	打开设备，一旦应用需要使用设备，可通过该接口使能设备。
参数	struct pm_device *dev 表示待打开的设备。
注意事项	无。
返回值	返回 0 表示打开成功，其它值表示打开失败。

4.10.2.13 pm_device_runtime_put

表 4- 21 pm_device_runtime_put

接口	说明
函数	int pm_device_runtime_put(struct pm_device *dev);
功能	关闭设备，一旦应用不需要使用设备了，可通过该接口关闭设备。
参数	struct pm_device *dev 表示待关闭的设备。
注意事项	无。
返回值	返回 0 表示关闭成功，其它值表示关闭失败。

4.11 KVR & KVF

4.11.1 简介

SDK 提供两种键值对（Key-Value）的存储系统，一种存储在 SRAM（KVR），一种存储在 FLASH（KVF）。KVR 存储在 SRAM 的低地址，分系统和应用，共占 1KB。掉电丢失，系统存储在初次上电时清除初始化，应用存储在每次系统复位清除重新初始化。实现在 `subsys/kv/kvr`。

KVF 存储在 FLASH 尾部 8KB，有效存储大小为 4KB（乒乓操作共 8KB）。掉电不丢失，全片擦除时会清除。实现在 `subsys/kv/kvf`。

系统初始化时在 `Board.c` 中默认初始化。

4.11.2 接口

4.11.2.1 KVR_Save

表 4- 22 KVR_Save

接口	说明
函数	<code>int KVR_Save(uint16_t id, void *pdata, uint32_t size);</code>
功能	创建并保存数据至 SRAM。
参数	uint16_t id 存储 ID, 若是已创建的 ID 则覆盖存储，否则创建新的存储 ID。 若是系统存储则最高 bit 置位 1。 void *pdata 存储数据 uint32_t size 存储数据长度，同一个 ID, 若新存储数据长度大于前一次存储长度则当前 ID 的存储空间会扩容。
注意事项	无。
返回值	若为少于等于 0 表示存储失败，若大于 0 表示存储数据长度。

4.11.2.2 KVR_Load

表 4- 23 KVR_Load

接口	说明
函数	<code>int KVR_Load(uint16_t id, void *pdata, uint32_t size);</code>
功能	从指定 ID，读取存储数据

接口	说明
参数	uint16_t id 存储 ID，若是系统存储则最高 bit 置位 1。 void *pdata 存储数据 uint32_t size 存储数据长度，若是未创建的 I 则返回错误码，若读取长度少于等于存储长度则返回请求长度；若请求长度大于存储长度则只返回存储长度的数据。
注意事项	无。
返回值	若为少于等于 0 表示读取失败，若大于 0 表示读取数据长度。

4.11.2.3 KVR_Delete

表 4- 24 KVR_Delete

接口	说明
函数	int KVR_Delete(uint16_t id);
功能	从存储系统中删除指定 ID 数据。
参数	uint16_t id 存储 ID，若是系统存储则最高 bit 置位 1。
注意事项	无。
返回值	若为少于等于 0 表示读取失败，若大于 0 表示读取数据长度。

4.11.2.4 KVF_Save

表 4- 25 KVF_Save

接口	说明
函数	int KVF_Save(uint16_t id, void *pdata, uint32_t size);
功能	创建并保存数据至 FLASH。
参数	uint16_t id 存储 ID，若是已创建的 ID 则覆盖存储，否则创建新的存储 ID。 若是系统存储则最高 bit 置位 1。

接口	说明
	void *pdata 存储数据 uint32_t size 存储数据长度，同一个 ID，若新存储数据长度大于前一次存储长度则当前 ID 的存储空间会扩容。
注意事项	无。
返回值	若为少于等于 0 表示存储失败，若大于 0 表示存储数据长度。

4.11.2.5 KVF_Load

表 4- 26 KVF_Load

接口	说明
函数	int KVF_Load(uint16_t id, void *pdata, uint32_t size);
功能	从指定 ID，读取存储数据
参数	uint16_t id 存储 ID，若是系统存储则最高 bit 置位 1。 void *pdata 存储数据 uint32_t size 存储数据长度，若是未创建的 I 则返回错误码，若读取长度少于等于存储长度则返回请求长度；若请求长度大于存储长度则只返回存储长度的数据。
注意事项	无。
返回值	若为少于等于 0 表示读取失败，若大于 0 表示读取数据长度。

4.11.2.6 KVF_Delete

表 4- 27 KVF_Delete

接口	说明
函数	int KVF_Delete(uint16_t id);
功能	从存储系统中删除指定 ID 数据。

接口	说明
参数	uint16_t id 存储 ID，若是系统存储则最高 bit 置位 1。
注意事项	无。
返回值	若为少于等于 0 表示读取失败，若大于 0 表示读取数据长度。

4.12 Version

4.12.1 简介

提供一组 API 获取系统各个 img 的版本。各 img 的 ID 如下。

```
typedef enum {  
    VER_TYPE_SYS                = 0, /* 会根据当前系统自动识别是 sys0 还是 sys1 */  
    VER_TYPE_SYS0               = 1,  
    VER_TYPE_SYS1               = 2,  
    VER_TYPE_MMW                = 3,  
    VER_TYPE_MMW_ALG            = 4,  
    VER_TYPE_MPW                = 5,  
    VER_TYPE_CUSTOM_CFG         = 6,  
    VER_TYPE_FLASH_LIST         = 7,  
    VER_TYPE_MAX  
} VER_SubType_t;
```

实现在 platform/utils/version.h。

通常子 img 版本为：

```
Major.minor.revision.build_num
```

Sys img 的版本由以下定义宏决定，可以在各个 prj_config.h 中定义应用的版本。

```
#ifndef CONFIG_PROJECT_VERSION_MAJOR  
#define CONFIG_PROJECT_VERSION_MAJOR 1  
#endif  
  
#ifndef CONFIG_PROJECT_VERSION_MINOR  
#define CONFIG_PROJECT_VERSION_MINOR 0  
#endif  
  
#ifndef CONFIG_PROJECT_VERSION_REVISION  
#define CONFIG_PROJECT_VERSION_REVISION 0  
#endif
```

```
#ifndef CONFIG_PROJECT_VERSION_BUILD
#define
```

merge img 的版本为：

```
R minor.type - M minor.project.number-V major.minor.revision
```

此版本通常由 POSSUMIC 提供。

4.12.2 接口

4.12.2.1 VER_GetSdkVersion

表 4- 28 VER_GetSubimgVersion

接口	说明
函数	int VER_GetSubimgVersion(VER_SubType_t verType, VER_SubVersion_t * pver);
功能	获取子 img 版本号。
参数	VER_SubType_t verType 子 img 版本类型。 typedef enum { VER_TYPE_SYS = 0, VER_TYPE_SYS0 = 1, VER_TYPE_SYS1 = 2, VER_TYPE_MMW = 3, VER_TYPE_MMW_ALG = 4, VER_TYPE_MPW = 5, VER_TYPE_CUSTOM_CFG = 6, VER_TYPE_FLASH_LIST = 7, VER_TYPE_MAX } VER_SubType_t; VER_SubVersion_t * pver 子 img 版本号。 typedef struct { uint8_t iv_major; uint8_t iv_minor; uint16_t iv_revision; uint32_t iv_build_num; } VER_SubVersion_t;
注意事项	无。
返回值	0 成功，非 0 表示失败。

4.12.2.2 VER_GetSubimgVersion

表 4- 29 VER_GetSdkVersion

接口	说明
函数	int VER_GetSdkVersion(VER_SdkVersion_t * pver);
功能	获取 SDK 版本。
参数	VER_SdkVersion_t * pver SDK 版本号。 定义如下： <pre>typedef struct { uint16_t ir_minor; uint8_t ir_type; uint8_t im_project; uint16_t im_number; uint16_t im_minor; uint8_t iv_major; uint8_t iv_minor; uint16_t iv_revision; } VER_SdkVersion_t;</pre>
注意事项	无。
返回值	0 成功，非 0 表示失败。

4.12.2.3 VER_GetBootVersion

表 4- 30 VER_GetBootVersion

接口	说明
函数	int VER_GetBootVersion(VER_BootVersion_t * pver);
功能	获取 bootloader 版本号。
参数	VER_BootVersion_t * pver bootloader 版本号。 定义如下： <pre>typedef struct { uint8_t iv_major; uint8_t iv_minor; uint16_t iv_revision; } VER_BootVersion_t;</pre>

接口	说明
注意事项	无。
返回值	0 成功，非 0 表示失败。

4.12.2.4 VER_GetBuildTime

表 4- 31 VER_GetBuildTime

接口	说明
函数	void VER_GetBuildTime(char *pdate, char *ptime);
功能	获取应用的编译时间
参数	char *pdate 编译日期，如 Aug 25 2025。 char *ptime 编译时间，如 09:22:53。
注意事项	无。
返回值	无。

4.13 HIF

4.13.1 功能简介

SDK 提供 HIF 子模块与其他设备通信，此模块实现了 HIF 协议格式和通信流程，主要分为三层：应用接口层、hif 层、物理接口层。其中包括 uart、spi 硬件通信接口，用户可通过宏开关选择相应的硬件接口通信。相关的时序要求和数据帧格式定义请结合《RS6x_7x_HIF 通信格式_说明手册》阅读。接口使用可参考 project/subsys/hif/hif_uart 工程。

4.13.2 宏定义开关

在使用 HIF 模块之前，需要在 prj_config.h 头文件中开启 HIF 总开关，然后选定相应的通信接口和唤醒方式。

表 4- 32 CONFIG_HIF

宏	说明
---	----

宏	说明
CONFIG_HIF	HIF 总开关
CONFIG_HIF_DEVICE_COM_UART	使能 HIF 通信口为串口
CONFIG_HIF_UART_PORT	选择 HIF 串口、若不定义则默认为 uart0
CONFIG_HIF_DEVICE_WAKEUP_UART	使能 HIF 串口唤醒
CONFIG_HIF_DEVICE_COM_SPI	使能 HIF 通信口为 SPI
CONFIG_HIF_DEVICE_WAKEUP_SPI	使能 HIF SPI 唤醒
CONFIG_HIF_SEND_DMA	使能 dma(仅支持 uart/spi)、若不是开启此宏定义，则默认使用中断模式，暂不支持中断模式。

4.13.3 接口说明

4.13.3.1 HIF_Init

表 4- 33 HIF_Init

接口	说明
函数	int HIF_Init(HIF_InitParam_t *pInitParam);
功能	HIF 模块初始化，调用此接口后会初始化 HIF 相关的资源。
参数	<pre>typedef enum { HIF_COM_TYPE_DIS = 0, HIF_COM_TYPE_UART = 1, HIF_COM_TYPE_IIC = 2, //暂不支持 HIF_COM_TYPE_SPI = 3, } HIF_ComType_t; typedef enum { HIF_NOTIFY_TYPE_DIS = 0, HIF_NOTIFY_TYPE_IO = 1, //拉高 io 通知 host 读取数据 HIF_NOTIFY_TYPE_DLY = 2, //延时一定时间后向 host 发送数据，除 uart 外，其他通信类型不建议使用 } HIF_NotifyType_t; typedef struct { HIF_ComType_t comType; //通信接口 uint32_t comRate;</pre>

接口	说明
	//通信频率 SPI：无需修改，支持 host 60M 以下频率访问；uart 支持 2M 以下常用波特率 uint32_t comParam; //通信参数 HIF_NotifyType_t notifyType; //通知类型 HIF_IOLevel_t notifyIoLevel; //通知 io 电平 uint32_t notifyIoNum; //通知 io 口 HIF_WakeType_t wakeType; //唤醒类型 HIF_IOLevel_t wakeIoLevel; //唤醒 io 电平 uint32_t wakeIoNum; //唤醒 io 口 uint32_t wakeComParam; //唤醒阈值 uint32_t recvFrameTo; //接收超时 uint32_t sendFrameTo; //发送超时 uint32_t sendDelay; //延时发送时间 uint32_t sleepTo; //睡眠超时 uint32_t splitSize; //分包大小，暂不支持 uint32_t splitEna; //分包使能，暂不支持 uint32_t simplex; //单工模式使能，暂不支持 uint32_t retry_enable; //hif 层重传使能，暂不支持 } HIF_InitParam_t;
注意事项	1、在调用此接口之前，可参照 HIF_DefaultInitParam 函数先给 pInitParam 参数赋值。 2、此接口不可重复调用。
返回值	等于 0 为初始化成功，其他值为失败。

4.13.3.2 HIF_DelInit

表 4- 34 HIF_DelInit

接口	说明
函数	int HIF_DelInit(void);
功能	HIF 模块反初始化，调用此接口后会释放 HIF 层相关资源
参数	无

接口	说明
注意事项	此接口不可重复调用。
返回值	0 为反初始化成功，其他值为失败。

4.13.3.3 HIF_DefaultInitParam

表 4- 35 HIF_DefaultInitParam

接口	说明
函数	int HIF_DefaultInitParam(HIF_InitParam_t * cfgParam, HIF_ComType_t comType, uint32_t comRate);
功能	HIF_InitParam_t 类型参数初始化为默认值，初始化后用于 HIF_Init。
参数	HIF_InitParam_t * cfgParam cfgParam 参数地址 HIF_ComType_t comType 通信类型 typedef enum { HIF_COM_TYPE_DIS = 0, HIF_COM_TYPE_UART = 1, HIF_COM_TYPE_IIC = 2, //暂不支持 HIF_COM_TYPE_SPI = 3, } HIF_ComType_t; uint32_t comRate 波特率
注意事项	无
返回值	0 为设置默认值成功，其他为失败。

4.13.3.4 HIF_ExtControl

表 4- 36 HIF_ExtControl

接口	说明
函数	int HIF_ExtControl(HIF_Attr_t attr, void * arg, uint32_t argSize);
功能	根据类型单独设置 HIF_InitParam_t 成员参数。
参数	HIF_Attr_t attr 要设置的参数类型

接口	说明
	<pre>typedef enum { HIF_SET_COM_TYPE = 0, HIF_SET_COM_RATE = 1, HIF_SET_COM_PARAM = 2, HIF_SET_NOTIFY_TYPE = 3, HIF_SET_NOTIFY_IO_LEVEL = 4, HIF_SET_NOTIFY_IO_NUM = 5, HIF_SET_WAKE_TYPE = 6, HIF_SET_WAKE_IO_LEVEL = 7, HIF_SET_WAKE_IO_NUM = 8, HIF_SET_WAKE_COM_PARAM = 9, HIF_SET_RECV_FRAME_TO = 10, HIF_SET_SEND_FRAME_TO = 11, HIF_SET_SEND_DELAY = 12, HIF_SET_SLEEP_TO = 13, HIF_SET_SPLIT_SIZE = 14, //暂不支持 HIF_SET_SPLIT_ENA = 15, //暂不支持 HIF_SET_SIMPLEX_MODE = 16, //暂不支持 HIF_SET_RETRY = 17, //暂不支持 } HIF_Attr_t;</pre> void * arg 参数指针 uint32_t argSize 参数长度
注意事项	在 HIF_DefaultInitParam 函数之后、HIF_Init 之前调用。
返回值	0 为设置成功，其他为失败。

4.13.3.5 HIF_ParamGet

表 4- 37 HIF_ParamGet

接口	说明
函数	int HIF_ParamGet(HIF_Attr_t attr, void * arg, uint32_t argSize);
功能	根据类型单独获取 HIF_InitParam_t 成员参数。
参数	HIF_Attr_t attr 要获取的参数类型 <pre>typedef enum {</pre>

接口	说明
	HIF_SET_COM_TYPE = 0, HIF_SET_COM_RATE = 1, HIF_SET_COM_PARAM = 2, HIF_SET_NOTIFY_TYPE = 3, HIF_SET_NOTIFY_IO_LEVEL = 4, HIF_SET_NOTIFY_IO_NUM = 5, HIF_SET_WAKE_TYPE = 6, HIF_SET_WAKE_IO_LEVEL = 7, HIF_SET_WAKE_IO_NUM = 8, HIF_SET_WAKE_COM_PARAM = 9, HIF_SET_RECV_FRAME_TO = 10, HIF_SET_SEND_FRAME_TO = 11, HIF_SET_SEND_DELAY = 12, HIF_SET_SLEEP_TO = 13, HIF_SET_SPLIT_SIZE = 14, //暂不支持 HIF_SET_SPLIT_ENA = 15, //暂不支持 HIF_SET_SIMPLEX_MODE = 16, //暂不支持 HIF_SET_RETRY = 17, //暂不支持 } HIF_Attr_t; void * arg 参数指针 uint32_t argSize 参数长度
注意事项	无
返回值	0 为获取成功，其他为失败。

4.13.3.6 HIF_MsgHdl_Regist

表 4- 38 HIF_MsgHdl_Regist

接口	说明
函数	int HIF_MsgHdl_Regist(uint8_t msgId, HIF_MsgHdl_t msgHdl);
功能	通过 msgId 注册相应的回调函数，注册成功后，当 device 接收到此 msgID 的合法 HIF 数据帧之后会调用此接口注册的回调函数。
参数	uint8_t msgId Msg ID

接口	说明
	HIF_MsgHdl_t msgHdl 回调函数：为空时相当于 unregister typedef int (* HIF_MsgHdl_t)(HIF_MsgHdr_t *msg);
注意事项	无。
返回值	0 为注册成功，其他为失败。

4.13.3.7 HIF_MsgResp

表 4- 39 HIF_MsgResp

接口	说明
函数	int HIF_MsgResp(HIF_MsgHdr_t *msg, uint16_t dataLen, uint8_t status);
功能	接收到 host 指令后通过此接口进行回复。当 dataLen 为 0 时,payload 长度为 1,payload 内容为 status；当 dataLen 不为 0 时，通过 HIF_Msg_AckDataPtr 接口获取回复数据的地址，然后将回复数据拷贝到此地址即可。
参数	HIF_MsgHdr_t *msg HIF 帧头，详细信息请查阅 host 驱动开发手册 typedef struct hif_message_header_t { uint32_t type : 2; uint32_t flag : 6; uint32_t msg_id : 8; uint32_t length : 12; uint32_t seq : 3; uint32_t frag : 1; } HIF_MsgHdr_t uint16_t dataLen 数据长度 uint8_t status 回复状态
注意事项	此接口形参 msg 用回调函数的形参 msg 即可。 示例： int ft_voltage_handle(HIF_MsgHdr_t *msg) { uint8_t *ack_data = (uint8_t *)HIF_Msg_AckDataPtr(msg); uint32_t adc_val = mp_ctrl_adc_read_mv(); memcpy(ack_data, &adc_val, sizeof(adc_val)); }

接口	说明
	<pre>return HIF_MsgResp(msg, sizeof(ack_data), status); }</pre>
返回值	0 为注册成功，其他为失败。

4.13.3.8 HIF_MsgReport

表 4- 40 HIF_MsgReport

接口	说明
函数	<pre>int HIF_MsgReport(HIF_MsgHdr_t *msg, void *data, HIF_MsgTrans_Callback sentCb);</pre>
功能	将上报数据添加到发送队列。
参数	HIF_MsgHdr_t *msg HIF 帧头，详细信息请查阅 host 驱动开发手册 <pre>typedef struct hif_message_header_t { uint32_t type : 2; uint32_t flag : 6; uint32_t msg_id : 8; uint32_t length : 12; uint32_t seq : 3; uint32_t frag : 1; } HIF_MsgHdr_t</pre> void *data 上报数据 HIF_MsgTrans_Callback sentCb 数据上报回调函数：当回调函数不为空时，异步发送；当回调函数为空时，阻塞发送 <pre>typedef void (* HIF_MsgTrans_Callback)(hif_queue_item_t *hif_queue_item, int8_t status);</pre>
注意事项	1、数据发送完成或者发送失败会调用注册的回调函数，用户应根据回调函数的 status 判断是否发送成功。
返回值	0 为加入发送队列成功，其他为失败。

4.13.3.9 HIF_MsgReport_ListAddData

表 4- 41 HIF_MsgReport_ListAddData

接口	说明
函数	<pre>payload_node_t *HIF_MsgReport_ListAddData(payload_node_t * payload_head, void *data, uint16_t len);</pre>

接口	说明
功能	按块添加 payload 上报数据
参数	payload_node_t * payload_head Payload 头节点指针，第一个 payload 时传入 NULL，返回值会返回一个申请成功的头节点 void *data 要发送的数据地址 uint16_t len 要发送的数据大小
注意事项	无。
返回值	返回 payload head 指针。

4.13.3.10 :HIF_MsgReport_ListStart

表 4- 42 HIF_MsgReport_ListStart

接口	说明
函数	int HIF_MsgReport_ListStart(HIF_MsgHdr_t *msg, payload_node_t * payload_head, HIF_MsgTrans_Callback sentCb);
功能	按块添加 payload 完成后，开始发送。
参数	HIF_MsgHdr_t *msg HIF 帧头，详细信息请查阅 host 驱动开发手册 payload_node_t * payload_head 添加 payload 时申请的头节点地址 HIF_MsgTrans_Callback sentCb 数据上报回调函数：当回调函数不为空时，异步发送；当回调函数为空时，阻塞发送。
注意事项	1、数据发送完成或者发送失败会调用注册的回调函数，用户应根据回调函数的 status 判断是否发送成功。
返回值	0 为加入发送队列成功，其他为失败。

4.14 Other

SDK 还提供了一些通用的接口，如获取系统时钟，获取系统频率等。

4.14.1 接口

4.14.1.1 HAL_BOARD_GetFreq

表 4-43 HAL_BOARD_GetFreq

接口	说明
函数	int32_t HAL_BOARD_GetFreq(CLOCK_Type_t clkType);
功能	获取系统频率。
参数	CLOCK_Type_t clkType 频率类型。 <pre>typedef enum { CLOCK_LSI = 0, CLOCK_MSI = 1, CLOCK_LSE = 2, CLOCK_DCXO = 3, CLOCK_LPCLK = 4, CLOCK_LPCLK_INTE_DIV = 5, CLOCK_LPCLK_FRAC_DIV = 6, CLOCK_PLL = 7, CLOCK_PLL_SOC = 8, CLOCK_PLL_DEV = 9, CLOCK_PLL_DEV1 = 10, CLOCK_PLL_CDAA = 11, CLOCK_SYS = 12, CLOCK_AHB = 13, CLOCK_APB0 = 14, CLOCK_APB1 = 15, CLOCK_CURR_CPU = 16, CLOCK_CPUS = 17, CLOCK_CPUF = 18, CLOCK_MAX = 19 } CLOCK_Type_t;</pre>
注意事项	无。
返回值	大于 0 为系统频率，少于 0 表示获取失败，0 表示当前类型的时钟未开启。

4.14.1.2 HAL_BOARD_GetTime

表 4-44 HAL_BOARD_GetTime

接口	说明
函数	uint32_t HAL_BOARD_GetTime(HAL_TimeType_t timeType);
功能	获取系统时钟。
参数	HAL_TimeType_t timeType 时钟类型。 <pre>typedef enum { HAL_TIME_CYCLE = 0, // AHB 时钟 cycle HAL_TIME_US = 1, // 微秒 HAL_TIME_MS = 2, // 毫秒 HAL_TIME_S = 3, // 秒 } HAL_TimeType_t;</pre>
注意事项	可用于系统软件运行时间统计。
返回值	大于等于 0 为系统频率，少于 0 表示获取失败。

4.14.1.3 HAL_BOARD_BlDelay

表 4-45 HAL_BOARD_BlDelay

接口	说明
函数	void HAL_BOARD_BlDelay(uint32_t delayVal, HAL_TimeType_t timeType);
功能	阻塞式延时。
参数	uint32_t delayVal 延时值。 HAL_TimeType_t timeType 延时单位。
注意事项	延迟不会让出 CPU，在高优先级使用时可能阻止低优先级调度。
返回值	无

4.14.1.4 HAL_BOARD_GetRand

表 4- 46 HAL_BOARD_GetRand

接口	说明
函数	uint32_t HAL_BOARD_GetRand(void);
功能	获取随机数。
参数	无
注意事项	无
返回值	随机数。

4.14.1.5 HAL_BOARD_Reset

表 4- 47 HAL_BOARD_Reset

接口	说明
函数	void HAL_BOARD_Reset(HAL_ResetType_t resetType);
功能	复位系统。
参数	HAL_ResetType_t resetType 复位类型。 typedef enum { HAL_RESET_CPU = 0, // 仅复位 CPU HAL_RESET_SYS = 1, // 复位 CPU 和其他外设 } HAL_ResetType_t;
注意事项	无
返回值	随机数。

5 外设驱动

本章节主要介绍片上外设驱动，帮助用户快速上手开发片上支持的外设资源；

5.1 GPIO

GPIO(General Purpose Input Output)是 RS 系列 SOC 基本输入输出模块。SOC 以 pin 为基本控制单元，每个 pin 可选择支持输入/输出/复用等功能，多个 pin 可组成一组如 Port-A/Port-B。SOC 支持的 pin 个数和各 pin 支持的功能参考对应 SOC 的 datasheet。

例如：

RS6240 SOC 参考文档 SDK/01_Document/RS6240_Datasheet_V1.3_EN.pdf 的章节 4.3 GPIO_Multiplexing。

RS6240 SOC 支持 A 和 B 两组 GPIO，A 组支持 16 个 pin，B 组支持 8 个 pin。

RS6240 SOC 的每个 pin 支持最多 16 类功能可选择，同一时刻只能选择一种。

5.1.1 功能简介

GPIO 单个 pin 的基本结构示意图如下图。

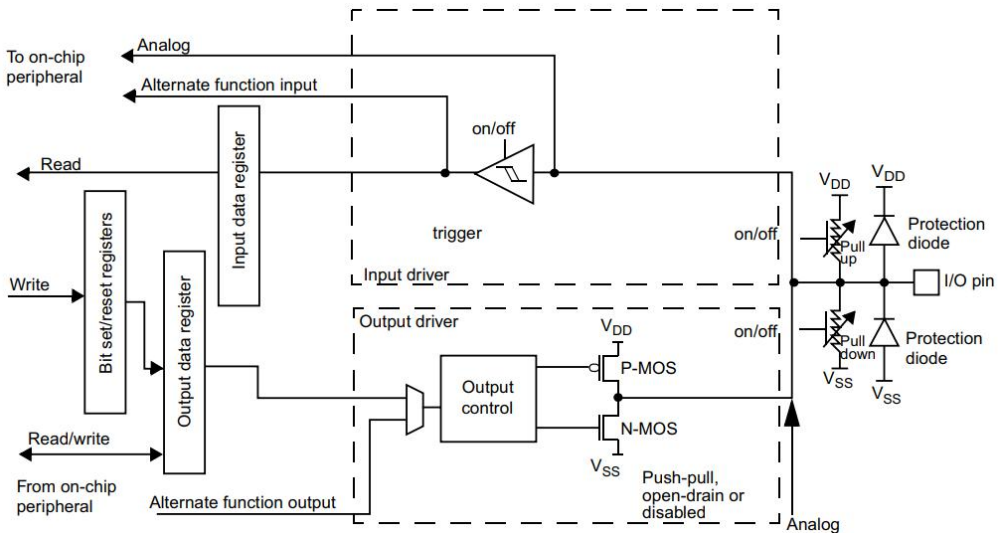


图 5-1 GPIO 框架示意图

5.1.1.1 上下拉电阻

在 SOC 内部设计有软件可独立控制的上拉电阻和下拉电阻，其阻值通常在 50~100 千欧左右（具体以 datasheet 为准，参考 Digital IO Characteristics 章节）。

内部上下拉电阻通常用在输入模式和输入型复用功能模式，用于维持特定电平。当上拉电阻和下拉电阻都关闭时即浮空输入，浮空输入时内部电平不确定容易造成干扰，因此类似 UART Rx 需配置成上拉模式。输出模式也有少量场景使用上下拉电阻，例如 I2C，输出为开漏，上拉电阻实现线与功能。

上下拉电阻选择对应 API 参数如下：

```
typedef enum {
    GPIO_PULL_FLOATING = 0U,    /* 关闭内部上拉电阻和下拉电阻 */
    GPIO_PULL_UP        = 1U,    /* 使能内部上拉电阻 */
    GPIO_PULL_DOWN      = 2U    /* 使能内部下拉电阻 */
} GPIO_Pull_t;
```

5.1.1.2 输出驱动能力

输出功能由 P-MOS 和 N-MOS 驱动，当 P-MOS 和 N-MOS 同时工作驱动时为推挽模式，当只有 N-MOS 工作 P-MOS 不工作时为开漏模式。SOC 为推挽模式设置了 4 档驱动能力，对应 20mA~80mA(3.3V)/6mA~24mA(1.8V)的电流驱动能力（具体以 datasheet 为准，参考 GPIO 章节，不同电压下其驱动步进不一样）。不同通信速率对信号建立时间要求不同，驱动电流越大信号建立时间越快，但并不是驱动能力越强越好，匹配即可。开漏模式驱动能力主要受上拉电阻影响。

输出模式选择对应 API 参数如下：

```
typedef enum {
    GPIO_PUSH_PULL      = 0U,    /* 推挽输出 */
    GPIO_OPEN_DRAIN     = 1U    /* 开漏输出 */
} GPIO_Push_t;
```

推挽输出模式驱动能力选择对应 API 参数如下：

```
typedef enum {
    GPIO_DRIVING_LEVEL_0 = 0U,    /* 约 20mA(3.3V) / 6mA(1.8V) 输出能力 */
    GPIO_DRIVING_LEVEL_1 = 1U,    /* 约 40mA(3.3V) / 12mA(1.8V) 输出能力 */
    GPIO_DRIVING_LEVEL_2 = 2U,    /* 约 60mA(3.3V) / 18mA(1.8V) 输出能力 */
    GPIO_DRIVING_LEVEL_3 = 3U    /* 约 80mA(3.3V) / 24mA(1.8V) 输出能力 */
} GPIO_Driving_t;
```

5.1.1.3 功能模式

如上图，每个 IO Pin 按照功能模式可分为：通用 IO 输出模式、通用 IO 输入模式、通用 IO 中断模式、复用功能模式和关闭模式。其中复用功能包括多种，例 UART, SPI, PWM 等。通常同一时刻只有输入通路或输出通路工作，如通用 IO 输出模式时输入功能是关闭的，但在通信复用功能时输入通路和输出通路是同时工作的，如 I2C 通信。

模式选择对应 API 参数如下：

```
typedef enum {
    HAL_WKIO_MODE_GPIO      = 0U,    /* gpio */
    HAL_WKIO_MODE_HOLD      = 1U,    /* output */
    HAL_WKIO_MODE_WAKE      = 2U,    /* input wake-up */
    HAL_WKIO_MODE_DISA      = 3U    /* disable */
}
```

```
} WKIO_PinMode_t;
```

5.1.1.4 GPIO与电源管理WKIO

SOC 设计时为满足低功耗需求，通常会设计进入休眠时关闭部分功能的电源和时钟。RS 系列一些 SOC 的 GPIO 在休眠时会关闭供电，WKIO 模块是 GPIO 掉电后的简化功能。

WKIO 只有保持输出电平和通过 IO 输入唤醒功能，因此其参数配置有上下拉电阻配置和唤醒电平/沿配置（输出模式为推挽，驱动能力为 LEVEL0）。

由于 WKIO 为 GPIO 前一级，只有将 WKIO 配置为 GPIO 模式，GPIO 功能才有效。

并不是每一个 GPIO 都有对应的 WKIO，例如 RS6240 其只有 Port-A 有 WKIO，Port-B 没有 WKIO，具体请参考 datasheet。

```
typedef enum {
    GPIOx_Pn_F0_INPUT           = 0U,    /* 通用 IO 输入模式 */
    GPIOx_Pn_F1_OUTPUT          = 1U,    /* 通用 IO 输出模式 */
    GPIOx_Pn_F14_EINT           = 14U,   /* 通用 IO 输入中断模式 */
    GPIOx_Pn_F15_DIS            = 15U,   /* 关闭模式，即输入输出功能都关闭 */

    /* 其它复用功能模式 */
} GPIO_Mode_t;
```

由于 WKIO 使用的是低速时钟，因此修改其配置不会立即生效，存在一定延迟（延迟由其功能时钟决定，如使用 LCLK_FDIV，通常最多会延迟 30us 左右）。

5.1.1.5 Debounce

在 GPIO 的 Interrupt 模式以及 WKIO 的 wake-up 模式（其它模式无效），由于 IO 的抖动会触发多次中断或唤醒造成干扰，因此增加 Debounce 软件消抖。每组 GPIO 和每组 WKIO 有独立的 Debounce。

Debounce 的原理是使用功能时钟连续采样 N 次，若 N 次采样数据一致则输出给后一级电路，若跳变则记录新的采样值并重新计数，计数达到时输出记录值至后级电路。以下举例说明：

- 1. Debounce 配置为 N，低电平触发，那么连续采集到 N 次低电平则触发一次，若 N 次中有几次高电平则恢复低电平后重新计数，直至连续采集到 N 次低电平触发中断或唤醒。
- 2. Debounce 配置为 N，下降沿触发，那么连续采集到 N 次高电平，紧接着采集到 N 次低电平则触发一次中断或唤醒。

5.1.2 接口

5.1.2.1 HAL_GPIO_Init

表 5- 1 HAL_GPIO_Init

接口	说明
函数	HAL_Dev_t * HAL_GPIO_Init(GPIO_Port_t id);
功能	初始化一组 GPIO 设备，申请系统资源并开启时钟。
参数	GPIO_Port_t id GPIO 外设 ID，支持的 Port 由 SOC 决定； <pre>typedef enum { #if (CONFIG_SOC_GPIOA == 1) GPIO_PORT_A = 0U, #endif #if (CONFIG_SOC_GPIOB == 1) GPIO_PORT_B = 1U, #endif #if (CONFIG_SOC_GPIOC == 1) GPIO_PORT_C = 2U, #endif GPIO_PORT_NUM, } GPIO_Port_t;</pre>
注意事项	1. 初始化内部设计了防呆，反复初始化不会 Reset GPIO，仅返回控制句柄； 2. 每次初始化只初始化一组 GPIO，若 SOC 有两组 GPIO POART-A 和 POART-B 则需分别初始化； 3. 系统默认在 Board.c 中初始化了全部组 GPIO；
返回值	GPIO 控制句柄，若初始化失败则返回 NULL。

5.1.2.2 HAL_GPIO_DeInit

表 5- 2 HAL_GPIO_DeInit

接口	说明
函数	void HAL_GPIO_DeInit(HAL_Dev_t *gpio);
功能	反初始化 GPIO。
参数	HAL_Dev_t *gpio GPIO 句柄
注意事项	1. 反初始化前需确认各模块不再使用此组 GPIO，通常不使用此 API，；

接口	说明
返回值	无。

5.1.2.3 HAL_GPIO_SetPinParam

表 5- 3 HAL_GPIO_SetPinParam

接口	说明
函数	HAL_Status_t HAL_GPIO_SetPinParam(HAL_Dev_t *gpio, GPIO_Pin_t pin, GPIO_PinParam_t *param);
功能	设置 GPIO 对应 pin 的配置参数。
参数	HAL_Dev_t *gpio GPIO 控制句柄，通过 HAL_GPIO_Init 或 HAL_DEV_Find 返回值获得。 GPIO_Pin_t pin GPIO 的 pin 值。 GPIO_PinParam_t *param 设置配置参数 typedef struct { GPIO_Mode_t mode; // 功能模式，同 HAL_GPIO_SetPinMode GPIO_Pull_t pull; // 上拉电阻和下拉电阻配置，同 HAL_GPIO_SetPinPull GPIO_Push_t push; // 输出模式配置，同 HAL_GPIO_SetPinPush GPIO_Driving_t driving; // 输出驱动能力配置，同 HAL_GPIO_SetPinDriving } GPIO_PinParam_t;
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.1.2.4 HAL_GPIO_GetPinParam

表 5- 4 HAL_GPIO_GetPinParam

接口	说明
函数	HAL_Status_t HAL_GPIO_GetPinParam(HAL_Dev_t *gpio, GPIO_Pin_t pin, GPIO_PinParam_t *param);
功能	获取 GPIO 对应 pin 的配置参数。

接口	说明
参数	HAL_Dev_t *gpio GPIO 控制句柄，通过 HAL_GPIO_Init 或 HAL_DEV_Find 返回值获得。 GPIO_Pin_t pin GPIO 的 pin 值。 GPIO_PinParam_t *param 设置配置参数 <pre>typedef struct { GPIO_Mode_t mode; // 功能模式，同 HAL_GPIO_SetPinMode GPIO_Pull_t pull; // 上拉电阻和下拉电阻配置，同 HAL_GPIO_SetPinPull GPIO_Push_t push; // 输出模式配置，同 HAL_GPIO_SetPinPush GPIO_Driving_t driving; // 输出驱动能力配置，同 HAL_GPIO_SetPinDriving } GPIO_PinParam_t;</pre>
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.1.2.5 HAL_GPIO_DisablePin

表 5- 5 HAL_GPIO_DisablePin

接口	说明
函数	HAL_Status_t HAL_GPIO_DisablePin(HAL_Dev_t *gpio, GPIO_Pin_t pin);
功能	关闭 GPIO 对应 pin。
参数	HAL_Dev_t *gpio GPIO 控制句柄，通过 HAL_GPIO_Init 或 HAL_DEV_Find 返回值获得。 GPIO_Pin_t pin GPIO 的 pin 值。
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.1.2.6 HAL_GPIO_SetPinMode

表 5-6 HAL_GPIO_SetPinMode

接口	说明
函数	HAL_Status_t HAL_GPIO_SetPinMode(HAL_Dev_t *gpio, GPIO_Pin_t pin, GPIO_Mode_t mode);
功能	设置 GPIO 对应 pin 的功能模式。
参数	HAL_Dev_t *gpio GPIO 控制句柄，通过 HAL_GPIO_Init 或 HAL_DEV_Find 返回值获得。 GPIO_Pin_t pin GPIO 的 pin 值。 GPIO_Mode_t mode typedef enum { GPIOx_Pn_F0_INPUT = 0U, // 通用输入模式 GPIOx_Pn_F1_OUTPUT = 1U, // 通用输出模式 GPIOx_Pn_F14_EINT = 14U, // 中断模式 GPIOx_Pn_F15_DIS = 15U, // 关闭模式 /* 其它复用功能 */ } GPIO_Mode_t;
注意事项	1. 关闭模式下，输入通路和输出通路都关闭，上拉电阻和下拉电阻关闭； 2. 对应 pin 的 GPIO 模式必须在 WKIO 处于 HAL_WKIO_MODE_GPIO 才有效，见 HAL_WKIO_SetWakeParam；
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.1.2.7 HAL_GPIO_SetPinPull

表 5-7 HAL_GPIO_SetPinPull

接口	说明
函数	HAL_Status_t HAL_GPIO_SetPinPull(HAL_Dev_t *gpio, GPIO_Pin_t pin, GPIO_Pull_t pull);
功能	设置 GPIO 对应 pin 的上拉电阻或下拉电阻。
参数	HAL_Dev_t *gpio GPIO 控制句柄，通过 HAL_GPIO_Init 或 HAL_DEV_Find 返回值获得。

接口	说明
	GPIO_Pin_t pin GPIO 的 pin 值。 GPIO_Pull_t pull typedef enum { GPIO_PULL_FLOATING = 0U, // 上拉电阻和下拉电阻都关闭 /* 35Kohm ~ 75Kohm (3.3V) / 70K ohm ~150K ohm (1.8V) */ GPIO_PULL_UP = 1U, // 使能上拉电阻，下拉电阻关闭 /* 30Kohm ~ 75Kohm (3.3V) / 60K ohm ~150K ohm (1.8V) */ GPIO_PULL_DOWN = 2U // 使能下拉电阻，上拉电阻关闭 } GPIO_Pull_t;
注意事项	1. 上拉电阻和下拉电阻通常在输入时配置，且当配置成浮空时，由于电路的不确定态容易造成干扰； 2. 开漏输出时也可以配置上拉； 上拉的阻值若不满足通信需求则需要外部再接上拉电阻；
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.1.2.8 HAL_GPIO_SetPinPush

表 5- 8 HAL_GPIO_SetPinPush

接口	说明
函数	HAL_Status_t HAL_GPIO_SetPinPush(HAL_Dev_t *gpio, GPIO_Pin_t pin, GPIO_Push_t push);
功能	设置 GPIO 对应 pin 的输出模式。
参数	HAL_Dev_t *gpio GPIO 控制句柄，通过 HAL_GPIO_Init 或 HAL_DEV_Find 返回值获得。 GPIO_Pin_t pin GPIO 的 pin 值。 GPIO_Push_t push typedef enum { GPIO_PUSH_PULL = 0U, // 推挽输出 GPIO_OPEN_DRAIN = 1U // 开漏输出

接口	说明
	} GPIO_Push_t;
注意事项	1. 开漏输出常用于 IIC 的线与功能; 2. 开漏输出也可用于大电流驱动能力输出;
返回值	返回值为 HAL_STATUS_OK, 表示函数执行成功; 返回值为负数表示函数执行失败, 具体参考 HAL_STATUS_*。

5.1.2.9 HAL_GPIO_SetPinDriving

表 5- 9 HAL_GPIO_SetPinDriving

接口	说明
函数	HAL_Status_t HAL_GPIO_SetPinDriving(HAL_Dev_t *gpio, GPIO_Pin_t pin, GPIO_Driving_t drinving);
功能	设置 GPIO 对应 pin 的输出驱动能力。
参数	HAL_Dev_t *gpio GPIO 控制句柄, 通过 HAL_GPIO_Init 或 HAL_DEV_Find 返回值获得。 GPIO_Pin_t pin GPIO 的 pin 值。 GPIO_Driving_t drinving 驱动能力配置。 <pre>typedef enum { GPIO_DRIVING_LEVEL_0 = 0U, /* 20mA(3.3V) / 6mA(1.8V) */ GPIO_DRIVING_LEVEL_1 = 1U, /* 40mA(3.3V) / 12mA(1.8V) */ GPIO_DRIVING_LEVEL_2 = 2U, /* 60mA(3.3V) / 18mA(1.8V) */ GPIO_DRIVING_LEVEL_3 = 3U /* 80mA(3.3V) / 24mA(1.8V) */ } GPIO_Driving_t;</pre>
注意事项	1. 驱动能力在输出模式为推挽时有效; 2. 输出驱动能力影响 IO 信号的建立时间, 当使用高速时需要增加驱动能力, 例如 SPI 速度大于 64MHz 可将驱动能力配置成 LEVEL_1。
返回值	返回值为 HAL_STATUS_OK, 表示函数执行成功; 返回值为负数表示函数执行失败, 具体参考 HAL_STATUS_*。

5.1.2.10 HAL_GPIO_ReadPort

表 5- 10 HAL_GPIO_ReadPort

接口	说明
函数	int32_t HAL_GPIO_ReadPort(HAL_Dev_t *gpio);
功能	获取 GPIO 所有 pin 输入电平
参数	HAL_Dev_t *gpio GPIO 控制句柄，通过 HAL_GPIO_Init 或 HAL_DEV_Find 返回值获得。
注意事项	若 pin 配置成非 input 模式，其对应 bit 为 0，不表示其 pin 为低电平。
返回值	GPIO 输入模式的电平或状态。 返回值为 0 表示对应 IO 为低电平； 返回值为 1 表示对应 IO 为高电平； 返回值为负表示获取失败，具体参考 HAL_STATUS_*；

5.1.2.11 HAL_GPIO_WritePort

表 5- 11 HAL_GPIO_WritePort

接口	说明
函数	HAL_Status_t HAL_GPIO_WritePort(HAL_Dev_t *gpio, uint32_t pinMask);
功能	写 GPIO 所有 pin 输出对应电平
参数	HAL_Dev_t *gpio GPIO 控制句柄，通过 HAL_GPIO_Init 或 HAL_DEV_Find 返回值获得。 uint32_t pinMask GPIO 的输出电平值，对应 bit 为 1 则输出高电平，为 0 则输出低电平。
注意事项	只对对应 pin 配置成 output 模式有效，若配置成其它模式无论 0/1 都无效。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.1.2.12 HAL_GPIO_ReadPin

表 5- 12 HAL_GPIO_ReadPin

接口	说明
函数	int32_t HAL_GPIO_ReadPin(HAL_Dev_t *gpio, GPIO_Pin_t pin);

接口	说明
功能	获取对应 pin 的输入电平。
参数	HAL_Dev_t *gpio GPIO 控制句柄，通过 HAL_GPIO_Init 或 HAL_DEV_Find 返回值获得。 GPIO_Pin_t pin GPIO 的 pin 值。
注意事项	需配合上下拉 API 一起使用 HAL_GPIO_SetPinPull。
返回值	返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*； 返回值为 0 表示对应 pin 为低电平； 返回值为 1 表示对应 pin 为高电平；

5.1.2.13 HAL_GPIO_WritePin

表 5- 13 HAL_GPIO_WritePin

接口	说明
函数	HAL_Status_t HAL_GPIO_WritePin(HAL_Dev_t *gpio, GPIO_Pin_t pin, GPIO_PinState_t PinState);
功能	对应 pin 输出指定电平。
参数	HAL_Dev_t *gpio GPIO 控制句柄，通过 HAL_GPIO_Init 或 HAL_DEV_Find 返回值获得。 GPIO_Pin_t pin GPIO 的 pin 值。 GPIO_PinState_t PinState typedef enum { GPIO_PIN_RESET = 0U, // 输出低电平 GPIO_PIN_SET // 输出高电平 } GPIO_PinState_t;
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.1.2.14 HAL_GPIO_TogglePin

表 5- 14 HAL_GPIO_TogglePin

接口	说明
函数	HAL_Status_t HAL_GPIO_TogglePin(HAL_Dev_t *gpio, GPIO_Pin_t pin);
功能	对应 pin 电平反转输出。
参数	HAL_Dev_t *gpio GPIO 控制句柄，通过 HAL_GPIO_Init 或 HAL_DEV_Find 返回值获得。 GPIO_Pin_t pin GPIO 的 pin 值。
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.1.2.15 HAL_GPIO_SetIRQParam

表 5- 15 HAL_GPIO_SetIRQParam

接口	说明
函数	HAL_Status_t HAL_GPIO_SetIRQParam(HAL_Dev_t *gpio, GPIO_Pin_t pin, GPIO_IrqParam_t *param);
功能	设置对应 pin 的中断模式参数。
参数	HAL_Dev_t *gpio GPIO 控制句柄，通过 HAL_GPIO_Init 或 HAL_DEV_Find 返回值获得。 GPIO_Pin_t pin GPIO 的 pin 值。 GPIO_IrqParam_t *param 中断参数配置 <pre>typedef struct { GPIO_IrqEvent_t event; // 中断触发模式 GPIO_IrqCallback_t callback; // 中断触发回调函数 void *arg; // 中断参数，回调函数执行时传回 } GPIO_IrqParam_t;</pre> <pre>typedef enum {</pre>

接口	说明
	<pre>GPIO_IRQ_EVT_RISING_EDGE = 0U, // 上升沿触发 GPIO_IRQ_EVT_FALLING_EDGE = 1U, // 下降沿触发 GPIO_IRQ_EVT_BOTH_EDGE = 2U, // 双边沿触发 GPIO_IRQ_EVT_HIGH_LEVEL = 3U, // 高电平触发 GPIO_IRQ_EVT_LOW_LEVEL = 4U, // 低电平触发 } GPIO_IrqEvent_t; HAL_Dev_t * HAL_GPIO_Init(GPIO_Port_t id); // 回调函数类型</pre>
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.1.2.16 HAL_GPIO_SetIRQDebounce

表 5- 16 HAL_GPIO_SetIRQDebounce

接口	说明
函数	<pre>HAL_Status_t HAL_GPIO_SetIRQDebounce(HAL_Dev_t *gpio, uint32_t debounce);</pre>
功能	设置对应 Port 所有 pin 的中断模式消抖值。
参数	<pre>uint32_t debounce</pre> 消抖计数值，采样时钟为 GPIO 功能时钟。 GPIO 默认功能时钟为 AHB，在 HAL_GPIO_Init 中配置。
注意事项	消抖时间需要功能时钟频率与 Debounce 配合使用，例如：功能时钟频率为 32MHz，Debounce 值为 32000000 则消抖时间为 1 秒。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.1.2.17 HAL_GPIO_EnableIRQ

表 5- 17 HAL_GPIO_EnableIRQ

接口	说明
函数	<pre>HAL_Status_t HAL_GPIO_EnableIRQ(HAL_Dev_t *gpio, GPIO_Pin_t pin);</pre>
功能	开启对应 pin 的中断功能，触发方式按照 HAL_GPIO_SetIRQParam 进行。

接口	说明
参数	HAL_Dev_t *gpio GPIO 控制句柄，通过 HAL_GPIO_Init 或 HAL_DEV_Find 返回值获得。 GPIO_Pin_t pin GPIO 的 pin 值。
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.1.2.18 HAL_GPIO_DisableIRQ

表 5- 18 HAL_GPIO_DisableIRQ

接口	说明
函数	HAL_Status_t HAL_GPIO_DisableIRQ(HAL_Dev_t *gpio, GPIO_Pin_t pin, int8_t ClearPending);
功能	关闭 GPIO 对应 pin 的中断模式。
参数	HAL_Dev_t *gpio GPIO 控制句柄，通过 HAL_GPIO_Init 或 HAL_DEV_Find 返回值获得。 GPIO_Pin_t pin GPIO 的 pin 值。 Int8_t ClearPending 是否清除中断 pending 标志，0 不清除，1 清除。
注意事项	此 API 只是关闭对应 pin 的中断，中断模式相关配置并不会丢失。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.1.2.19 HAL_WKIO_Init

表 5- 19 HAL_WKIO_Init

接口	说明
函数	HAL_Dev_t *HAL_WKIO_Init(WKIO_Port_t id);
功能	初始化 WKIO 驱动。

接口	说明
参数	WKIO_Port_t id 设备 ID，如下图，CONFIG_SOC_WKIOA 等宏见 rsxxx.h，如 rs6240.h <pre>typedef enum { #if (CONFIG_SOC_WKIOA == 1) WKIO_PORT_A = 0U, #endif #if (CONFIG_SOC_WKIOB == 1) WKIO_PORT_B = 1U, #endif WKIO_PORT_NUM } WKIO_Port_t;</pre>
注意事项	1. 初始化内部设计了防呆，反复初始化不会 Reset WKIO，仅返回控制句柄； 2. 每次初始化只初始化一组 WKIO，若 SOC 有两组 WKIO POART-A 和 POART-B 则需分别初始化； 3. 与 GPIO 不同，系统默认不初始化 WKIO；
返回值	返回值为 NULL，表示初始化失败； 返回值为非 NULL，表示 WKIO 控制句柄。

5.1.2.20 HAL_WKIO_DeInit

表 5- 20 HAL_WKIO_DeInit

接口	说明
函数	void HAL_WKIO_DeInit(HAL_Dev_t *wkio);
功能	反初始化 WKIO 驱动。
参数	HAL_Dev_t *wkio 唤醒 IO 控制句柄，通过 HAL_WKIO_Init 或 HAL_DEV_Find 返回值获得。
注意事项	反初始化内部设计了防呆，反复调用无影响。
返回值	无

5.1.2.21 HAL_WKIO_SetPinMode

表 5- 21 HAL_WKIO_SetPinMode

接口	说明
函数	HAL_Status_t HAL_WKIO_SetPinMode(HAL_Dev_t *wkio, WKIO_Pin_t pin, WKIO_PinMode_t mode);

接口	说明
功能	设置 WKIO 对应 pin 的工作模式。
参数	HAL_Dev_t *wkio WKIO 控制句柄，通过 HAL_WKIO_Init 或 HAL_DEV_Find 返回值获得。 WKIO_Pin_t pin WKIO 的 pin 值。 WKIO_PinMode_t mode 设置工作模式： typedef enum { HAL_WKIO_MODE_GPIO = 0U, // 通用 IO 模式，即 GPIO HAL_WKIO_MODE_HOLD = 1U, // 输出保持模式，soc 内部休眠电平保持 HAL_WKIO_MODE_WAKE = 2U, // 输入唤醒模式，可唤醒 soc 自休眠 HAL_WKIO_MODE_DISA = 3U // IO 彻底关闭 } WKIO_PinMode_t;
注意事项	只有当 mode 配置成 HAL_WKIO_MODE_GPIO 时，对应 pin 的 GPIO 配置才有效。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.1.2.22 HAL_WKIO_GetPinMode

表 5- 22 HAL_WKIO_GetPinMode

接口	说明
函数	HAL_Status_t HAL_WKIO_GetPinMode(HAL_Dev_t *wkio, WKIO_Pin_t pin, WKIO_PinMode_t *mode);
功能	获取 WKIO 对应 pin 的工作模式
参数	HAL_Dev_t *wkio WKIO 控制句柄，通过 HAL_WKIO_Init 或 HAL_DEV_Find 返回值获得。 WKIO_Pin_t pin WKIO 的 pin 值。 WKIO_PinMode_t mode 获取 pin 的工作模式： typedef enum {

接口	说明
	<pre>HAL_WKIO_MODE_GPIO = 0U, // 通用 IO 模式，即 GPIO HAL_WKIO_MODE_HOLD = 1U, // 输出保持模式，soc 内部休眠电平保持 HAL_WKIO_MODE_WAKE = 2U, // 输入唤醒模式，可唤醒 soc 自休眠 HAL_WKIO_MODE_DISA = 3U // IO 彻底关闭 } WKIO_PinMode_t;</pre>
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.1.2.23 HAL_WKIO_SetPinPull

表 5- 23 HAL_WKIO_SetPinPull

接口	说明
函数	<code>HAL_Status_t HAL_WKIO_SetPinPull(HAL_Dev_t *wkio, WKIO_Pin_t pin, WKIO_PinPull_t pull);</code>
功能	获取 WKIO 对应 pin 的上下拉电阻。
参数	HAL_Dev_t *wkio WKIO 控制句柄，通过 HAL_WKIO_Init 或 HAL_DEV_Find 返回值获得。 WKIO_Pin_t pin WKIO 的 pin 值。 WKIO_PinPull_t pull 设置 pin 的上下拉： typedef enum { HAL_WKIO_PULL_FLOATING = 0U, // 上拉电阻和下拉电阻都关闭 /* 35Kohm ~ 75Kohm (3.3V) / 70K ohm ~150K ohm (1.8V) */ HAL_WKIO_PULL_UP = 1U, // 开启上拉电阻，关闭下拉电阻 /* 30Kohm ~ 75Kohm (3.3V) / 60K ohm ~150K ohm (1.8V) */ HAL_WKIO_PULL_DOWN = 2U // 关闭下拉电阻，开启上拉电阻 } WKIO_PinPull_t;
注意事项	上下拉选择需结合功能配置，如推挽输出时，配置关闭上下拉。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功；

接口	说明
	返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.1.2.24 HAL_WKIO_GetPinPull

表 5- 24 HAL_WKIO_GetPinPull

接口	说明
函数	HAL_Status_t HAL_WKIO_GetPinPull(HAL_Dev_t *wkio, WKIO_Pin_t pin, WKIO_PinPull_t *pull);
功能	获取 WKIO 对应 pin 的上下拉电阻配置。
参数	HAL_Dev_t *wkio WKIO 控制句柄，通过 HAL_WKIO_Init 或 HAL_DEV_Find 返回值获得。 WKIO_Pin_t pin WKIO 的 pin 值。 WKIO_PinPull_t pull 获取 pin 的上下拉电阻配置： typedef enum { HAL_WKIO_PULL_FLOATING = 0U, // 上拉电阻和下拉电阻都关闭 /* 35Kohm ~ 75Kohm (3.3V) / 70K ohm ~150K ohm (1.8V) */ HAL_WKIO_PULL_UP = 1U, // 开启上拉电阻，关闭下拉电阻 /* 30Kohm ~ 75Kohm (3.3V) / 60K ohm ~150K ohm (1.8V) */ HAL_WKIO_PULL_DOWN = 2U // 关闭下拉电阻，开启上拉电阻 } WKIO_PinPull_t;
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.1.2.25 HAL_WKIO_WritePin

表 5- 25 HAL_WKIO_WritePin

接口	说明
函数	HAL_Status_t HAL_WKIO_WritePin(HAL_Dev_t *wkio, WKIO_Pin_t pin, WKIO_PinState_t PinState);
功能	写 WKIO 对应 pin 的输出电平。

接口	说明
参数	HAL_Dev_t *wkio WKIO 控制句柄，通过 HAL_WKIO_Init 或 HAL_DEV_Find 返回值获得。 WKIO_Pin_t pin WKIO 的 pin 值。 WKIO_PinState_t PinState 设置电平值： typedef enum { WKIO_PIN_RESET = 0U, // 输出低电平 WKIO_PIN_SET // 输出高电平 } WKIO_PinState_t;
注意事项	此接口可能存在延迟，最大延迟一个功能时钟周期。例如 WKIO 的功能时钟为 1MHz，那么将延迟 1us。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.1.2.26 HAL_WKIO_WritePort

表 5- 26 HAL_WKIO_WritePort

接口	说明
函数	HAL_Status_t HAL_WKIO_WritePort(HAL_Dev_t *wkio, uint32_t pin_mask);
功能	写 WKIO 所有 pin 输出对应电平。
参数	HAL_Dev_t *wkio WKIO 控制句柄，通过 HAL_WKIO_Init 或 HAL_DEV_Find 返回值获得。 uint32_t pin_mask WKIO 的输出电平值，对应 bit 为 1 则输出高电平，为 0 则输出低电平。
注意事项	此接口可能存在延迟，最大延迟一个功能时钟周期。例如 WKIO 的功能时钟为 1MHz，那么将延迟 1us。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.1.2.27 HAL_WKIO_SetWakeParam

表 5- 27 HAL_WKIO_SetWakeParam

接口	说明
函数	HAL_Status_t HAL_WKIO_SetWakeParam(HAL_Dev_t *wkio, WKIO_Pin_t pin, WKIO_WakeParam_t *param);
功能	设置 WKIO 对应 pin 的唤醒参数。
参数	HAL_Dev_t *wkio WKIO 控制句柄，通过 HAL_WKIO_Init 或 HAL_DEV_Find 返回值获得。 WKIO_Pin_t pin WKIO 的 pin 值。 WKIO_PinState_t PinState 设置电平值： typedef struct { WKIO_WakeEvent_t event; // 唤醒事件类型 WKIO_PinPull_t pull; // 上下拉电阻配置 WKIO_WakeCallback_t callback; // 唤醒回调函数 void *arg; // 唤醒回调函数出入参数 } WKIO_WakeParam_t;
注意事项	event 参数可参考 HAL_WKIO_SetWakeEvent; pull 参数可参考 HAL_WKIO_SetPinPull
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.1.2.28 HAL_WKIO_EnableWake

表 5- 28 HAL_WKIO_EnableWake

接口	说明
函数	HAL_Status_t HAL_WKIO_EnableWake(HAL_Dev_t *wkio);
功能	使能 WKIO 唤醒模式。
参数	HAL_Dev_t *wkio 唤醒 IO 控制句柄，通过 HAL_WKIO_Init 或 HAL_DEV_Find 返回值获得。
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功；

接口	说明
	返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.1.2.29 HAL_WKIO_DisableWake

表 5- 29 HAL_WKIO_DisableWake

接口	说明
函数	HAL_Status_t HAL_WKIO_DisableWake(HAL_Dev_t *wkio);
功能	关闭 WKIO 唤醒模式。
参数	HAL_Dev_t *wkio 唤醒 IO 控制句柄，通过 HAL_WKIO_Init 或 HAL_DEV_Find 返回值获得。
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.1.2.30 HAL_WKIO_SetWakeEvent

表 5- 30 HAL_WKIO_SetWakeEvent

接口	说明
函数	HAL_Status_t HAL_WKIO_SetWakeEvent(HAL_Dev_t *wkio, WKIO_Pin_t pin, WKIO_WakeEvent_t waketype);
功能	设置 WKIO 对应 Pin 的唤醒事件类型。
参数	HAL_Dev_t *wkio 唤醒 IO 控制句柄，通过 HAL_WKIO_Init 或 HAL_DEV_Find 返回值获得。 WKIO_Pin_t pin 唤醒 IO 的 pin 值。 WKIO_WakeEvent_t waketype 唤醒事件类型，如下： typedef enum { WKIO_WAKE_EVT_RISING_EDGE = 0U, //上升沿触发 WKIO_WAKE_EVT_FALLING_EDGE = 1U, //下降沿触发 WKIO_WAKE_EVT_BOTH_EDGE = 2U, //双边沿触发 WKIO_WAKE_EVT_HIGH_LEVEL = 3U, //高电平触发 WKIO_WAKE_EVT_LOW_LEVEL = 4U, //低电平触发

接口	说明
	} WKIO_WakeEvent_t;
注意事项	注意与上下拉电阻接口（HAL_WKIO_SetPinPull）配合使用。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.1.2.31 HAL_WKIO_GetWakeEvent

表 5- 31 HAL_WKIO_GetWakeEvent

接口	说明
函数	HAL_Status_t HAL_WKIO_GetWakeEvent(HAL_Dev_t *wkio, WKIO_Pin_t pin, WKIO_WakeEvent_t *waketype);
功能	获取 WKIO 对应 pin 的唤醒事件类型。
参数	HAL_Dev_t *wkio 唤醒 IO 控制句柄，通过 HAL_WKIO_Init 或 HAL_DEV_Find 返回值获得。 WKIO_Pin_t pin 唤醒 IO 的 pin 值。 WKIO_WakeEvent_t waketype 获取唤醒事件类型，如下： typedef enum { WKIO_WAKE_EVT_RISING_EDGE = 0U, //上升沿触发 WKIO_WAKE_EVT_FALLING_EDGE = 1U, //下降沿触发 WKIO_WAKE_EVT_BOTH_EDGE = 2U, //双边沿触发 WKIO_WAKE_EVT_HIGH_LEVEL = 3U, //高电平触发 WKIO_WAKE_EVT_LOW_LEVEL = 4U, //低电平触发 } WKIO_WakeEvent_t;
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.1.2.32 HAL_WKIO_ReadWakePin

表 5- 32 HAL_WKIO_ReadWakePin

接口	说明
----	----

接口	说明
函数	int32_t HAL_WKIO_ReadWakePin(HAL_Dev_t *wkio, WKIO_Pin_t pin);
功能	获取 WKIO 唤醒模式下对应 pin 的电平值。
参数	HAL_Dev_t *wkio 唤醒 IO 控制句柄，通过 HAL_WKIO_Init 或 HAL_DEV_Find 返回值获得。 WKIO_Pin_t pin 唤醒 IO 的 pin 值。
注意事项	无
返回值	唤醒 IO 的电平或状态。 返回值为 0 表示对应 IO 为低电平； 返回值为 1 表示对应 IO 为高电平； 返回值为负表示获取失败，具体参考 HAL_STATUS_*；

5.1.2.33 HAL_WKIO_SetWakeDebounce

表 5- 33 HAL_WKIO_SetWakeDebounce

接口	说明
函数	HAL_Status_t HAL_WKIO_SetWakeDebounce(HAL_Dev_t *wkio, uint32_t debounce);
功能	设置对应 Port 所有 pin 的唤醒模式消抖值。
参数	HAL_Dev_t *wkio 唤醒 IO 控制句柄，通过 HAL_WKIO_Init 或 HAL_DEV_Find 返回值获得。 uint32_t debounce 消抖计数值，采样时钟为 WKIO 功能时钟。 WKIO 默认功能时钟为 LPCLK_FRAC，在 HAL_WKIO_Init 中配置，对应配置宏为 CONFIG_WKIO_SRC。
注意事项	消抖时间需要功能时钟频率与 Debounce 配合使用，例如：功能时钟频率为 1MHz，Debounce 值为 1000000 则消抖时间为 1 秒。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.2 UART

5.2.1 功能简介

5.2.1.1 特性

1. 支持 5/6/7/8 位数据宽度
2. 支持 1/2 位停止位
3. 支持无/奇/偶校验
4. 支持最高 6MHz 波特率
5. 部分支持硬流控
6. 支持特殊序列唤醒
7. TX/RX FIFO 最大 32bit x 64
8. 支持轮询、中断、DMA 方式传输

5.2.1.2 使用

UART 的 HAL API 提供了两种方式实现上述特性：

1. 集成 API，通过 Init，Open，Receive，Transmit 等 API 即可实现收发不需要关注细节。
2. 分散 API，通过单一功能 API 实现更复杂和个性化化的功能，需要更多关注细节。

建议使用集成 API。

Sample：

[project/driver/uart/uart_poll](#)

[project/driver/uart/uart_dma](#)

[project/driver/uart/uart_interrupt](#)

[project/driver/uart/callback](#)

集成 API 方式使用流程如下：

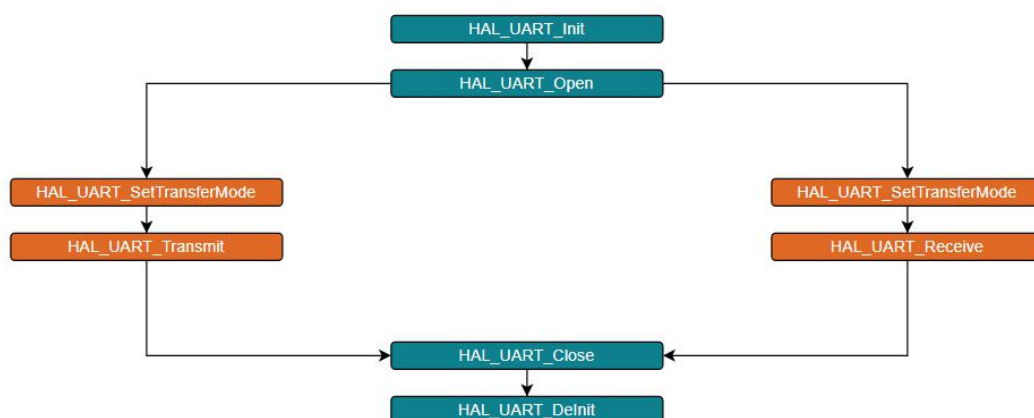


图 5-2 UART 集成 API 使用流程

HAL_UART_Init 配置了 UART 基本特性，如下

```
typedef struct {
    uint32_t      baudRate;      /* 波特率 */
    UART_DataWidth_t dataBits;    /* 数据位宽 */
    UART_StopBit_t stopBits;     /* 停止位个数 */
    UART_Parity_t  parity;       /* 校验位类型 */
    UART_FlowCtrl_t autoFlowCtrl; /* 硬件流控 */
} UART_InitParam_t;
```

HAL_UART_SetTransferMode 设置 Tx 或 Rx 是使用轮询、中断、DMA 传输方式。

5.2.2 接口

5.2.2.1 HAL_UART_Init

表 5-34 HAL_UART_Init

接口	说明
函数	HAL_Dev_t * HAL_UART_Init(UART_ID_t id, const UART_InitParam_t *param)
功能	初始化 UART 设备，申请系统资源，获取控制设备句柄。
参数	UART_ID_t id UART 外设 ID; typedef enum { UART0_ID = 0U, UART1_ID, UART2_ID, UART_NUM, UART_INVALID_ID = 0xFFU } UART_ID_t; const UART_InitParam_t * param UART 初始化参数的结构体地址; 波特率; 数据位宽; 停止位宽; 校验位; 流控 typedef struct { uint32_t baudRate; UART_DataWidth_t dataBits; UART_StopBit_t stopBits; UART_Parity_t parity; UART_FlowCtrl_t autoFlowCtrl; } UART_InitParam_t;

接口	说明
注意事项	无
返回值	UART 设备实例句柄(HAL_Dev_t *类型)

5.2.2.2 HAL_UART_DeInit

表 5- 35 HAL_UART_DeInit

接口	说明
函数	HAL_Status_t HAL_UART_DeInit(HAL_Dev_t *uart);
功能	释放指定 UART 驱动资源。
参数	HAL_Dev_t *uart Uart 设备控制句柄。
注意事项	释放前需先 Close
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.2.2.3 HAL_UART_ExtControl

表 5- 36 HAL_UART_ExtControl

接口	说明
函数	HAL_Status_t HAL_UART_ExtControl(const HAL_Dev_t *uart, UART_ParamType_t ptype, void *param);
功能	扩展控制，获取或设置不常用参数。
参数	HAL_Dev_t *uart Uart 设备控制句柄。 UART_ParamType_t ptype 参数类似 typedef enum { UART_PARAM_WAKEUP_THRESHOLD = 0, // 唤醒阈值 UART_PARAM_WAKEUP_CTRL = 1, // 唤醒使能状态 UART_PARAM_WAKEUP_CALLBACK = 2, // 唤醒回调 UART_PARAM_WAKEUP_MODE = 3, // 唤醒模式，0 自动唤醒，1 应用主动调用唤醒 };

接口	说明
	} UART_ParamType_t; void *param 读取或设置的参数 buff
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.2.2.4 HAL_UART_Open

表 5- 37 HAL_UART_Open

接口	说明
函数	HAL_Status_t HAL_UART_Open(HAL_Dev_t *uart);
功能	开启 UART 硬件。
参数	HAL_Dev_t *uart Uart 设备控制句柄。
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.2.2.5 HAL_UART_Close

表 5- 38 HAL_UART_Close

接口	说明
函数	HAL_Status_t HAL_UART_Close(HAL_Dev_t *uart);
功能	关闭 UART 硬件。
参数	HAL_Dev_t *uart Uart 设备控制句柄。
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.2.2.6 HAL_UART_SetTransferMode

表 5- 39 HAL_UART_SetTransferMode

接口	说明
函数	HAL_Status_t HAL_UART_SetTransferMode(const HAL_Dev_t *uart, UART_TransDir_t transDir, UART_TransMode_t Mode);
功能	设置传输模式
参数	HAL_Dev_t *uart Uart 设备控制句柄。 UART_TransDir_t transDir 传输方向 typedef enum { UART_DIR_TX = 0, UART_DIR_RX = 1, } UART_TransDir_t; UART_TransMode_t Mode 传输模式 typedef enum { UART_TRANS_MODE_NOMA = 0, // 轮询方式传输 UART_TRANS_MODE_INTERRUPT = 1, // 中断方式传输 UART_TRANS_MODE_DMA = 2, // DMA 方式传输 } UART_TransMode_t;
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.2.2.7 HAL_UART_Receive

表 5- 40 HAL_UART_Receive

接口	说明
函数	int32_t HAL_UART_Receive(const HAL_Dev_t *uart, uint8_t *buff, int32_t size, uint32_t msec);
功能	接收数据
参数	HAL_Dev_t *uart

接口	说明
	Uart 设备控制句柄。 uint8_t *buff 接收数据 buffer int32_t size 期望接收数据长度 uint32_t msec 数据接收超时
注意事项	无
返回值	返回值负数表示执行失败，为错误码 返回值为非负数表示接收数据长度。

5.2.2.8 HAL_UART_Transmit

表 5- 41 HAL_UART_Transmit

接口	说明
函数	int32_t HAL_UART_Transmit(const HAL_Dev_t *uart, const uint8_t *buff, int32_t size, uint32_t msec);
功能	
参数	HAL_Dev_t *uart Uart 设备控制句柄。 uint8_t *buff 发送数据 buffer int32_t size 期望发送数据长度 uint32_t msec 数据发送超时
注意事项	无
返回值	返回值负数表示执行失败，为错误码

接口	说明
	返回值为非负数表示发送数据长度。

5.2.2.9 HAL_UART_GetRxData

表 5-42 HAL_UART_GetRxData

接口	说明
函数	uint8_t HAL_UART_GetRxData(const HAL_Dev_t *uart);
功能	从 UART Rx FIFO 中读取数据。
参数	HAL_Dev_t *uart Uart 设备控制句柄。
注意事项	无
返回值	读取的数据。

5.2.2.10 HAL_UART_PutTxData

表 5-43 HAL_UART_PutTxData

接口	说明
函数	void HAL_UART_PutTxData(const HAL_Dev_t *uart, uint8_t data);
功能	写入数据至 UART tx FIFO。
参数	HAL_Dev_t *uart Uart 设备控制句柄。
注意事项	无
返回值	空

5.2.2.11 HAL_UART_IsTxReady

表 5-44 HAL_UART_IsTxReady

接口	说明
函数	bool HAL_UART_IsTxReady(const HAL_Dev_t *uart);
功能	检查 UART Tx FIFO 没满，可以继续写入数据。
参数	HAL_Dev_t *uart Uart 设备控制句柄。

接口	说明
注意事项	无
返回值	True TX FIFO 没有满可以继续写入数据； False Tx FIFO 已经满了不能继续写入数据。

5.2.2.12 HAL_UART_IsTxEmpty

表 5- 45 HAL_UART_IsTxEmpty

接口	说明
函数	bool HAL_UART_IsTxEmpty(const HAL_Dev_t *uart);
功能	检查 UART TX FIFO 是否空。
参数	HAL_Dev_t *uart Uart 设备控制句柄。
注意事项	无
返回值	True TX FIFO 空可以结束发送了； False Tx FIFO 没空，还有数据需要继续发送。

5.2.2.13 HAL_UART_IsRxReady

表 5- 46 HAL_UART_IsRxReady

接口	说明
函数	bool HAL_UART_IsRxReady(const HAL_Dev_t *uart);
功能	检查 UART Rx FIFO 有数据。
参数	HAL_Dev_t *uart Uart 设备控制句柄。
注意事项	无
返回值	True RX FIFO 已接收到数据； False Rx FIFO 没有数据了。

5.2.2.14 HAL_UART_RegisterIRQ

表 5- 47 HAL_UART_RegisterIRQ

接口	说明
----	----

接口	说明
函数	HAL_Status_t HAL_UART_RegisterIRQ(HAL_Dev_t *uart, UART_TransDir_t transDir, const HAL_Callback_t *irqCb);
功能	注册传输中断回调函数。
参数	HAL_Dev_t *uart Uart 设备控制句柄。 UART_TransDir_t transDir 传输方向 typedef enum { UART_DIR_TX = 0, UART_DIR_RX = 1, } UART_TransDir_t; HAL_Callback_t *irqCb 回调函数 typedef struct { HAL_CbFunc_t cb; // 回调函数指针 void * arg; // 回调函数输入参数 } HAL_Callback_t;
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.2.2.15 HAL_UART_UnregisterIRQ

表 5- 48 HAL_UART_UnregisterIRQ

接口	说明
函数	HAL_Status_t HAL_UART_UnregisterIRQ(const HAL_Dev_t *uart, UART_TransDir_t transDir);
功能	反注册指定方向的中断回调函数。
参数	HAL_Dev_t *uart Uart 设备控制句柄。 UART_TransDir_t transDir 传输方向 typedef enum { UART_DIR_TX = 0,

接口	说明
	UART_DIR_RX = 1, } UART_TransDir_t;
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.2.2.16 HAL_UART_EnableTxReadyIRQ

表 5- 49 HAL_UART_EnableTxReadyIRQ

接口	说明
函数	HAL_Status_t HAL_UART_EnableTxReadyIRQ(const HAL_Dev_t *uart);
功能	使能 UART TX FIFO 少于阈值中断
参数	HAL_Dev_t *uart Uart 设备控制句柄。
注意事项	TX FIFO 数据低于设定阈值触发中断。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.2.2.17 HAL_UART_EnableRxReadyIRQ

表 5- 50 HAL_UART_EnableRxReadyIRQ

接口	说明
函数	HAL_Status_t HAL_UART_EnableRxReadyIRQ(const HAL_Dev_t *uart);
功能	使能 UART RX FIFO 大于阈值中断
参数	HAL_Dev_t *uart Uart 设备控制句柄。
注意事项	RX FIFO 数据高于设定阈值触发中断。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.2.2.18 HAL_UART_DisableTxReadyIRQ

表 5- 51 HAL_UART_DisableTxReadyIRQ

接口	说明
函数	HAL_Status_t HAL_UART_DisableTxReadyIRQ(const HAL_Dev_t *uart);
功能	关闭 UART TX FIFO 少于阈值中断
参数	HAL_Dev_t *uart Uart 设备控制句柄。
注意事项	TX FIFO 数据低于设定阈值触发中断。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.2.2.19 HAL_UART_DisableRxReadyIRQ

表 5- 52 HAL_UART_DisableRxReadyIRQ

接口	说明
函数	HAL_Status_t HAL_UART_DisableRxReadyIRQ(const HAL_Dev_t *uart);
功能	关闭 UART RX FIFO 大于阈值中断
参数	HAL_Dev_t *uart Uart 设备控制句柄。
注意事项	RX FIFO 数据高于设定阈值触发中断。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.2.2.20 HAL_UART_RegisterDMA

表 5- 53 HAL_UART_RegisterDMA

接口	说明
函数	HAL_Status_t HAL_UART_RegisterDMA(HAL_Dev_t *uart, UART_TransDir_t transDir, HAL_Callback_t *dmaCb);
功能	注册 DMA 传输完成回调函数。
参数	HAL_Dev_t *uart Uart 设备控制句柄。 UART_TransDir_t transDir

接口	说明
	传输方向 <pre>typedef enum { UART_DIR_TX = 0, UART_DIR_RX = 1, } UART_TransDir_t;</pre> HAL_Callback_t *irqCb 回调函数 <pre>typedef struct { HAL_CbFunc_t cb; // 回调函数指针 void * arg; // 回调函数输入参数 } HAL_Callback_t;</pre>
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.2.2.21 HAL_UART_UnregisterDMA

表 5- 54 HAL_UART_UnregisterDMA

接口	说明
函数	HAL_Status_t HAL_UART_UnregisterDMA(const HAL_Dev_t *uart, UART_TransDir_t transDir);
功能	反初注册 DMA 传输完成中断回调。
参数	HAL_Dev_t *uart Uart 设备控制句柄。 UART_TransDir_t transDir 传输方向 <pre>typedef enum { UART_DIR_TX = 0, UART_DIR_RX = 1, } UART_TransDir_t;</pre>
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.2.2.22 HAL_UART_StartTransmitDMA

表 5- 55 HAL_UART_StartTransmitDMA

接口	说明
函数	HAL_Status_t HAL_UART_StartTransmitDMA(const HAL_Dev_t *uart, const uint8_t *buf, int32_t size);
功能	开始 DMA 传输
参数	HAL_Dev_t *uart Uart 设备控制句柄。 const uint8_t *buf 发送数据 buffer int32_t size 发送数据长度
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.2.2.23 HAL_UART_StopTransmitDMA

表 5- 56 HAL_UART_StopTransmitDMA

接口	说明
函数	int32_t HAL_UART_StopTransmitDMA(const HAL_Dev_t *uart);
功能	停止 DMA 传输
参数	HAL_Dev_t *uart Uart 设备控制句柄。
注意事项	无
返回值	若为负数则表示错误码，若为正数表示已发送长度。

5.2.2.24 HAL_UART_StartReceiveDMA

表 5- 57 HAL_UART_StartReceiveDMA

接口	说明
函数	HAL_Status_t HAL_UART_StartReceiveDMA(const HAL_Dev_t *uart, uint8_t *buf, int32_t size);
功能	开始 DMA 接收数据
参数	HAL_Dev_t *uart

接口	说明
	Uart 设备控制句柄。 const uint8_t *buf 接收数据 buffer int32_t size 接收数据长度
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.2.2.25 HAL_UART_StopReceiveDMA

表 5- 58 HAL_UART_StopReceiveDMA

接口	说明
函数	int32_t HAL_UART_StopReceiveDMA(const HAL_Dev_t *uart);
功能	停止 DMA 数据接收
参数	HAL_Dev_t *uart Uart 设备控制句柄。
注意事项	无
返回值	若为负数则表示错误码，若为正数表示已经接收长度。

5.3 SPI

5.3.1 功能简介

5.3.1.1 特性

- 1. 支持 Standard, Dual, Quad 传输
- 2. 支持半双工和全双工传输
- 3. 支持 Master 和 Slave
- 4. 支持多 Master 和多 Slave
- 5. 支持 8 位、16 位、32 位数据位传输
- 6. 通信频率最高可支持 100MHz
- 7. 支持高位数据先收发
- 8. 支持软件或硬件自动 CS
- 9. TX/RX FIFO 最大 32bit x 32
- 10. 支持轮询、中断、DMA 方式传输

SPI 的通信由以下 Pin 构成，各 Pin 对应的 PinNumber 请参考对应的 soc 的 datasheet。

表 5- 59 SPI pin 功能

Pin Name	Description
CLK	时钟，即 SPI 工作频率。 1. Master 输出，Slave 输入 2. 由 Master 提供，Slave 根据此时钟收发数据。 3. 可配置在上升沿或下降沿采集数据。
CS(NSS)	从机选择信号，低电平有效。 1. Master 输出，Slave 输入 2. 可配置为硬件自动控制或软件控制，若是软件控制 Master 端需使用 API HAL_SPI_EnableCS/HAL_SPI_DisableCS，Slave 端需使用 API HAL_SPI_IsCsInterrupt 3. Master 可以输出多个 CS 用于控制多个 Slave。
MOSI (Q0)	Master Output Slave Input，主机数据输出，从机数据输入端口。 1. 可以与 MISO 同时传输实现全双工 2. 若是 Standard SPI，则 Master 端为输出，Slave 端为输入端口，若是 Dual SPI 和 Quad SPI 则为输入和输出。 3. 若是 Dual SPI 则与 MISO 组合成 2 位数据线，此为低位。

Pin Name	Description
	4. 若为 Quad SPI 则与 MISO,WP,HOLD 组成 4 位数据线，此为最低位。
MISO (Q1)	同 MOSI，若为 Standard SPI 则是从机数据输出，若为 Dual SPI 或 Quad SPI 则是第 2 位数据。
WP (Q2)	同 MOSI, QUAD SPI 时有效。
HOLD (Q3)	同 MOSI, QUAD SPI 时有效。

SPI 的通信需要双方选择一样的时钟相位和极性，即对应 Mode 0~Mode3。其定义如下：

- 1. Mode 0：CPOL=0，CPHA=0，即接收者须在 CS 拉低后 CLK 的第一个上升沿采样数据。
- 2. Mode 1：CPOL=0，CPHA=1，即接收者须在 CS 拉低后 CLK 的第二个下降沿采样数据。
- 3. Mode 2：CPOL=1，CPHA=0，即接收者须在 CS 拉低后 CLK 的第一个下降沿采样数据。
- 4. Mode 3：CPOL=1，CPHA=1，即接收者须在 CS 拉低后 CLK 的第二个上升沿采样数据。

要保持接收者采集数据时数据信号的稳定，需发送者提前半个周期改变数据。

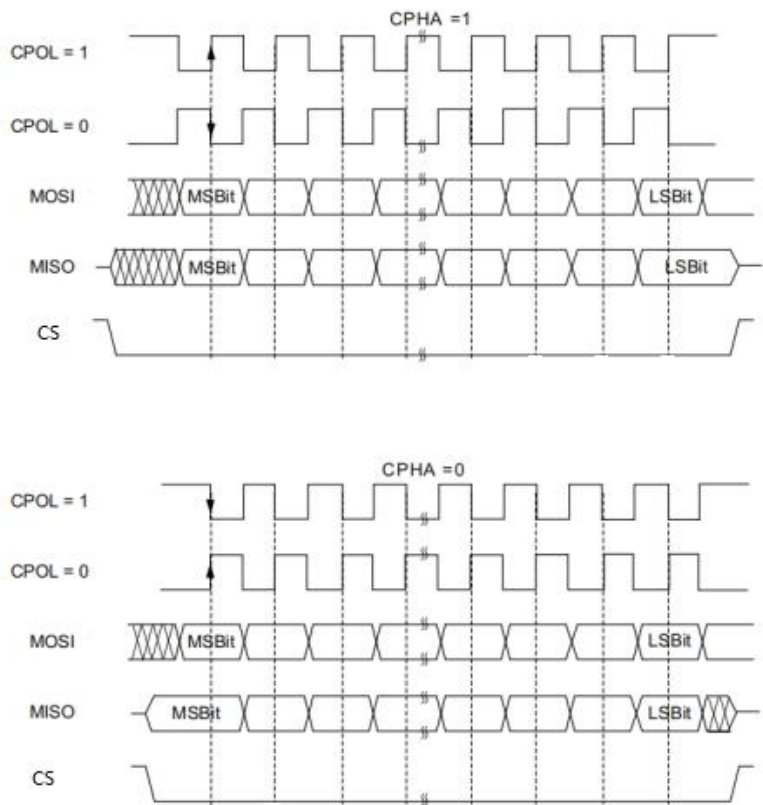


图 5-3 SPI Mode 示意图

5.3.1.2 使用

SPI 的 HAL API 提供了两种方式实现上述特性：

- 1. 集成 API，通过 Init，Open，Receive，Transmit 等 API 即可实现 SP 的收发不需要关注细节。
- 2. 分散 API，通过单一功能 API 实现更复杂和个性化化的功能，需要更多关注细节。

建议使用继承 API。

Sample：

project/driver/spi/spi_poll_master
project/driver/spi/spi_poll_slave
project/driver/spi/spi_interrupt_master
project/driver/spi/spi_interrupt_slave
project/driver/spi/spi_dma_master
project/driver/spi/spi_dma_slave
project/driver/spi/spi_dual_master
project/driver/spi/spi_dual_slave
project/driver/spi/spi_quad_master
project/driver/spi/spi_quad_slave

集成 API 方式使用流程如下：

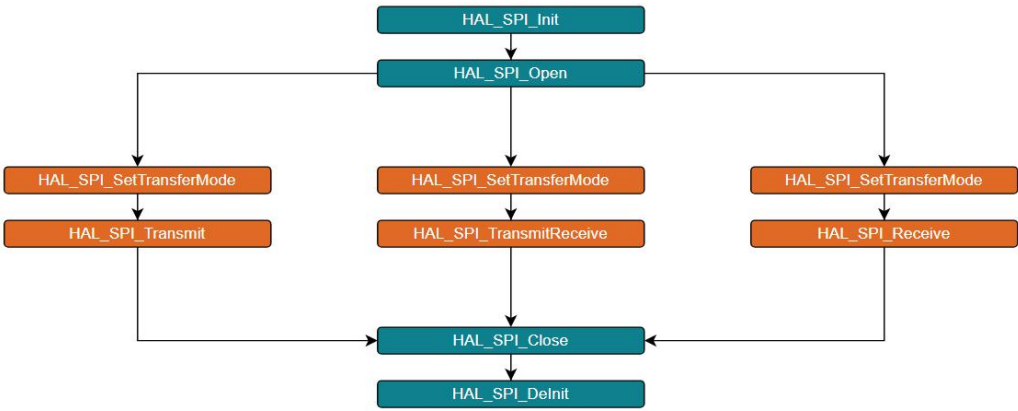


图 5-4 SPI 集成 API 使用流程

HAL_SPI_Init 用于配置 SPI 特性，配置参数如下。

```
typedef struct {
    SPI_Mode_t      mode;      /* Master / Slave */
    SPI_SclkMode_t  sclkMode;   /* CPOL 和 CPHA */
    SPI_IOMode_t    ioMode;     /* Standard, dual, quad */
    SPI_Cs_t        csMode;     /* 软件 CS 还是硬件 CS */
    SPI_DataWidth_t dataWidth;  /* 数据位宽，可选择 8/16/32 */
    uint32_t        sclk;       /* 运行频率 */
} SPI_InitParam_t;
```

HAL_SPI_Init 还隐藏配置了 Pinmux。Pinmux 默认在板级配置文件中集中定义（如 mrs6240_p2512.c 中弱定义）如下，但也可在工程文件中再定义覆盖板级配置文件中的定义。

```
/* 弱定义 SPI 正常功能 pin 配置， IO 的驱动能力根据 SPI 速度配置，参考 GPIO 章节*/
DEFINE_PINMUX_WEAK(spi0, 4,
    { GPIO_PORT_A, GPIO_PIN_0, {GPIOA_P0_F2_SPI_CS,    GPIO_DRIVING_LEVEL_0,
    GPIO_PUSH_PULL, GPIO_PULL_UP }}, /* CS */
    { GPIO_PORT_A, GPIO_PIN_1, {GPIOA_P1_F2_SPI_CLK,    GPIO_DRIVING_LEVEL_0,
    GPIO_PUSH_PULL, GPIO_PULL_UP }}, /* CLK */
    { GPIO_PORT_A, GPIO_PIN_2, {GPIOA_P2_F2_SPI_MOSI,    GPIO_DRIVING_LEVEL_0,
    GPIO_PUSH_PULL, GPIO_PULL_UP }}, /* MOSI */
    { GPIO_PORT_A, GPIO_PIN_3, {GPIOA_P3_F2_SPI_MISO,    GPIO_DRIVING_LEVEL_0,
    GPIO_PUSH_PULL, GPIO_PULL_UP }}, /* MISO */
);

/* 弱定义 SPI 休眠时 pin 配置*/
DEFINE_PINMUX_WEAK(spi0_deinit, 4,
    { GPIO_PORT_A, GPIO_PIN_0, {GPIOx_Pn_F15_DIS,    GPIO_DRIVING_LEVEL_0, GPIO_PUSH_PULL,
    GPIO_PULL_UP }}, /* CS */
    { GPIO_PORT_A, GPIO_PIN_1, {GPIOx_Pn_F15_DIS,    GPIO_DRIVING_LEVEL_0, GPIO_PUSH_PULL,
    GPIO_PULL_UP }}, /* CLK */
    { GPIO_PORT_A, GPIO_PIN_2, {GPIOx_Pn_F15_DIS,    GPIO_DRIVING_LEVEL_0, GPIO_PUSH_PULL,
    GPIO_PULL_UP }}, /* MOSI */
    { GPIO_PORT_A, GPIO_PIN_3, {GPIOx_Pn_F15_DIS,    GPIO_DRIVING_LEVEL_0, GPIO_PUSH_PULL,
    GPIO_PULL_UP }}, /* MISO */
);
```

```
/* 可以在工程文件定义同名变量覆盖配置 */
const HAL_PinmuxConfig_t pinmux_spi0[] = {
    { GPIO_PORT_A, GPIO_PIN_0, {GPIOA_P0_F2_SPI_CS,    GPIO_DRIVING_LEVEL_3,
    GPIO_PUSH_PULL, GPIO_PULL_UP }}, /* CS */
    { GPIO_PORT_A, GPIO_PIN_1, {GPIOA_P1_F2_SPI_CLK,    GPIO_DRIVING_LEVEL_2,
    GPIO_PUSH_PULL, GPIO_PULL_UP }}, /* CLK */
    { GPIO_PORT_A, GPIO_PIN_2, {GPIOA_P2_F2_SPI_MOSI,    GPIO_DRIVING_LEVEL_2,
    GPIO_PUSH_PULL, GPIO_PULL_UP }}, /* MOSI */
    { GPIO_PORT_A, GPIO_PIN_3, {GPIOA_P3_F2_SPI_MISO,    GPIO_DRIVING_LEVEL_2,
    GPIO_PUSH_PULL, GPIO_PULL_UP }}, /* MISO */
    { GPIO_PORT_A, GPIO_PIN_4, {GPIOA_P4_F2_SPI_HOLD,    GPIO_DRIVING_LEVEL_2,
    GPIO_PUSH_PULL, GPIO_PULL_UP }}, /* HOLD */
    { GPIO_PORT_A, GPIO_PIN_5, {GPIOA_P5_F2_SPI_WP,    GPIO_DRIVING_LEVEL_2,
    GPIO_PUSH_PULL, GPIO_PULL_UP }}, /* WP */
};
```

HAL_SPI_Open 开启 SPI 硬件。若 SPI 为 Master 且连接多 Slave 时，HAL_SPI_Init 的配置参数包括每个 Slave 的配置（param_num 指明配置 Slave 参数个数），HAL_SPI_Open 的 slave_addrIndex 参数指定与哪个 Slave 通信。

HAL_SPI_SetTransferMode 设置发送或接收使用轮询、中断、DMA 方式。

HAL_SPI_Receive 和 HAL_SPI_Transmit 是半双工收和发，HAL_SPI_TransmitReceive 是全双工收发。

分散 API 的使用重点是理解 FIFO 相关的配置。在 Tx 或 Rx FIFO 中有一个 Trigger level，它是触发 Flag、中断、DMA 传输的分界点。

中断模式接收，Rx FIFO 默认配置 1，即收到数据就产生中断确保一帧最后的数据能产生中断被读走。

中断模式发送，Tx FIFO 默认配置 1/2 总深度，确保数据不会 overflow。

DMA 模式，FIFO level 需与 DMA FIFO 关联，详情参考 DMA 章节。

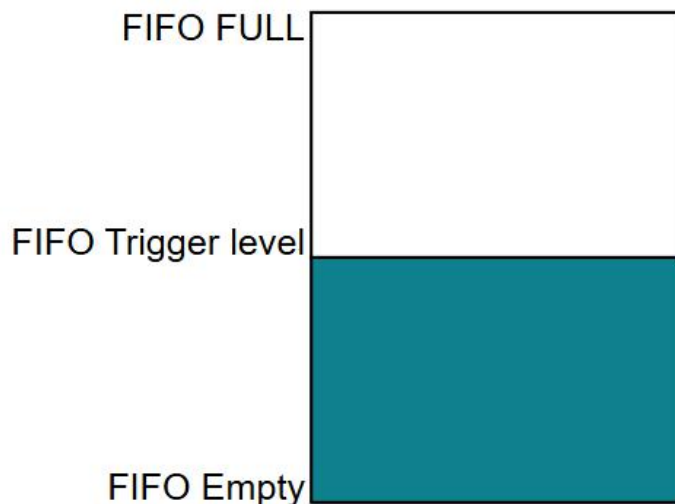


图 5-5 SPI FIFO 使用示意图

5.3.2 模式介绍

SPI 提供了两种不同的 MODE 去适应不同的传输频率。

SDK 中默认在传输频率小于等于 30MHz 时，使用 MODE1 进行 spi 配置；传输频率大于 30MHz 时，若使用 MODE1 则需要调节 master delay;使用 MODE2 进行配置则需要打 DUMMY(发送无效数据)。（不同 MODE 的具体使用方式见 5.3.2.1 和 5.3.2.2）。

模式的选择通过用户配置的频率自动选择，也可通过 LL_SPI_SlvClkSelect 函数进行配置，参数选择 LL_QSPI_SLVCLK_SSICLK 配置为 MODE1，参数选择 LL_QSPI_SLVCLK_FUNCLK 配置为 MODE2。

具体的 MODE 选择方式见下图。

```
../*mode1.or.mode2.set*/
..tmp_div=.LL_SPI_GetPrescaler(spi_instance[temp_handle->spi_id]);
..if.(tmp_param->sclk.<=.30000000)&&(tmp_div->SPI_CLOCK_DEV_4)){
...../*mode1*/
.....LL_SPI_SlvClkSelect(spi_instance[temp_handle->spi_id],.LL_QSPI_SLVCLK_SSICLK);
..}else{
...../*mode2*/
.....LL_SPI_SetMode(spi_instance[temp_handle->spi_id],.LL_QSPI_MST_MODE);
.....LL_SPI_EnableSlvClkIn(spi_instance[temp_handle->spi_id]);
.....LL_SPI_SetMode(spi_instance[temp_handle->spi_id],.LL_QSPI_SLV_MODE);
.....LL_SPI_SlvClkSelect(spi_instance[temp_handle->spi_id],.LL_QSPI_SLVCLK_FUNCCLK);
.....LL_SPI_DisableSlvClkIn(spi_instance[temp_handle->spi_id]);
..}
```

图 5- 6 SPI 模式选择

5.3.2.1 SPI_MODE1

SPI_MODE1 需要工作在 30MHz 以下，并且分频系数 PSC 大于 4(默认选择系统时钟作为 SPI 外设时钟源时，PSC = 系统时钟 / SPI 传输速率)。如果要在高速下使用 SPI_MODE1,则分频系数至少大于 5，越大越好。

SPI_MODE1 大于 30MHz 以上需要调节 master delay 的原因说明：

当 CPOL=0,CPHA=0 时，SPI 硬件设计上采取 CLK 下降沿变化数据信号，在上升沿进行数据采样。因此数据的变化必须在下一个 CLK 上升沿到来前完成。实际提供给数据变化的时间裕度只有 CLK 的半周期长度。在 60MHz 情况下，时间裕度约为 8.3ns。而芯片仅从机单端测得的端口延时约 6 到 10ns。因此如果主机仍然使用自身发出的 CLK 进行数据采样，几乎是不可能的。

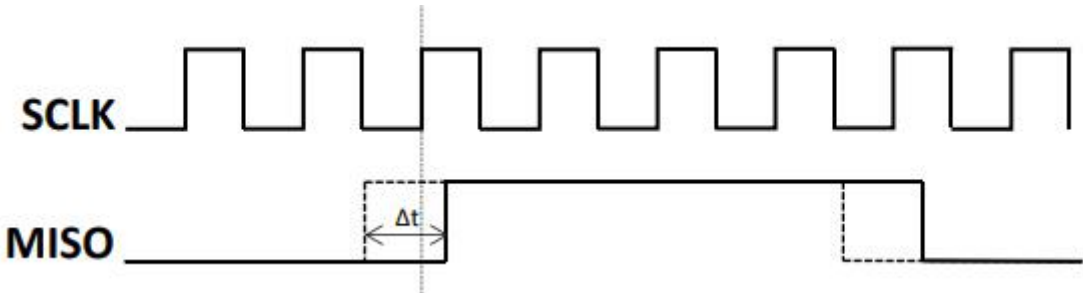


图 5- 7 SPI 由于延时导致上升沿采样错误

SPI 主机通常会采用延迟采样的方案来解决这一问题，不使用原始的 CLK,而是使用另一比 CLK 延时的 CLK_0 进行采样，即调节 master delay。此时由于数据与时钟均具有延时，互相抵消，SPI 正确采数。

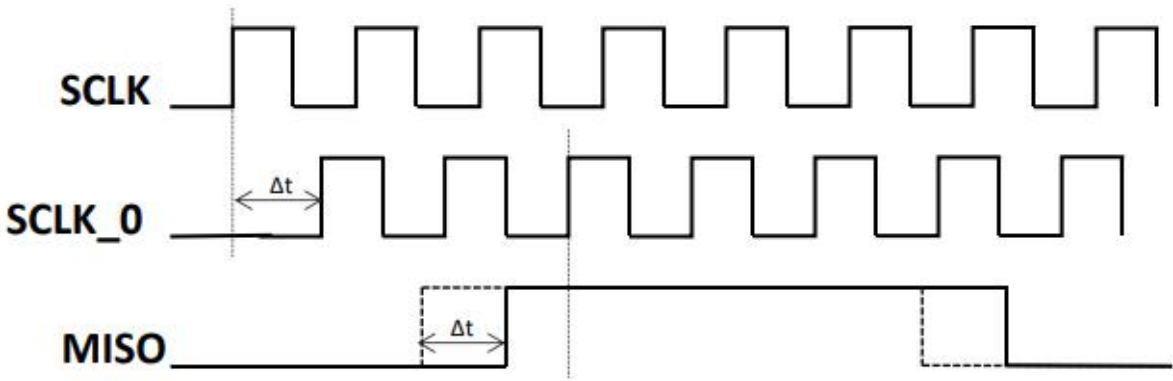


图 5- 8 SPI 采用 CLK_0 采数效果图

5.3.2.2 SPI_MODE2

SPI_MODE2 没有分频系数要求，但因为硬件设计问题需要打 DUMMY。

SPI_MODE2 需要打 DUMMY(发送无效数据)的原因说明：

在 MODE2 的硬件设计中，有两种 pop(数据读取) FIFO 中的数据的情况，第一种情况是在 CS 的下降沿，目的是为了在第一个上升沿读取数据前准备好数据。第二种情况是在每一笔数据传输结束时，目的是为了自动完成 FIFO 的数据更新，确保下一笔数据能无缝传输。

因此在将一组数据拆分成多个 CS 时，会出现重复的 pop（每一笔数据完成的 pop,以及 CS 再次被拉低的 pop）。导致后续的数据被提前 pop，却没有被传输，最终表现为丢数。因此需要通过发送 DUMMY(发送无效数据)的方式去完成 SPI 传输。

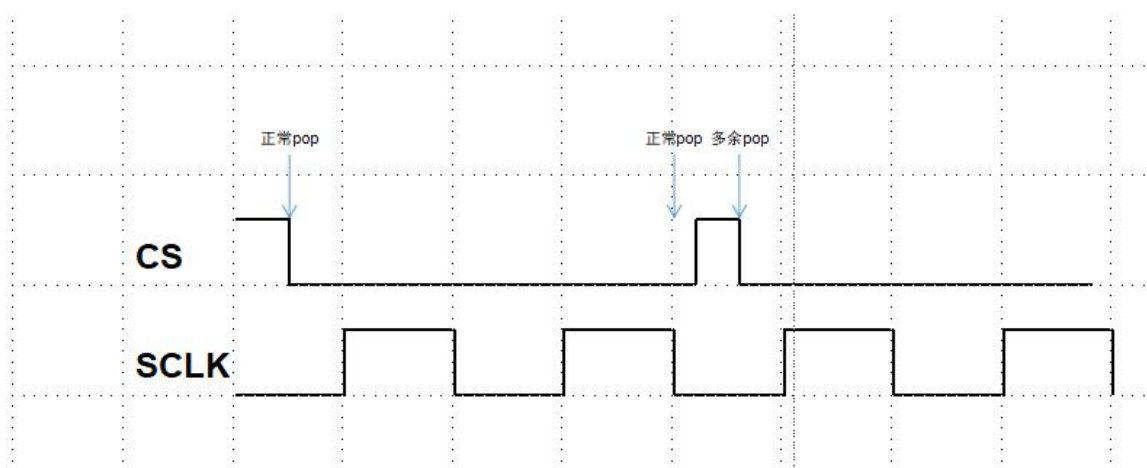


图 5-9 SPI 传输中 POP 示意图

5.3.3 接口

5.3.3.1 HAL_SPI_Init

表 5- 60 HAL_SPI_Init

接口	说明
函数	HAL_Dev_t * HAL_SPI_Init(SPI_ID_t id, const SPI_InitParam_t *param, const uint8_t param_num);
功能	初始化 SPI 设备，申请系统资源，获取控制设备句柄。
参数	SPI_ID_t id SPI 外设 ID; const SPI_InitParam_t * param SPI 初始化参数的结构体地址; typedef struct { SPI_Mode_t mode; // MASTER 或 SLAVE SPI_SclkMode_t sclkMode; // 时钟模式 0~3 SPI_IOMode_t ioMode; // SPI 模式，Stand/dual/quad SPI_Cs_t csMode; // CS 控制模式，硬件控制还是软件 SPI_DataWidth_t dataWidth; // 数据位宽 uint32_t sclk; // 时钟频率 } SPI_InitParam_t;
注意事项	无
返回值	UART 设备控制句柄(HAL_Dev_t *类型)

5.3.3.2 HAL_SPI_DeInit

表 5- 61 HAL_SPI_DeInit

接口	说明
函数	HAL_Status_t HAL_SPI_DeInit(HAL_Dev_t *spi);
功能	释放指定 SPI 驱动资源。
参数	HAL_Dev_t *spi SPI 设备控制句柄。
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.3.3.3 HAL_SPI_Open

表 5- 62 HAL_SPI_Open

接口	说明
函数	HAL_Status_t HAL_SPI_Open(HAL_Dev_t *spi, uint8_t slave_addrIndex);
功能	开启 SPI 硬件
参数	HAL_Dev_t *spi SPI 设备控制句柄。 uint8_t slave_addrIndex 从机选择索引
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.3.3.4 HAL_SPI_Close

表 5- 63 HAL_SPI_Close

接口	说明
函数	HAL_Status_t HAL_SPI_Close(HAL_Dev_t *spi);
功能	关闭 SPI 硬件
参数	HAL_Dev_t *spi SPI 设备控制句柄。
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.3.3.5 HAL_SPI_ExtControl

表 5- 64 HAL_SPI_ExtControl

接口	说明
函数	HAL_Status_t HAL_SPI_ExtControl(const HAL_Dev_t *spi, SPI_ParamType_t ptype, void *param);
功能	SPI 不常用扩展控制。

接口	说明
参数	HAL_Dev_t *spi SPI 设备控制句柄。 SPI_ParamType_t ptype 配置参数类型 <pre>typedef enum { SPI_PARAM_TX_START_LEVEL = 0, // Tx FIFO 启动发送阈值 SPI_PARAM_TX_EMPTY_LEVEL = 1, // Tx FIFO 发送空阈值 SPI_PARAM_RX_FULL_LEVEL = 2, // Rx FIFO 接收满阈值 SPI_PARAM_TX_DMA_LEVEL = 3, // Tx FIFO DMA 触发阈值 SPI_PARAM_RX_DMA_LEVEL = 4, // Rx FIFO DMA 触发阈值 SPI_PARAM_FRAME_CTRL = 5, // cmd 格式, cmd, addr, data 各段设置 SPI_PARAM_WAKEUP_THRESHOLD = 6, // 唤醒 CLK 阈值 SPI_PARAM_WAKEUP_CTRL = 7, // 唤醒控制 SPI_PARAM_WAKEUP_CALLBACK = 8, // 唤醒回调函数 SPI_PARAM_WAKEUP_MODE = 9, // 唤醒模式 } SPI_ParamType_t;</pre> void *param 读取或写入的配置参数
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.3.3.6 HAL_SPI_SetTransferMode

表 5- 65 HAL_SPI_SetTransferMode

接口	说明
函数	HAL_Status_t HAL_SPI_SetTransferMode(const HAL_Dev_t *spi, SPI_TransDir_t transDir, SPI_TransMode_t Mode);
功能	设置传输模式
参数	HAL_Dev_t *spi SPI 设备控制句柄。

接口	说明
	SPI_TransDir_t transDir 传输方向 typedef enum { SPI_DIR_TX = 0, // 只发送 SPI_DIR_RX = 1, // 只接收 SPI_DIR_FULL = 2, // 发送和接收同时进行 } SPI_TransDir_t; SPI_TransMode_t Mode 传输模式 typedef enum { SPI_TRANS_MODE_NOMA = 0, // 轮询方式 SPI_TRANS_MODE_INTERRUPT = 1, // 中断方式 SPI_TRANS_MODE_DMA = 2, // DMA 方式 } SPI_TransMode_t;
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.3.3.7 HAL_SPI_Receive

表 5- 66 HAL_SPI_Receive

接口	说明
函数	<code>int32_t HAL_SPI_Receive(const HAL_Dev_t *spi, uint8_t *buff, int32_t size, uint32_t msec);</code>
功能	SPI 接收数据
参数	HAL_Dev_t *spi SPI 设备控制句柄。 uint8_t *buff 接收数据 buff int32_t size

接口	说明
	期望接收到的数据长度 uint32_t msec 超时，单位 ms
注意事项	无
返回值	若值为负数则表示错误码，若为正数表示接收成功的数据个数。

5.3.3.8 HAL_SPI_Transmit

表 5- 67 HAL_SPI_Transmit

接口	说明
函数	int32_t HAL_SPI_Transmit(const HAL_Dev_t *spi, const uint8_t *buff, int32_t size, uint32_t msec);
功能	发送数据
参数	HAL_Dev_t *spi SPI 设备控制句柄。 uint8_t *buff 发送数据 buff int32_t size 期望发送的数据长度 uint32_t msec 超时，单位 ms
注意事项	无
返回值	若值为负数则表示错误码，若为正数表示发送成功的数据个数。

5.3.3.9 HAL_SPI_TransmitReceive

表 5- 68 HAL_SPI_TransmitReceive

接口	说明
函数	int32_t HAL_SPI_TransmitReceive(const HAL_Dev_t *spi, uint8_t *tx_data,

接口	说明
	uint8_t *rx_data, int32_t size, uint32_t msec);
功能	发送和接收通信进行的传输模式
参数	HAL_Dev_t *spi SPI 设备控制句柄。 uint8_t *tx_data 发送数据 buff uint8_t *rx_data 接收数据 buff int32_t size 期望传输的数据长度 uint32_t msec 超时，单位 ms
注意事项	无
返回值	若值为负数则表示错误码，若为正数表示传输成功的数据个数。

5.3.3.10 HAL_SPI_GetRxData

表 5- 69 HAL_SPI_GetRxData

接口	说明
函数	int32_t HAL_SPI_GetRxData(const HAL_Dev_t *spi);
功能	从 FIFO 获取接收数据
参数	HAL_Dev_t *spi SPI 设备控制句柄。
注意事项	需与 HAL_SPI_IsRxReady 配合使用。
返回值	从 FIFO 读取的数据。

5.3.3.11 HAL_SPI_PutTxData

表 5- 70 HAL_SPI_PutTxData

接口	说明
函数	void HAL_SPI_PutTxData(const HAL_Dev_t *spi, uint32_t data);
功能	写入数据到 Tx FIFO
参数	HAL_Dev_t *spi SPI 设备控制句柄。 uint32_t data 待写入的数据
注意事项	无
返回值	空

5.3.3.12 HAL_SPI_StartTrans

表 5- 71 HAL_SPI_StartTrans

接口	说明
函数	HAL_Status_t HAL_SPI_StartTrans(const HAL_Dev_t *spi , SPI_TransDir_t dir);
功能	开始传输
参数	HAL_Dev_t *spi SPI 设备控制句柄。 SPI_TransDir_t dir 传输方向
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.3.3.13 HAL_SPI_StopTrans

表 5- 72 HAL_SPI_StopTrans

接口	说明
函数	HAL_Status_t HAL_SPI_StopTrans(const HAL_Dev_t *spi);
功能	停止传输

接口	说明
参数	HAL_Dev_t *spi SPI 设备控制句柄。
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.3.3.14 HAL_SPI_IsTxReady

表 5- 73 HAL_SPI_IsTxReady

接口	说明
函数	int HAL_SPI_IsTxReady(HAL_Dev_t *spi);
功能	检查 Tx FIFO 是否超过阈值
参数	HAL_Dev_t *spi SPI 设备控制句柄。
注意事项	无
返回值	0 超过，1 未超过

5.3.3.15 HAL_SPI_IsTxEmpty

表 5- 74 HAL_SPI_IsTxEmpty

接口	说明
函数	int HAL_SPI_IsTxEmpty(HAL_Dev_t *spi);
功能	检查发送 FIFO 是否空
参数	HAL_Dev_t *spi SPI 设备控制句柄。
注意事项	无
返回值	1 空，0 非空

5.3.3.16 HAL_SPI_IsRxReady

表 5- 75 HAL_SPI_IsRxReady

接口	说明
函数	int HAL_SPI_IsRxReady(HAL_Dev_t *spi);

接口	说明
功能	检查 Rx FIFO 非空
参数	HAL_Dev_t *spi SPI 设备控制句柄。
注意事项	无
返回值	1 非空，0 空

5.3.3.17 HAL_SPI_IsBusy

表 5- 76 HAL_SPI_IsBusy

接口	说明
函数	int HAL_SPI_IsBusy(HAL_Dev_t *spi);
功能	检查 SPI 是否处于收发状态
参数	HAL_Dev_t *spi SPI 设备控制句柄。
注意事项	无
返回值	1 处于收发状态，0 已空闲

5.3.3.18 HAL_SPI_GetFifoLevel

表 5- 77 HAL_SPI_GetFifoLevel

接口	说明
函数	int HAL_SPI_GetFifoLevel(HAL_Dev_t *spi, SPI_FifoType_t type);
功能	获取 FIFO 阈值
参数	HAL_Dev_t *spi SPI 设备控制句柄。 SPI_FifoType_t type FIFO 类型 typedef enum { SPI_FIFO_TX = 0, SPI_FIFO_RX = 1, } SPI_FifoType_t;

接口	说明
注意事项	无
返回值	FIFO 设置的阈值

5.3.3.19 HAL_SPI_IsCsInterrupt

表 5- 78 HAL_SPI_IsCsInterrupt

接口	说明
函数	int HAL_SPI_IsCsInterrupt(HAL_Dev_t *spi);
功能	检查从机是否被选择
参数	HAL_Dev_t *spi SPI 设备控制句柄。
注意事项	无
返回值	1 被选择, 0 不被选择

5.3.3.20 HAL_SPI_EnableDMA

表 5- 79 HAL_SPI_EnableDMA

接口	说明
函数	HAL_Status_t HAL_SPI_EnableDMA(const HAL_Dev_t *spi , SPI_TransDir_t dir);
功能	使能 DMA 传输
参数	HAL_Dev_t *spi SPI 设备控制句柄。 SPI_TransDir_t dir 传输方向
注意事项	无
返回值	返回值为 HAL_STATUS_OK, 表示函数执行成功; 返回值为负数表示函数执行失败, HAL_STATUS_*表示失败类型。

5.3.3.21 HAL_SPI_DisableDMA

表 5- 80 HAL_SPI_DisableDMA

接口	说明
函数	HAL_Status_t HAL_SPI_DisableDMA(const HAL_Dev_t *spi , SPI_TransDir_t dir);
功能	关闭 DMA 传输

接口	说明
参数	HAL_Dev_t *spi SPI 设备控制句柄。 SPI_TransDir_t dir 传输方向
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.3.3.22 HAL_SPI_EnableCS

表 5- 81 HAL_SPI_EnableCS

接口	说明
函数	HAL_Status_t HAL_SPI_EnableCS(const HAL_Dev_t *spi, uint8_t csId);
功能	软件控制 CS 使能
参数	HAL_Dev_t *spi SPI 设备控制句柄。 uint8_t csId 从机选择索引
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.3.3.23 HAL_SPI_DisableCS

表 5- 82 HAL_SPI_DisableCS

接口	说明
函数	HAL_Status_t HAL_SPI_DisableCS(const HAL_Dev_t *spi, uint8_t csId);
功能	软件控制 CS 关闭
参数	HAL_Dev_t *spi SPI 设备控制句柄。 uint8_t csId

接口	说明
	从机选择索引
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.3.3.24 HAL_SPI_RegisterIRQ

表 5- 83 HAL_SPI_RegisterIRQ

接口	说明
函数	HAL_Status_t HAL_SPI_RegisterIRQ(HAL_Dev_t *spi, SPI_IRQ_t irq_type, const HAL_Callback_t *irqCb);
功能	注册中断服务函数
参数	HAL_Dev_t *spi SPI 设备控制句柄。 SPI_IRQ_t irq_type 中断类型 typedef enum { SPI_IRQ_TX = 0, SPI_IRQ_RX = 1, SPI_IRQ_CS = 2, } SPI_IRQ_t; HAL_Callback_t *irqCb 回调函数
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.3.3.25 HAL_SPI_UnregisterIRQ

表 5- 84 HAL_SPI_UnregisterIRQ

接口	说明
函数	HAL_Status_t HAL_SPI_UnregisterIRQ(const HAL_Dev_t *spi, SPI_IRQ_t irq_type);
功能	注销中断处理
参数	HAL_Dev_t *spi

接口	说明
	SPI 设备控制句柄。 SPI_IRQ_t irq_type 中断类型 <pre>typedef enum { SPI_IRQ_TX = 0, SPI_IRQ_RX = 1, SPI_IRQ_CS = 2, } SPI_IRQ_t;</pre>
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.3.3.26 HAL_SPI_EnableIRQ

表 5- 85 HAL_SPI_EnableIRQ

接口	说明
函数	HAL_Status_t HAL_SPI_EnableIRQ(const HAL_Dev_t *spi, SPI_IRQ_t irq_type);
功能	使能中断
参数	HAL_Dev_t *spi SPI 设备控制句柄。 SPI_IRQ_t irq_type 中断类型 <pre>typedef enum { SPI_IRQ_TX = 0, SPI_IRQ_RX = 1, SPI_IRQ_CS = 2, } SPI_IRQ_t;</pre>
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.3.3.27 HAL_SPI_DisableIRQ

表 5- 86 HAL_SPI_DisableIRQ

接口	说明
函数	HAL_Status_t HAL_SPI_DisableIRQ(const HAL_Dev_t *spi, SPI_IRQ_t irq_type);
功能	关闭中断
参数	HAL_Dev_t *spi SPI 设备控制句柄。 SPI_IRQ_t irq_type 中断类型 typedef enum { SPI_IRQ_TX = 0, SPI_IRQ_RX = 1, SPI_IRQ_CS = 2, } SPI_IRQ_t;
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.3.3.28 HAL_SPI_RegisterDMA

表 5- 87 HAL_SPI_RegisterDMA

接口	说明
函数	HAL_Status_t HAL_SPI_RegisterDMA(HAL_Dev_t *spi, SPI_TransDir_t transDir, HAL_Callback_t *dmaCb);
功能	注册 DMA 传输回调函数。
参数	HAL_Dev_t *spi SPI 设备控制句柄。 SPI_TransDir_t transDir 传输方向 HAL_Callback_t *irqCb 回调函数
注意事项	无

接口	说明
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.3.3.29 HAL_SPI_UnregisterDMA

表 5- 88 HAL_SPI_UnregisterDMA

接口	说明
函数	HAL_Status_t HAL_SPI_UnregisterDMA(const HAL_Dev_t *spi, SPI_TransDir_t transDir);
功能	注销中断服务
参数	HAL_Dev_t *spi SPI 设备控制句柄。 SPI_TransDir_t transDir 传输方向
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.3.3.30 HAL_SPI_StartDMA

表 5- 89 HAL_SPI_StartDMA

接口	说明
函数	HAL_Status_t HAL_SPI_StartDMA(const HAL_Dev_t *spi, SPI_TransDir_t transDir, const uint8_t *buf, int32_t size);
功能	开始 DMA 传输 SPI
参数	HAL_Dev_t *spi SPI 设备控制句柄。 SPI_TransDir_t transDir 传输方向 uint8_t *buf 传输 buff

接口	说明
	int32_t size 传输长度
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.3.3.31 HAL_SPI_StopDMA

表 5- 90 HAL_SPI_StopDMA

接口	说明
函数	int32_t HAL_SPI_StopDMA(const HAL_Dev_t *spi, SPI_TransDir_t transDir);
功能	停止 DMA 传输 SPI
参数	HAL_Dev_t *spi SPI 设备控制句柄。 SPI_TransDir_t transDir 传输方向
注意事项	无
返回值	负数为错误码 正数为传输成功数据个数

5.3.3.32 HAL_SPI_PM_Suspend

表 5- 91 HAL_SPI_PM_Suspend

接口	说明
函数	HAL_Status_t HAL_SPI_PM_Suspend(HAL_Dev_t *spi);
功能	休眠 SPI
参数	HAL_Dev_t *spi SPI 设备控制句柄。
注意事项	无

接口	说明
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.3.3.33 HAL_SPI_PM_Resume

表 5- 92 HAL_SPI_PM_Resume

接口	说明
函数	HAL_Status_t HAL_SPI_PM_Resume(HAL_Dev_t *spi);
功能	唤醒恢复 SPI
参数	HAL_Dev_t *spi SPI 设备控制句柄。
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.4 I2C

5.4.1 功能简介

5.4.1.1 特性

- 1. 支持主机和从机模式；
- 2. 支持 100K，400k，1M 速率；
- 3. 支持从机选配地址；

5.4.1.2 使用

I2C 的 HAL API 提供了两种方式实现上述特性：

- 1. 集成 API，通过 Init，Open，Receive，Transmit 等 API 即可实现 SP 的收发不需要关注细节。
- 2. 分散 API，通过单一功能 API 实现更复杂和个性化化的功能，需要更多关注细节。

建议使用继承 API。

Sample:

```
project/driver/i2c/i2c_poll_master
project/driver/i2c/i2c_poll_slave
project/driver/i2c/i2c_interrupt_master
project/driver/i2c/i2c_interrupt_slave
```

集成 API 方式使用流程如下：

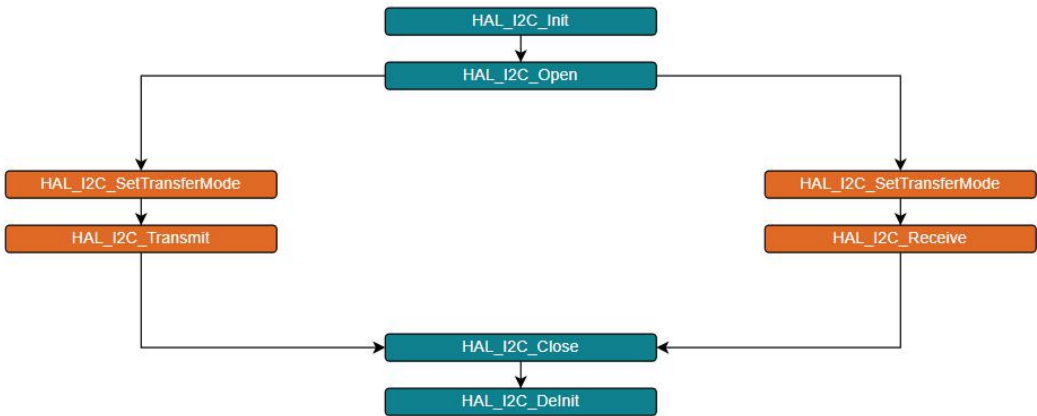


图 5-10 I2C 集成 API 使用流程

HAL_I2C_Init 用于配置 i2c 特性，配置参数如下。

```
typedef struct {
    I2C_Mode_t      mode;      /* master 或 slave */
    I2C_Speed_t     speed;     /* 100k, 400k, 1M 选择 */
    I2C_Callback_t  busErrCb; /* 出错回调 */
}
```

```
} I2C_InitParam_t;
```

I2C 总线容易挂死，busErrCb 提供了一个恢复总线的接口，通常根据类型调用 HAL_I2C_RecoverBus 恢复。

功能相关配置宏

/* 是否开启 I2C Slave 模式，0 为关闭；1 为开启 */ 注：默认选择 I2C_SLAVE 模式	
#define CONFIG_I2C_SLAVE	1
/* 是否开启 I2C master 模式，0 为关闭；1 为开启 */	
#define CONFIG_I2C_MASTER	0
/* 是否开启 I2C 中断传输模式，0 为关闭；1 为开启 */	
#define CONFIG_I2C_TRANS_IT_ENABLE	0
/* 是否开启 I2C 参数检查，0 为关闭；1 为开启 */	
#define CONFIG_I2C_ASSERT	0
/* 是否开启 I2C 休眠模式通过寻址唤醒，0 为关闭；1 为开启 */	
#define CONFIG_I2C_WAKEUP	0
Prj_config.h	
/* 开启板级 MRS6130_p1812 */	
#define CONFIG_BOARD_MRS6130_P1812	1
/* 是否开启 I2C master 模式，0 为关闭；1 为开启 */	
#define CONFIG_I2C_MASTER	1
/* 是否开启 I2C 中断传输模式，0 为关闭；1 为开启 */	
#define CONFIG_I2C_TRANS_IT_ENABLE	0

5.4.2 接口

5.4.2.1 HAL_I2C_Init

表 5-93 HAL_I2C_init

接口	说明
函数	HAL_Dev_t *HAL_I2C_Init(I2C_ID_t id, const I2C_InitParam_t *pinitParam);
功能	初始化 I2C 设备，申请系统资源，获取控制设备句柄。
参数	I2C_ID_t id I2C 外设 ID; typedef enum {

接口	说明
	<pre>I2C0_ID = 0U, I2C1_ID = 1U, I2C_NUM, } I2C_ID_t; const I2C_InitParam_t *pinitParam I2C 初始化参数的结构体地址; I2C 速度; I2C 模式; 总线异常触发回调; typedef struct { I2C_Mode_t mode; I2C_Speed_t speed; I2C_Callback_t busErrCb; } I2C_InitParam_t;</pre>
注意事项	无。
返回值	I2C 设备实例句柄(HAL_Dev_t *类型)。

5.4.2.2 HAL_I2C_DeInit

表 5- 94 HAL_I2C_Deinit

接口	说明
函数	HAL_Status_t HAL_I2C_DeInit(HAL_Dev_t *i2c);
功能	反初始化 I2C 设备，释放系统资源。
参数	HAL_Dev_t *i2c I2C 设备实例句柄。
注意事项	无。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.4.2.3 HAL_I2C_Open

表 5- 95 HAL_I2C_Open

接口	说明
函数	HAL_Status_t HAL_I2C_Open(HAL_Dev_t *i2c, I2C_AddrWidth_t addrWidth, const uint16_t addr);
功能	打开配置地址的总线设备，启动外设。

接口	说明
参数	HAL_Dev_t *i2c I2C 设备实例句柄。 I2C_AddrWidth_t addrWidth 启动时配置的地址位宽； <pre>typedef enum { I2C_ADDR_WIDTH_7BIT = 0U, I2C_ADDR_WIDTH_10BIT = 1U, } I2C_AddrWidth_t;</pre> const uint16_t addr 启动时配置的地址；
注意事项	1. 初始化为 I2C_MODE_MASTER 时，配置的地址为目标地址； 2. 初始化为 I2C_MODE_SLAVE 时，配置的地址为从机地址； 3. 注意不可重复调用该接口； 4. Open 行为执行完后，如需修改地址需要 Close 后再 Open。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.4.2.4 HAL_I2C_Open

表 5- 96 HAL_I2C_Close

接口	说明
函数	HAL_Status_t HAL_I2C_Open(HAL_Dev_t *i2c)
功能	关闭外设。
参数	HAL_Dev_t *i2c I2C 设备实例句柄。
注意事项	关闭外设需要首先对外设有 Open 行为。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.4.2.5 HAL_I2C_RecoverBus

表 5- 97 HAL_I2C_RecoverBus

接口	说明
----	----

接口	说明
函数	HAL_Status_t HAL_I2C_RecoverBus(HAL_Dev_t *i2c)
功能	恢复总线。
参数	HAL_Dev_t *i2c I2C 设备实例句柄。
注意事项	无。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.4.2.6 HAL_I2C_SetTransferMode

表 5- 98 HAL_I2C_SetTransferMode

接口	说明
函数	HAL_Status_t HAL_I2C_SetTransferMode(HAL_Dev_t *i2c, I2C_TransfDir_t tranfDir, I2C_TransfMode_t tranfMode);
功能	配置 Transmit 和 Receive 两个方向的传输模式。
参数	HAL_Dev_t *i2c I2C 设备实例句柄。 I2C_TransfDir_t tranfDir 传输方向。 typedef enum { I2C_TRANSFDIR_TX = 0U, I2C_TRANSFDIR_RX = 1U, } I2C_TransfDir_t; I2C_TransfMode_t tranfMode 传输类型；轮询（NORMAL）、中断、DMA 三种模式。 typedef enum { I2C_TRANSFMODE_NORMAL = 0U, I2C_TRANSFMODE_INTERRUPT = 1U, I2C_TRANSFMODE_DMA = 2U, } I2C_TransfMode_t;
注意事项	默认为 NORMAL 模式。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功；

接口	说明
	返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.4.2.7 HAL_I2C_Transmit

表 5- 99 HAL_I2C_Transmit

接口	说明
函数	int32_t HAL_I2C_Transmit(HAL_Dev_t *i2c, const uint8_t *buff, uint32_t size, uint32_t msec);
功能	以 HAL_I2C_SetTransferMode 配置的传输类型，传输定长数据。
参数	HAL_Dev_t *i2c I2C 设备实例句柄。 const uint8_t *buff 传输数据的地址。 uint32_t size 传输数据的大小。 uint32_t msec 传输超时（ms）。
注意事项	◆ 默认不配置为 NORMAL 模式下进行传输数据。
返回值	返回值为非负整数时，表示函数传输的实际长度； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.4.2.8 HAL_I2C_Receive

表 5- 100 HAL_I2C_Receive

接口	说明
函数	int32_t HAL_I2C_Receive(HAL_Dev_t *i2c, const uint8_t *buff, uint32_t size, uint32_t msec);
功能	以 HAL_I2C_SetTransferMode 配置的接收类型，接收定长数据。
参数	HAL_Dev_t *i2c I2C 设备实例句柄。 const uint8_t *buff 接收数据的地址。

接口	说明
	uint32_t size 接收数据的大小。 uint32_t msec 接收超时（ms）。
注意事项	默认不配置为 NORMAL 模式下进行接收数据。
返回值	返回值为非负整数时，表示函数接收的实际长度； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.4.2.9 HAL_I2C_GetRxData

表 5- 101 HAL_I2C_GetRxData

接口	说明
函数	uint8_t HAL_I2C_GetRxData(HAL_Dev_t *i2c);
功能	获取 RX FIFO 数据。
参数	HAL_Dev_t *i2c I2C 设备实例句柄。
注意事项	无
返回值	返回值为获取 Rx FIFO 里的数据。

5.4.2.10 HAL_I2C_PutTxData

表 5- 102 HAL_I2C_PutTxData

接口	说明
函数	void HAL_I2C_PutTxData(HAL_Dev_t *i2c, uint8_t txData, I2C_Cmd_t cmd);
功能	放入 TX FIFO 数据。
参数	HAL_Dev_t *i2c I2C 设备实例句柄。 uint8_t txData 传输的 1 Byte 数据 I2C_Cmd_t cmd 该数据具备的总线信号。 typedef enum { I2C_CMD_WRITE = 0x00U, I2C_CMD_READ = CMD_Msk, I2C_CMD_WRITE_STOP = STOP_Msk, I2C_CMD_WRITE_RESTART = RESTART_Msk, I2C_CMD_WRITE_RESTART_STOP = STOP_Msk RESTART_Msk, I2C_CMD_READ_STOP = CMD_Msk STOP_Msk, I2C_CMD_READ_RESTART = CMD_Msk RESTART_Msk, I2C_CMD_READ_RESTART_STOP = CMD_Msk STOP_Msk RESTART_Msk, } I2C_Cmd_t;
注意事项	1. I2C_MODE_SLAVE 模式下，参数固定填 I2C_CMD_WRITE；

接口	说明
	2. 应该判断 TX FIFO 是否数据已满再放入数据。
返回值	void

5.4.2.11 HAL_I2C_IsTxReady

表 5- 103 HAL_I2C_IsTxReady

接口	说明
函数	bool HAL_I2C_IsTxReady(HAL_Dev_t *i2c);
功能	判断 Tx FIFO 是否就绪。
参数	HAL_Dev_t *i2c I2C 设备实例句柄。
注意事项	无
返回值	True : Tx FIFO 不为空, Tx FIFO 就绪; False: Tx FIFO 为满, Tx FIFO 不就绪。

5.4.2.12 HAL_I2C_IsRxReady

表 5- 104 HAL_I2C_IsRxReady

接口	说明
函数	bool HAL_I2C_IsRxReady(HAL_Dev_t *i2c);
功能	判断 Rx FIFO 是否就绪。
参数	HAL_Dev_t *i2c I2C 设备实例句柄。
注意事项	无
返回值	True : Rx FIFO 为满, Rx FIFO 就绪; False: Rx FIFO 不为满, Rx FIFO 不就绪。

5.4.2.13 HAL_I2C_IsTxEmpty

表 5- 105 HAL_I2C_IsTxEmpty

接口	说明
函数	bool HAL_I2C_IsTxEmpty(HAL_Dev_t *i2c);
功能	判断 Rx FIFO 是否就绪。
参数	HAL_Dev_t *i2c

接口	说明
	I2C 设备实例句柄。
注意事项	无
返回值	True : Tx FIFO 为空； False: Tx FIFO 不为空。

5.4.2.14 HAL_I2C_RegisterIRQ

表 5- 106 HAL_I2C_RegisterIRQ

接口	说明
函数	HAL_Status_t HAL_I2C_RegisterIRQ(HAL_Dev_t *i2c, I2C_Callback_t *irqCb);
功能	注册中断服务函数调用的回调函数。
参数	HAL_Dev_t *i2c I2C 设备实例句柄。 I2C_Callback_t *irqCb 注册中断回调函数的结构体地址；
注意事项	使用时，应该需要注意会使中断模式的 Transmit 和 Receive 函数失效。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.4.2.15 HAL_I2C_UnregisterIRQ

表 5- 107 HAL_I2C_UnregisterIRQ

接口	说明
函数	HAL_Status_t HAL_I2C_UnregisterIRQ(HAL_Dev_t *i2c);
功能	反注册中断服务函数调用的回调函数。
参数	HAL_Dev_t *i2c I2C 设备实例句柄。
注意事项	无。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.4.2.16 HAL_I2C_EnableIRQ

表 5- 108 HAL_I2C_EnableIRQ

接口	说明
函数	HAL_Status_t HAL_I2C_EnableIRQ(HAL_Dev_t *i2c, uint32_t irqStatus);
功能	使能中断类型。
参数	HAL_Dev_t *i2c I2C 设备实例句柄。 uint32_t irqStatus 使能的中断类型； <pre>typedef enum { I2C_IRQSTATUS_START = HAL_BIT(0), I2C_IRQSTATUS_STOP = HAL_BIT(1), I2C_IRQSTATUS_READ_REQ = HAL_BIT(2), I2C_IRQSTATUS_READ_DONE = HAL_BIT(3), I2C_IRQSTATUS_ERROR = HAL_BIT(4), I2C_IRQSTATUS_TX_FIFO_EMPTY = HAL_BIT(5), I2C_IRQSTATUS_RX_FIFO_FULL = HAL_BIT(6), I2C_IRQSTATUS_ALL = HAL_BIT(0) HAL_BIT(1) HAL_BIT(2) HAL_BIT(3) HAL_BIT(4) HAL_BIT(5) HAL_BIT(6), } I2C_IRQStatus_t;</pre>
注意事项	使能相关标志后，在中断服务函数里面要对相应的中断状态进行判断和清除；
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.4.2.17 HAL_I2C_DisableIRQ

表 5- 109 HAL_I2C_DisableIRQ

接口	说明
函数	HAL_Status_t HAL_I2C_DisableIRQ(HAL_Dev_t *i2c, uint32_t irqStatus);
功能	失能中断类型。
参数	HAL_Dev_t *i2c I2C 设备实例句柄。 uint32_t irqStatus 失能的中断类型；

接口	说明
	<pre>typedef enum { I2C_IRQSTATUS_START = HAL_BIT(0), I2C_IRQSTATUS_STOP = HAL_BIT(1), I2C_IRQSTATUS_READ_REQ = HAL_BIT(2), I2C_IRQSTATUS_READ_DONE = HAL_BIT(3), I2C_IRQSTATUS_ERROR = HAL_BIT(4), I2C_IRQSTATUS_TX_FIFO_EMPTY = HAL_BIT(5), I2C_IRQSTATUS_RX_FIFO_FULL = HAL_BIT(6), I2C_IRQSTATUS_ALL = HAL_BIT(0) HAL_BIT(1) HAL_BIT(2) HAL_BIT(3) HAL_BIT(4) HAL_BIT(5) HAL_BIT(6), } I2C_IRQStatus_t;</pre>
注意事项	无。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.4.2.18 HAL_I2C_ClearIRQ

表 5- 110 HAL_I2C_ClearIRQ

接口	说明
函数	HAL_Status_t HAL_I2C_ClearIRQ(HAL_Dev_t *i2c, uint32_t irqStatus);
功能	失能中断类型。
参数	HAL_Dev_t *i2c I2C 设备实例句柄。 uint32_t irqStatus 清除的中断类型； <pre>typedef enum { I2C_IRQSTATUS_START = HAL_BIT(0), I2C_IRQSTATUS_STOP = HAL_BIT(1), I2C_IRQSTATUS_READ_REQ = HAL_BIT(2), I2C_IRQSTATUS_READ_DONE = HAL_BIT(3), I2C_IRQSTATUS_ERROR = HAL_BIT(4), I2C_IRQSTATUS_TX_FIFO_EMPTY = HAL_BIT(5), I2C_IRQSTATUS_RX_FIFO_FULL = HAL_BIT(6), I2C_IRQSTATUS_ALL = HAL_BIT(0) HAL_BIT(1) HAL_BIT(2) HAL_BIT(3) HAL_BIT(4) HAL_BIT(5) HAL_BIT(6), } I2C_IRQStatus_t;</pre>

接口	说明
注意事项	无。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.4.2.19 HAL_I2C_GetIRQStatus

表 5- 111 HAL_I2C_GetIRQStatus

接口	说明
函数	uint32_t HAL_I2C_GetIRQStatus(HAL_Dev_t *i2c);
功能	获取已触发的中断类型。
参数	HAL_Dev_t *i2c I2C 设备实例句柄。 uint32_t irqStatus 清除的中断类型； typedef enum { I2C_IRQSTATUS_START = HAL_BIT(0), I2C_IRQSTATUS_STOP = HAL_BIT(1), I2C_IRQSTATUS_READ_REQ = HAL_BIT(2), I2C_IRQSTATUS_READ_DONE = HAL_BIT(3), I2C_IRQSTATUS_ERROR = HAL_BIT(4), I2C_IRQSTATUS_TX_FIFO_EMPTY = HAL_BIT(5), I2C_IRQSTATUS_RX_FIFO_FULL = HAL_BIT(6), I2C_IRQSTATUS_ALL = HAL_BIT(0) HAL_BIT(1) HAL_BIT(2) HAL_BIT(3) HAL_BIT(4) HAL_BIT(5) HAL_BIT(6), } I2C_IRQStatus_t;
注意事项	1. 需要再调用注册的中断回调函数中先获取中断触发类型后，根据获取的中断触发类型执行相应的方法函数； 2. GetRxData 函数需要触发 I2C_IRQSTATUS_RX_FIFO_FULL 信号再接收数据； 3. PutTxData 函数需要触发 I2C_IRQSTATUS_TX_FIFO_EMPTY 信号再放入数据。
返回值	返回值为获取的触发类型。 类型描述为以下枚举类型： typedef enum { I2C_IRQSTATUS_START = HAL_BIT(0), I2C_IRQSTATUS_STOP = HAL_BIT(1), I2C_IRQSTATUS_READ_REQ = HAL_BIT(2),

接口	说明
	I2C_IRQSTATUS_READ_DONE = HAL_BIT(3),
	I2C_IRQSTATUS_ERROR = HAL_BIT(4),
	I2C_IRQSTATUS_TX_FIFO_EMPTY = HAL_BIT(5),
	I2C_IRQSTATUS_RX_FIFO_FULL = HAL_BIT(6),
	I2C_IRQSTATUS_ALL = HAL_BIT(0) HAL_BIT(1) HAL_BIT(2) HAL_BIT(3) HAL_BIT(4) HAL_BIT(5) HAL_BIT(6),
	} I2C_IRQStatus_t;

5.5 DMA

5.5.1 功能简介

DMA 支持以下属性：

- 1. 支持内存到内存、内存到外设、外设到内存三种方向的数据传输；
- 2. 支持 8 通道并行使用；
- 3. 支持最大 32 位宽搬运数据；
- 4. 支持块传输、链表模式传输；
- 5. 每个通道 FIFO 32 bits X 8。

Sample：

[project/driver/dma/mem2mem_block](#)
[project/driver/dma/mem2mem_list](#)

5.5.1.1 块传输

块传输使用流程如下图 。

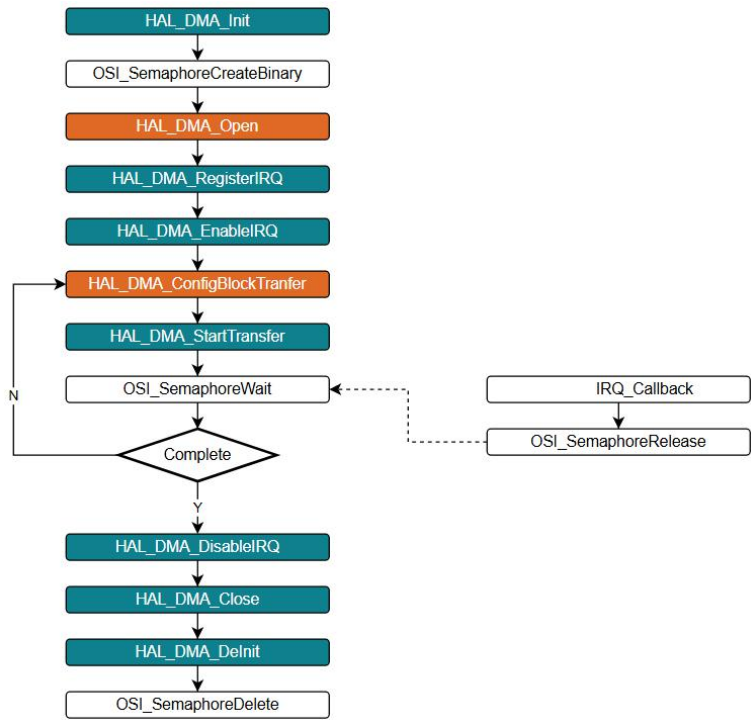


图 5- 11 DMA 块传输使用流程

其中重要的接口为“HAL_DMA_Open”和“HAL_DMA_ConfigBlockTranfer”。HAL_DMA_Open 用于申请一个未使用的通道和配置通道通用属性，调用一次即可。HAL_DMA_ConfigBlockTranfer 用于配置每次传输的数据参数，与 HAL_DMA_StartTransfer 搭配可多次调用。

HAL_DMA_Open 的通道参数如下。

```
typedef struct {
    uint32_t direction      : 4;
    /* 通道优先级，取值 0~7，多通道同时使用时数字越大优先级越高 */
    uint32_t priority       : 4;
    /* 源端地址增长模式，递增，递减，固定，外设选择固定，memory 选择递增或递减 */
    uint32_t srcIncrMode    : 4;
    /* 目的端地址增长模式，递增，递减，固定，外设选择固定，memory 选择递增或递减 */
    uint32_t dstIncrMode    : 4;
    /* 块传输数量，memory to memory 无效，外设与 memory 之间与外设和 DMA FIFO 深度有关 */
    uint32_t srcBrustNum    : 4;
    uint32_t dstBrustnum    : 4;
    /* 源端数据宽度，memory 类型可选择 8/16/32，外设类型需与传输位宽一致通常为 8 */
    uint32_t srcWidth       : 4;
    /* 目的端数据宽度，memory 类型可选择 8/16/32，外设类型需与传输位宽一致通常为 8 */
    uint32_t dstWidth       : 4;
    /* 源端接口 ID，可选择 UART0_TX, UART_RX 等 */
    uint16_t srcHandShake   : 4;
    /* 目的端接口 ID，可选择 UART0_TX, UART_RX 等 */
    uint16_t dstHandShake   : 4;
    /* 源端起始地址是否重载，见 HAL_DMA_ConfigBlockTranfer 的 DMA_BlockConf_t */
}
```

```
uint16_t  srcEnReload      : 1;
/* 目的端起始地址是否重载，见 HAL_DMA_ConfigBlockTranfer 的 DMA_BlockConf_t */
uint16_t  dstEnReload     : 1;
uint16_t  srcEnStatus     : 1;
uint16_t  dstEnStatus     : 1;
uint16_t  pad             : 4;
} DMA_ChannelConf_t;
```

Memory to memory

常用的配置有如下两种，位宽配置为 DMA_BITWIDTH_32 传输效率高但需要源地址或目标地址 32bit 对齐，位宽为 DMA_BITWIDTH_8 传输效率一般但不需要源地址或目标地址对齐,使用者可根据需要选择。IncrMode 在此场景选择 DMA_ADDR_INCREMENT，blockSize 根据传输长度填写单位 Bytes，如传输 64 Bytes 则配置为 64 与 Width 参数无关。

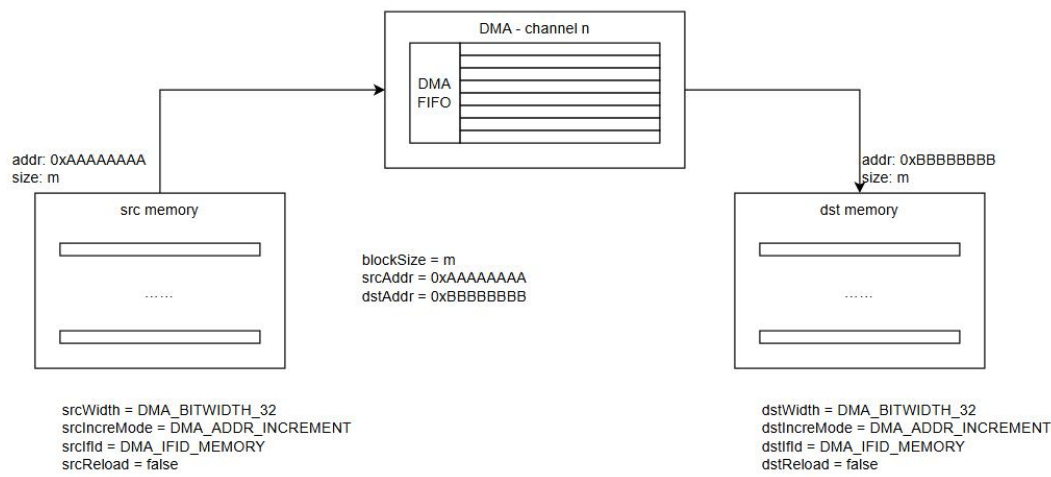


图 5- 12 DMA memory to memory 位宽 32bit 示意图

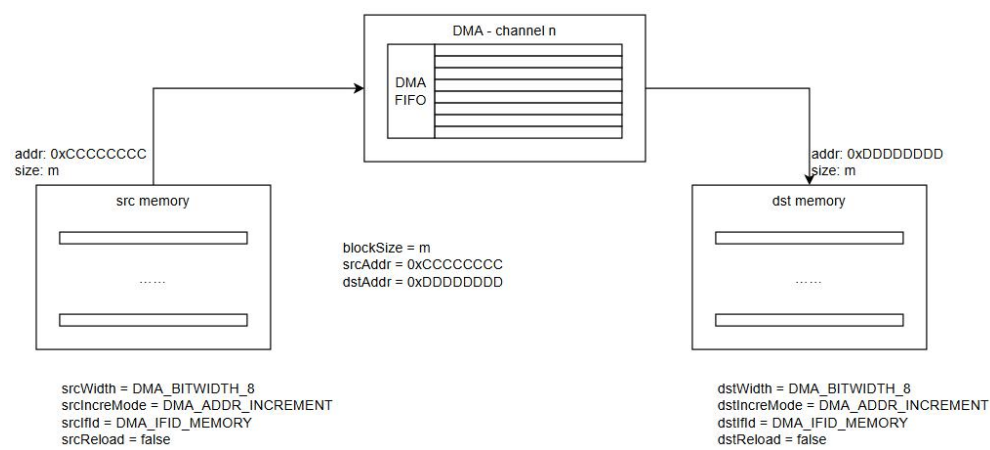


图 5- 13 DMA memory to memory 位宽 8bit 示意图

UART Rx to Memory

DMA 从外设搬运数据至 memory 有几点需要注意：

- 1. 源端位宽和目的端位宽都配置为 DMA_BITWIDTH_8
- 2. burstNum 建议配置为 1，即 UART-Rx FIFO 中只要有 1 byte 数据就触发 DMA 搬运，确保数据搬运干净。

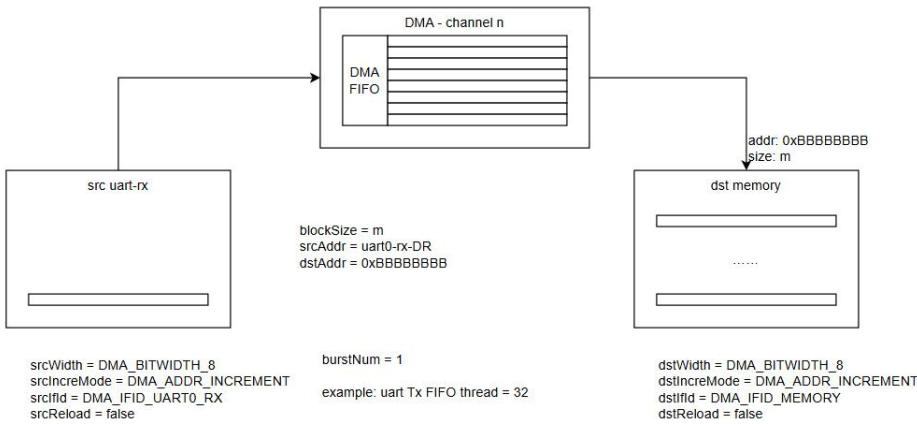


图 5- 14 DMA UART0-RX to memory 示意图

Memory to UART Tx

DMA 从 memory 搬运数据至外设几点需要注意：

- 1. 源端位宽和目的端位宽都配置为 DMA_BITWIDTH_8
- 2. burstNum 与设置的外设 TxFIFO 触发 Level 有关。假设 TxFIFO 设置为 64-n，那么 burstNum 应少于 n，否则触发一次 DMA 传输可能 TxFIFO overflow。假设 burstNum 设置为 1，TxFIFO 设置 1 那么触发 DMA 传输数据还未搬到 UART，TXFIFO 将发空了。所以建议尽量大且少于 TxFIFO size - trigger level，例如 TX FIFO 设置为 32，burstNum 设置为 8。

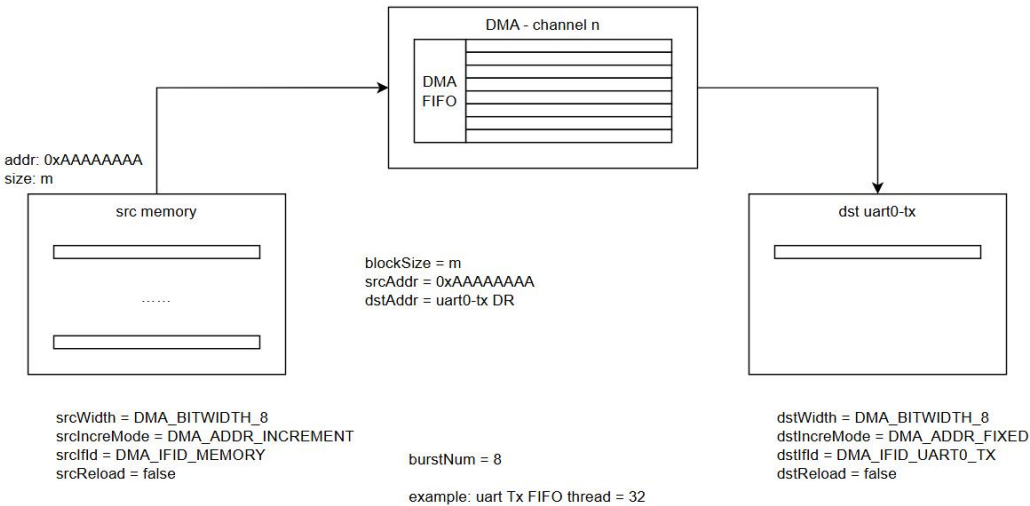


图 5- 15 DMA UART0-RX to memory 示意图

5.5.1.2 链表传输

链表传输使用流程如下图。

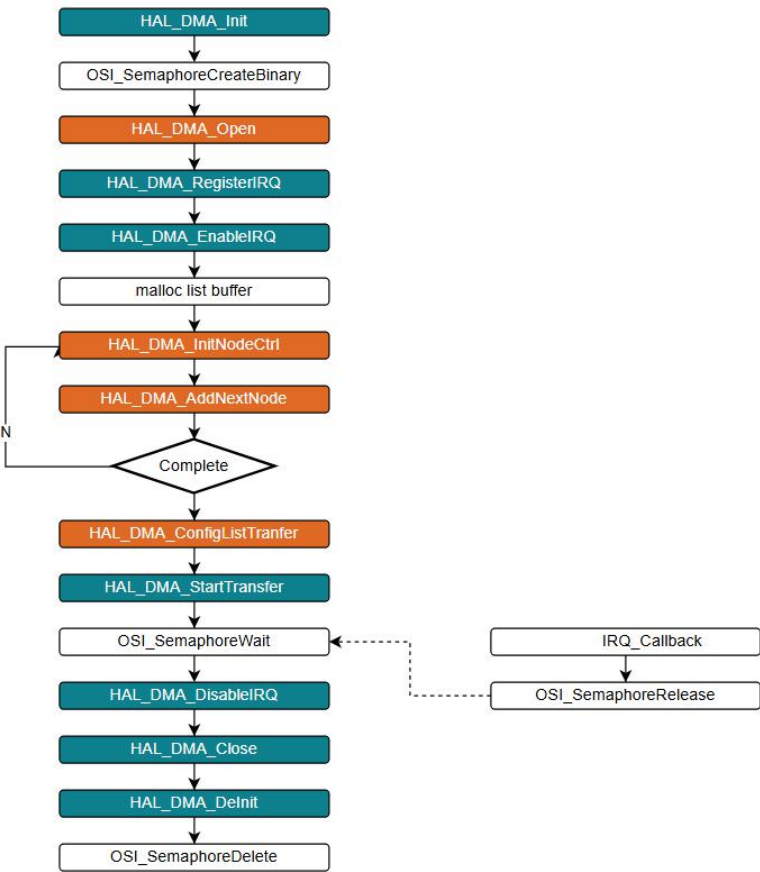


图 5- 16 DMA 链表传输使用流程

链表传输将 DMA 配置存储在 SRAM 并将多组配置初始化成链表，DMA 启动后逐个按照配置项进行传输。相对块传输 HAL_DMA_Open 配置是一致的差异在链表的初始化和组织成链表 HAL_DMA_InitNodeCtrl 初始化单个链表项，HAL_DMA_AddNextNode 组织成链表，HAL_DMA_ConfigListTransfer 则启动链表配置。

5.5.2 接口

5.5.2.1 HAL_DMA_Init

表 5- 112 HAL_DMA_Init

接口	说明
函数	HAL_Dev_t *HAL_DMA_Init(DMA_ID_t id);
功能	初始化 I2C 设备，申请系统资源，获取控制设备句柄。
参数	DMA_ID_t id DMA 外设 ID; typedef enum { DMA0_ID = (0x00U), DMA_NUM, } DMA_ID_t;
注意事项	无。

接口	说明
返回值	DMA 设备控制句柄(HAL_Dev_t *类型)。

5.5.2.2 HAL_DMA_DeInit

表 5- 113 HAL_DMA_DeInit

接口	说明
函数	HAL_Status_t HAL_DMA_DeInit(HAL_Dev_t *dma);
功能	反初始化 DMA 设备，释放系统资源。
参数	HAL_Dev_t *dma DMA 设备实例句柄。
注意事项	无。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.5.2.3 HAL_DMA_Open

表 5- 114 HAL_DMA_Open

接口	说明
函数	int32_t HAL_DMA_Open(HAL_Dev_t *dma, const DMA_DMA_ChannelConf_t *pchanCfg);
功能	配置相关参数，启动 DMA 通道。
参数	HAL_Dev_t *dma DMA 设备控制句柄。 const DMA_ChannelParam_t *pchanCfg 配置通道的相关参数； typedef struct { uint32_t direction : 4; uint32_t priority : 4; uint32_t srcIncrMode : 4; uint32_t dstIncrMode : 4; uint32_t srcBrustNum : 4; uint32_t dstBrustnum : 4; uint32_t srcWidth : 4; uint32_t dstWidth : 4; uint16_t srcHandShake : 4; uint16_t dstHandShake : 4;

接口	说明
	<pre>uint16_t srcEnReload : 1; uint16_t dstEnReload : 1; uint16_t srcEnStatus : 1; uint16_t dstEnStatus : 1; uint16_t pad : 4; } DMA_ChannelConf_t;</pre>
注意事项	无。
返回值	返回值为非负整数，代表申请的通道 ID； 返回值为负数表示申请通道失败；HAL_STATUS_*表示失败类型。

5.5.2.4 HAL_DMA_Close

表 5- 115 HAL_DMA_Close

接口	说明
函数	HAL_Status_t HAL_DMA_Close(HAL_Dev_t *dma, uint32_t chId);
功能	关闭已申请的 DMA 通道，释放通道资源。
参数	HAL_Dev_t *dma DMA 设备实例句柄。 uint32_t chId 关闭的已申请的通道 ID；
注意事项	关闭的通道 ID 必须来自 HAL_DMA_Open 函数申请成功的 ID。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.5.2.5 HAL_DMA_ConfigBlockTranfer

表 5- 116 HAL_DMA_ConfigBlockTransfer

接口	说明
函数	HAL_Status_t HAL_DMA_ConfigBlockTranfer(HAL_Dev_t *dma, uint32_t chId, const DMA_BlockConf_t *pblockCfg);
功能	配置块传输的参数，设置该通道为块传输模式搬运数据。
参数	HAL_Dev_t *dma DMA 设备实例句柄。

接口	说明
	uint32_t chId 申请成功的通道 ID； const DMA_BlockConf_t *pblockCfg 配置的块传输源地址；目标地址以及传输块的总大小的结构体地址； <pre>typedef struct { uint32_t srcAddr; uint32_t dstAddr; uint32_t blockSize; } DMA_BlockConf_t;</pre>
注意事项	1. 配置的通道 ID 必须来自 HAL_DMA_Open 函数申请成功的 ID； 2. 对同通道，通过此函数配置的参数，会覆盖其他 HAL_DMA_Config**函数配置参数，与 HAL_DMA_ConfigListTransfer 函数配置互斥。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.5.2.6 HAL_DMA_InitNodeCtrl

表 5- 117 HAL_DMA_InitNodeCtrl

接口	说明
函数	HAL_Status_t HAL_DMA_InitNodeCtrl(HAL_Dev_t *dma, uint32_t chId, DMA_NodeConf_t *pcurrentNode);
功能	初始化单个链表结点，配置单个结点中相关的 nodeCtrlx 的参数。
参数	HAL_Dev_t *dma DMA 设备实例句柄。 uint32_t chId 申请成功的通道 ID； DMA_NodeConf_t *pcurrentNode 申请的单个链表结点空间的地址； <pre>typedef struct DMA_Node { uint32_t srcAddr; uint32_t dstAddr; struct DMA_Node *pNextNode; uint32_t nodeCtrl0; uint32_t blockSize :12; uint32_t nodeCtrl1 :20;</pre>

接口	说明
	<pre>uint32_t nodeCtrl2; uint32_t nodeCtrl3; } DMA_NodeConf_t;</pre>
注意事项	1. 配置的通道 ID 必须来自 <code>HAL_DMA_Open</code> 函数申请成功的 ID； 2. 对 ID 相同的通道，此函数配置的参数，会覆盖其他 <code>HAL_DMA_Config**</code> 函数配置，与 <code>HAL_DMA_ConfigListTranfer</code>、<code>HAL_DMA_ConfigScatterGatherTranfer</code> 函数配置 互斥使用； 3. 申请传入的空间不可以是空指针； 4. 对成员 <code>srcAddr</code>、<code>dstAddr</code>、<code>blockSize</code> 进行赋值后，再传入该函数进行结点初始化； 5. 如果有多个结点，需要注意 多次调用 对每个结点的 <code>nodeCtrlx</code> 进行配置 <code>chuan'ru</code>。
返回值	返回值为 <code>HAL_STATUS_OK</code>，表示函数执行成功； 返回值为负数表示函数执行失败，<code>HAL_STATUS_*</code> 表示失败类型。

5.5.2.7 HAL_DMA_AddNextNode

表 5- 118 HAL_DMA_AddNextNode

接口	说明
函数	<pre>void HAL_DMA_AddNextNode(DMA_NodeConf_t *pcurrentNode, DMA_NodeConf_t *pnextListNode)</pre>
功能	链接当前结点和下个结点。
参数	DMA_NodeConf_t *pcurrentNode 当前结点的结构体地址； DMA_NodeConf_t *pnextNode 下个结点的结构体地址；
注意事项	1. 配置的通道 ID 必须来自 <code>HAL_DMA_Open</code> 函数申请成功的 ID； 2. 调用时注意两个结点的结点结构体已经通过 <code>HAL_DMA_InitNodeCtrl</code> 配置过结点中的 <code>nodeCtrlx</code> 参数；
返回值	Void

5.5.2.8 HAL_DMA_ConfigListTranfer

表 5- 119 HAL_DMA_ConfigListTranfer

接口	说明
函数	<pre>HAL_Status_t HAL_DMA_ConfigListTranfer(HAL_Dev_t *dma, uint32_t chId,</pre>

接口	说明
	const DMA_NodeConf_t *pheadlist);
功能	链接当前结点和下个结点。
参数	HAL_Dev_t *dma DMA 设备实例句柄。 uint32_t chId 申请成功的通道 ID； DMA_NodeConf_t *pheadlist 已初始化完的链表的头结点的结构体地址；
注意事项	1. 配置的通道 ID 必须来自 HAL_DMA_Open 函数申请成功的 ID； 2. 记录第一个申请的链表结点的地址。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.5.2.9 HAL_DMA_ConfigScatterGatherTranfer

表 5- 120 HAL_DMA_ConfigScatterGatherTranfer

接口	说明
函数	HAL_Status_t HAL_DMA_ConfigScatterGatherTranfer(HAL_Dev_t *dma, uint32_t chId, const DMA_SGConf_t *psgCfg);
功能	链接当前结点和下个结点。
参数	HAL_Dev_t *dma DMA 设备实例句柄。 uint32_t chId 申请成功的通道 ID； const DMA_SGConf_t *psgCfg 聚集离散模式的配置结构体地址； 源/目标地址、传输大小、 源聚集的块数量、目标离散的块数量、 源地址聚集间隔、目标地址聚集间隔、 源聚集功能使能、目标离散功能使能； typedef struct {

接口	说明
	<pre>uint32_t srcAddr; uint32_t dstAddr; uint32_t blockSize; uint32_t srcGatherCount; uint32_t srcGatherInter; uint32_t dstScatterCount; uint32_t dstScatterInter; uint16_t enSrcGather; uint16_t enDstScatter; } DMA_SGConf_t;</pre>
注意事项	1. 配置的通道 ID 必须来自 HAL_DMA_Open 函数申请成功的 ID； 2. 使能需要的 enSrcGather/enDstScatter；默认关闭，有参数配置时注意使能该标志为。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.5.2.10 HAL_DMA_StartTransfer

表 5- 121 HAL_DMA_StartTransfer

接口	说明
函数	HAL_Status_t HAL_DMA_StartTransfer(HAL_Dev_t *dma, uint32_t chId)
功能	启动配置成功的通道进行数据传输。
参数	HAL_Dev_t *dma DMA 设备实例句柄。 uint32_t chId 申请成功的通道 ID；
注意事项	1. 配置的通道 ID 必须来自 HAL_DMA_Open 函数申请成功的 ID； 2. 应该对该通道 ID 有进行相关的配置，再调用 HAL_DMA_StartTransfer 函数。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.5.2.11 HAL_DMA_AbortTransfer

表 5- 122 HAL_DMA_AbortTransfer

接口	说明
函数	HAL_Status_t HAL_DMA_AbortTransfer(HAL_Dev_t *dma, uint32_t chId)

接口	说明
功能	终止通道进行数据传输。
参数	HAL_Dev_t *dma DMA 设备实例句柄。 uint32_t chId 申请成功的通道 ID；
注意事项	配置的通道 ID 必须来自 HAL_DMA_Open 函数申请成功的 ID；
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.5.2.12 HAL_DMA_RegisterIRQ

表 5- 123 HAL_DMA_RegisterIRQ

接口	说明
函数	HAL_Status_t HAL_DMA_RegisterIRQ(HAL_Dev_t *dma, uint32_t chId, DMA_Event_t event, HAL_Callback_t *eventCb);
功能	注册中断服务函数调用的回调函数。
参数	HAL_Dev_t *dma DMA 设备实例句柄。 uint32_t chId 申请成功的通道 ID； DMA_Event_t event 注册中断回调触发的中断事件； 完成触发事件，链表结点/单块完成触发事件， 错误触发事件； <pre>typedef enum { DMA_EVENT_DONE = (HAL_BIT(0)), DMA_EVENT_BLOCK_DONE = (HAL_BIT(1)), DMA_EVENT_ERR = (HAL_BIT(2)), DMA_EVENT_ALL = (DMA_EVENT_DONE DMA_EVENT_BLOCK_DONE DMA_EVENT_ERR), } DMA_Event_t;</pre> HAL_Callback_t *eventCb 注册的中断回调函数的结构体地址；

接口	说明
注意事项	1. 配置的通道 ID 必须来自 <code>HAL_DMA_Open</code> 函数申请成功的 ID； 2. <code>event</code> 参数在配置时， 不可选择 <code>DMA_EVENT_ALL</code> 事件注册中断回调函数。
返回值	返回值为 <code>HAL_STATUS_OK</code> ，表示函数执行成功； 返回值为负数表示函数执行失败， <code>HAL_STATUS_*</code> 表示失败类型。

5.5.2.13 HAL_DMA_UnRegisterIRQ

表 5- 124 HAL_DMA_UnRegisterIRQ

接口	说明
函数	<code>HAL_Status_t HAL_DMA_UnRegisterIRQ(HAL_Dev_t *dma);</code>
功能	反注册中断服务函数调用的回调函数。
参数	HAL_Dev_t *dma DMA 设备实例句柄。
注意事项	配置的通道 ID 必须来自 <code>HAL_DMA_Open</code> 函数申请成功的 ID；
返回值	返回值为 <code>HAL_STATUS_OK</code> ，表示函数执行成功； 返回值为负数表示函数执行失败， <code>HAL_STATUS_*</code> 表示失败类型。

5.5.2.14 HAL_DMA_EnableIRQ

表 5- 125 HAL_DMA_EnableIRQ

接口	说明
函数	<code>HAL_Status_t HAL_DMA_EnableIRQ(HAL_Dev_t *dma, uint32_t chId, DMA_Event_t event);</code>
功能	使能触发事件的中断。
参数	HAL_Dev_t *dma DMA 设备实例句柄。 uint32_t chId 申请成功的通道 ID； DMA_Event_t event 注册中断回调触发的中断事件；

接口	说明
注意事项	配置的通道 ID 必须来自 <code>HAL_DMA_Open</code> 函数申请成功的 ID；
返回值	返回值为 <code>HAL_STATUS_OK</code> ，表示函数执行成功； 返回值为负数表示函数执行失败， <code>HAL_STATUS_*</code> 表示失败类型。

5.5.2.15 HAL_DMA_DisableIRQ

表 5- 126 HAL_DMA_DisableIRQ

接口	说明
函数	<code>HAL_Status_t HAL_DMA_DisableIRQ(HAL_Dev_t *dma, uint32_t chId, DMA_Event_t event);</code>
功能	失能触发事件的中断。
参数	HAL_Dev_t *dma DMA 设备实例句柄。 uint32_t chId 申请成功的通道 ID； DMA_Event_t event 注册中断回调触发的中断事件；
注意事项	配置的通道 ID 必须来自 <code>HAL_DMA_Open</code> 函数申请成功的 ID；
返回值	返回值为 <code>HAL_STATUS_OK</code> ，表示函数执行成功； 返回值为负数表示函数执行失败， <code>HAL_STATUS_*</code> 表示失败类型。

5.5.2.16 HAL_DMA_GetChanStatus

表 5- 127 HAL_DMA_DisableIRQ

接口	说明
函数	<code>int32_t HAL_DMA_GetChanStatus(HAL_Dev_t *dma, uint32_t chId, uint32_t*doneBlockNum);</code>
功能	获取通道状态。
参数	HAL_Dev_t *dma DMA 设备实例句柄。 uint32_t chId 申请成功的通道 ID；

接口	说明
	uint32_t*doneBlockNum 成功传输完成的块数量的变量地址；
注意事项	1. 配置的通道 ID 必须来自 HAL_DMA_Open 函数申请成功的 ID； 2. 单块传输模式下最大块数量为 1。
返回值	返回值为非负整数，表示该通道的使用状态；0 表示空闲，1 表示忙碌。 返回值为负数表示函数执行失败，HAL_STATUS_*表示失败类型。

5.6 FLASH

5.6.1 功能简介

Flash Control 支持操作 spi FLASH。支持 Dual SPI 和 Quad SPI 的 XIP；默认支持单线 SPI 读，擦，写；擦除默认支持 4K 单位擦除。

为兼容不同 spi FLASH，设计了 Flash 支持列表。支持列表存储了多类 Flash 的基本属性，如下。

```
typedef struct {
    uint32_t    flashSize;        /* Flash 存储总大小，单位 Byte */
    uint32_t    spiFreqMax;       /* 支持的 SPI 最高通信频率，单位 Hz */
    uint8_t     spiFormatMax;     /* 支持的 SPI 最高通信格式，0：单线，1：双线，2：四线 */
    uint8_t     qerType;          /* 若选择四线 SPI，Flash 内部 SR 寄存器中四线使能格式 */
    uint8_t     sleepSupport;     /* Flash 是否支持深度休眠 */
    uint8_t     sleepLatency;     /* Flash 休眠延迟时间，单位 us */
    uint8_t     resumeLatency;    /* Flash 休眠推出延迟时间，单位 us */
    uint8_t     chipId[3];        /* Flash Chip Id， 3 bytes */
} FLASH_ChipInfo_t;
```

以下以“GD25WQ80E” SPI FLASH 为例，介绍各参数获取和填写。

Chip ID

FLASH 以 3 byte 的 Flash Chip ID 为索引在 Flash List 中查找。Flash ID 通常标识了 Flash 厂家及类型定义，通过指令“9FH”可获取。HAL_FLASH_Open 时会从 SPI FLASH 中读取 Flash Chip ID，并通过 ID 查找 Flash List，若找到则使用这组参数配置 SPI Flash，若没找到则可以使用默认参数或直接失败退出。

GD25WQ80E

Operation Code	MID7-MID0	ID15-ID8	ID7-ID0
9FH	C8	65	14
90H	C8		13
ABH			13

图 5- 17 Flash Chip ID 示例

SPI Format

SPI Format 表示 FLASH 执行 XIP 时，数据传输是使用 2 线（Dual SPI）还是 4 线（Quad SPI）SPI 传输。如下图 Dual SPI 时，SI 和 SO 同时用作 Data 传输，Quad SPI 时 IO0, IO1, IO2, IO3 同时用作数据传输。

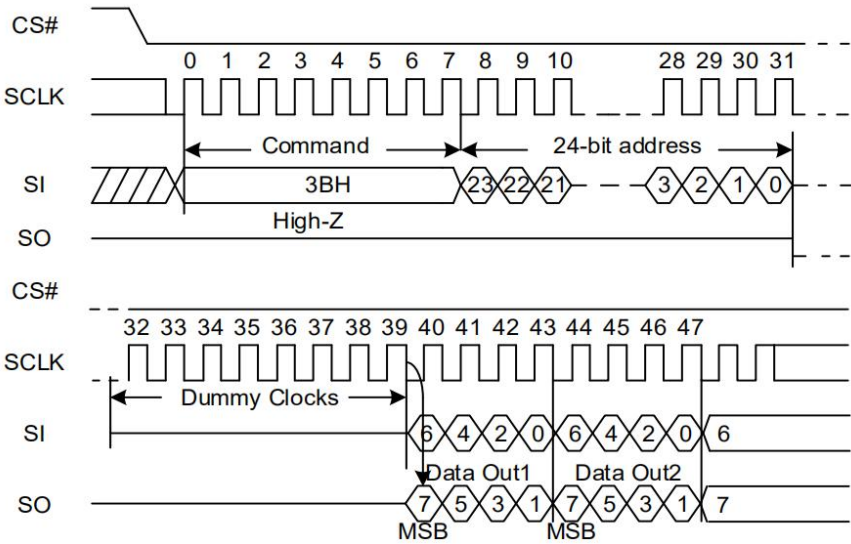


图 5- 18 Dual SPI 传输示例

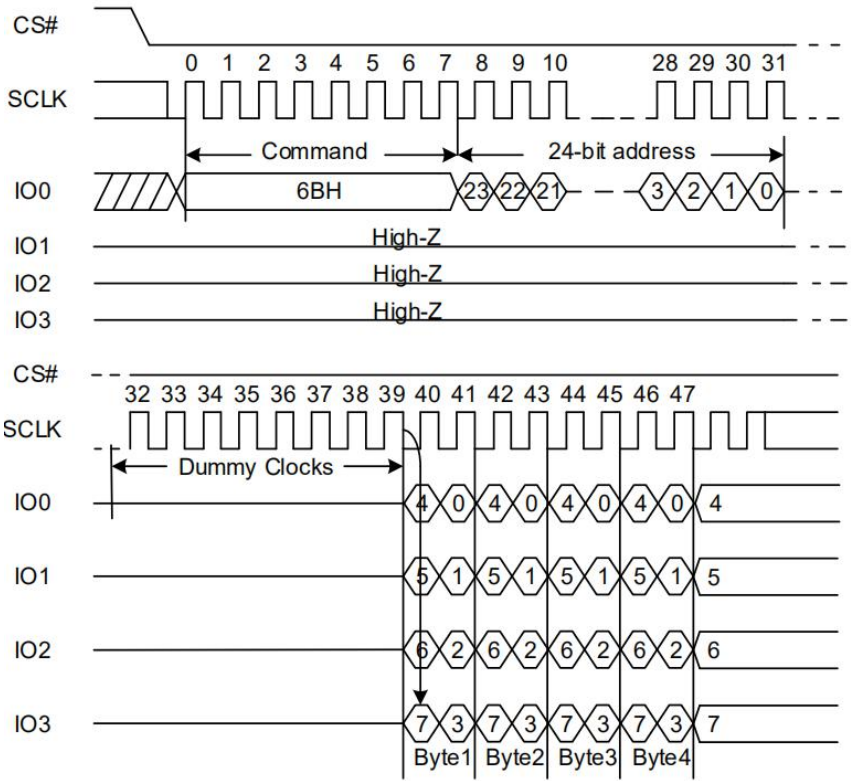


图 5- 19 Quad SPI 传输示例

XIP 时默认使用 Dual SPI，若需使用 Quad SPI 同时需满足以下三个条件：

1. 方案硬件需按照 Quad SPI 设计，HAL_FLASH_Open 的参数携带 spiFormat 需根据硬件方案配置。
2. 使用的 SPI Flash 需支持 Quad SPI，根据 FLASH Spec 填写的 ChipInfo 的 spiFormatMax 需为 2。
3. Flash List 中 ChipInfo 的 qerType 填写正确，Flash 的 Quad SPI 模式需要通过指令配置 Flash 中寄存器才能开启，QerType 即是使能开关所在位置。

通常 SPI FLASH Spec 的 Feature 章节会标注是否支持 Quad SPI，如下。

Standard, Dual, Quad SPI

- Standard SPI: SCLK, CS#, SI, SO, WP#, HOLD#
- Dual SPI: SCLK, CS#, IO0, IO1, WP#, HOLD#
- Quad SPI: SCLK, CS#, IO0, IO1, IO2, IO3

图 5- 20 Spec SPI Format 支持示例

QER 即 Quad Enable Register 即所在 Status Register 中的位置，如下，在 Status Register 2 的 bit 6，表示 S2B6 对应值为 2。

Table 5. Status Register-SR No.2			
No.	Name	Description	Note
S15	SUS	Suspend Bit	Volatile, read only
S14	CMP	Complement Protect Bit	Non-volatile writable
S13	Reserved	Reserved	Reserved
S12	DC	Dummy Configuration Bit	Non-volatile writable
S11	LB1	Security Register Lock Bit	Non-volatile writable (OTP)
S10	LB0	Security Register Lock Bit	Non-volatile writable (OTP)
S9	QE	Quad Enable Bit	Non-volatile writable
S8	SRP1	Status Register Protection Bit	Non-volatile writable

图 5- 21 Spec QER 示例

QER 定义如下，注意 Byte 的 bit 范围是 0~7 不是 1~8。

表 5- 128 QER Type

类型	值	说明
NONE	0	不支持 Quad SPI
S2B1	1	Quad SPI 的使能在 Status Register 2 的第 1 位。
S2B6	2	Quad SPI 的使能在 Status Register 2 的第 6 位。
S2B7	3	Quad SPI 的使能在 Status Register 2 的第 6 位。

注意：以上描述的 SPI Format 用于 XIP 运行指令和读取数据，若使用 HAL_FLASH_Read, HAL_FLASH_Write, HAL_FLASH_Erase 操作 Flash 默认使用单线 SPI。

SPI Max Freq

SPI 运行最高速度影响 XIP 的效率，其通常由以下三个因素决定：

1. 硬件方案 Layout 决定最高运行频率，HAL_FLASH_Open 的参数携带 clockFreq 需根据硬件方案配置。
2. SOC 最高支持频率限制，例如 RS6240 最高支持频率为 AHB 的 2 分频，可通过 HAL_CLOCK_GetFreq 获取系统具体频率。参考 BOARD_InitXIP 函数实现。
3. 使用的 SPI Flash 支持的最高频率，根据 FLASH Spec 填写在 Flash List 中的 spiFreqMax。

注意：SPI Flash Spec 上标注的最高频率与 Dual SPI, Quad SPI 及工作电压有关。

(T=-40℃~85℃, VCC=1.65~3.6V, CL=30pf)

Symbol	Parameter	Min.	Typ.	Max.	Unit.
FC1	Serial Clock Frequency For: all commands except Read (03H), DC=1 on 2.3-3.6V power supply			104	MHz
FC2	Serial Clock Frequency For: all commands except Read (03H), DC=1 on 1.65-2.3V power supply			80	MHz
fC	Serial Clock Frequency For: all commands except Read (03H), DC=0			66	MHz
FR	Serial Clock Frequency For: Read (03H)			50	MHz

图 5- 22 Spec SPI max freq 示例

Flash Size

在 OTA 和 KVF 文件系统时需要获知整个 SPI Flash 大小。Flash List 中的 flashSize 标注了这款 Flash 的大小，HAL_FLASH_Open 函数执行时会通过回绕比较的方式自动检测 Flash 大小。

Sleep

在低功耗场景，需要 SPI Flash 进入深度休眠状态。通常发生“B9H”指令让 Flash 进入休眠状态，发送“ABH”指令让 Flash 退出休眠状态。在进入或退出时通常需要一定延迟等待 Flash 进入状态。Flash List 中 sleepSupport, sleepLatency, resumeLatency 分别表示这些参数。

tDP	CS# High To Deep Power-Down Mode			3	μs
tRES1	CS# High To Standby Mode Without Electronic Signature Read			30	μs
tRES2	CS# High To Standby Mode With Electronic Signature Read			30	μs

图 5- 23 Spec sleep latency 示例

默认在 PM 开启时，Flash 驱动自动会进行休眠唤醒。如前面所述，唤醒 SPI Flash 需要增加额外功耗，而且频繁的休眠唤醒需要浪费保存和恢复的时间，但应用可能又不需要运行 XIP，在此场景下可以使用 HAL_FLASH_ExtControl 关闭自动休眠唤醒，应用在使用 XIP 时主动调用 HAL_FLASH_Resume 恢复 XIP，使用完后调用 HAL_FLASH_Suspend 休眠 XIP。

5.6.2 接口

5.6.2.1 HAL_FLASH_Init

表 5- 129 HAL_FLASH_Init

接口	说明
函数	HAL_Dev_t * HAL_FLASH_Init(FLASH_ID_t id);
功能	初始化 FLASH，申请系统资源但并不初始化任何硬件。
参数	FLASH_ID_t id FLASH 外设 ID，默认只有一个 FLASH_DEFT_ID 即 0；
注意事项	1. 初始化内部设计了防呆，反复初始化仅返回控制句柄；

接口	说明
	2. 系统默认在 Board.c 中初始化，不需要用户调用；
返回值	FLASH 控制句柄，若初始化失败则返回 NULL。

5.6.2.2 HAL_FLASH_DeInit

表 5- 130 HAL_FLASH_DeInit

接口	说明
函数	HAL_Status_t HAL_FLASH_DeInit(HAL_Dev_t * flash);
功能	反初始化 FLASH, 释放系统资源。
参数	HAL_Dev_t * flash FLASH 句柄
注意事项	1. 反初始化前需确认各模块不再使用 FLASH, 通常不使用此 API。
返回值	无。

5.6.2.3 HAL_FLASH_Open

表 5- 131 HAL_FLASH_Open

接口	说明
函数	HAL_Status_t HAL_FLASH_Open(HAL_Dev_t * flash, FLASH_InitParam_t * pInitParam);
功能	开启和初始化配置 Flash Ctrl。
参数	HAL_Dev_t * flash FLASH 句柄 FLASH_InitParam_t * pInitParam 配置参数 typedef struct { uint32_t clockFreq; /* spi 工作频率 */ FLASH_ClockSrc_t clockSrc; /* flash ctrl 时钟源 */ FLASH_SpiFormat_t spiFormat; /* spi 格式, Standard/Dual/Quad SPI */ bool psramEnable; /* psram 使能 */ } FLASH_InitParam_t;

接口	说明
注意事项	输入配置参数受限于 Flash List 中的参数，即最终选择参数的最高规格不会超过 Flash List 中的参数。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.6.2.4 HAL_FLASH_Close

表 5- 132 HAL_FLASH_Close

接口	说明
函数	HAL_Status_t HAL_FLASH_Close(HAL_Dev_t * flash);
功能	关闭 Flash Ctrl
参数	HAL_Dev_t * flash FLASH 句柄
注意事项	关闭后 Flash 将不能进行读，擦，写和执行 XIP 代码。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.6.2.5 HAL_FLASH_Read

表 5- 133 HAL_FLASH_Read

接口	说明
函数	HAL_Status_t HAL_FLASH_Read(const HAL_Dev_t *flash, uint32_t addr, uint8_t *pdata, uint32_t size);
功能	从 Flash 中读取数据
参数	HAL_Dev_t * flash FLASH 句柄 uint32_t addr 读取起始地址，取值范围为：0 ~ flash size uint8_t *pdata 数据读取存储 buffer。 uint32_t size 待读取数据长度。

接口	说明
注意事项	1. 当 Flash 未 open 或处于休眠状态，读取将失败。 2. 读取范围不能超过 flash size，否则将产生回绕。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.6.2.6 HAL_FLASH_Write

表 5- 134 HAL_FLASH_Write

接口	说明
函数	HAL_Status_t HAL_FLASH_Write(const HAL_Dev_t *flash, uint32_t addr, const uint8_t *pdata, uint32_t size);
功能	写入数据至 flash 中。
参数	HAL_Dev_t * flash FLASH 句柄 uint32_t addr 写入起始地址，取值范围为：0 ~ flash size。 uint8_t *pdata 写入数据 buffer。 uint32_t size 写入数据长度。
注意事项	1. 当 Flash 未 open 或处于休眠状态，写入将失败。 2. 写入范围不能超过 flash size，否则将产生回绕。 3. 写入前需擦除对应区域。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.6.2.7 HAL_FLASH_Erase

表 5- 135 HAL_FLASH_Erase

接口	说明
函数	HAL_Status_t HAL_FLASH_Erase(const HAL_Dev_t *flash, uint32_t addr, uint32_t size);

接口	说明
功能	擦除 Flash 指定区域。
参数	HAL_Dev_t * flash FLASH 句柄 uint32_t addr 擦除起始地址，取值范围为：0 ~ flash size，需 4KB 对齐。 uint32_t size 擦除长度，自动 4KB 对齐。
注意事项	1. 当 Flash 未 open 或处于休眠状态，擦除将失败。 2. 擦除起始地址需 4KB 对齐。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.6.2.8 HAL_FLASH_ExtControl

表 5- 136 HAL_FLASH_ExtControl

接口	说明
函数	HAL_Status_t HAL_FLASH_ExtControl(HAL_Dev_t *flash, FLASH_Attr_t attr, void * arg, uint32_t argSize);
功能	Flash 扩展控制
参数	HAL_Dev_t * flash FLASH 句柄 FLASH_Attr_t attr 参数类型 typedef enum { FLASH_GET_FLASH_SIZE = 0, // 获取 flash 总大小 FLASH_GET_PAGE_SIZE = 1, // 获取 flash 页大小，默认 256 FLASH_GET_BLOCK_SIZE = 2, // 获取 flash 块大小，默认 4KB FLASH_GET_PSRAM_SIZE = 3, // 获取 psram 大小，psram 开启时有效 FLASH_GET_CHIP_ID = 4, // 获取 flash chip id FLASH_EXT_GET_CLK_SRC = 5, // 获取 flash ctrl 使用的时钟源 FLASH_EXT_GET_CLK_DIV = 6, // 获取 flash ctrl 时钟分频

接口	说明
	FLASH_EXT_GET_CLK_FREQ = 7, // 获取 spi 使用的频率 FLASH_EXT_GET_SPI_FORMAT = 8, // 获取 spi 使用格式 FLASH_EXT_GET_SLEEP_STATE = 9, // 获取休眠控制状态 FLASH_EXT_SET_SLEEP_STATE = 10, // 设置是否开启自动休眠 FLASH_EXT_GET_SLEEP_LATENCY = 11, // 获取休眠延迟 FLASH_EXT_SET_SLEEP_LATENCY = 12, // 设置休眠延迟 FLASH_EXT_GET_RESUME_LATENCY = 13, // 获取恢复延迟 FLASH_EXT_SET_RESUME_LATENCY = 14, // 设置恢复延迟 FLASH_EXT_GET_AUTO_SLEEP_STATE = 15, // 设置自动休眠模式状态 FLASH_EXT_SET_AUTO_SLEEP_STATE = 16, // 获取自动休眠模式状态 FLASH_EXT_GET_LIST_SIZE = 17, // 获取 flash list 大小 FLASH_EXT_GET_CHIP_INFO_DEFAULT = 18, // 获取默认 flash list FLASH_EXT_GET_CHIP_INFO_LIST = 19, // 获取 flash list FLASH_EXT_GET_CHIP_INFO_DETECH = 20, // 获取 auto detect chip info FLASH_EXT_GET_CHIP_INFO_MODE = 21, // 获取 chip info 模式 FLASH_EXT_SET_CHIP_INFO_MODE = 22, // 设置 chip info 模式 FLASH_EXT_GET_USED_CHIP_INFO_MODE = 23, // 获取当前使用的模式 } FLASH_Attr_t; void * arg 设置或获取的参数值 uint32_t argSize 设置或获取的参数长度。
注意事项	1. 控制和获取参数可在 Open 前使用，但有些参数 Open 后才有效，例如 flash size。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.6.2.9 HAL_FLASH_Suspend

表 5- 137 HAL_FLASH_Suspend

接口	说明
----	----

接口	说明
函数	HAL_Status_t HAL_FLASH_Suspend(HAL_Dev_t * flash);
功能	休眠 flash
参数	HAL_Dev_t * flash FLASH 句柄
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.6.2.10 HAL_FLASH_Resume

表 5- 138 HAL_FLASH_Resume

接口	说明
函数	HAL_Status_t HAL_FLASH_Resume(HAL_Dev_t * flash);
功能	唤醒 flash
参数	HAL_Dev_t * flash FLASH 句柄
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.7 WDG

5.7.1 功能简介

Watchdog 常用于监控系统异常“挂死”，触发系统复位等纠错处理，达到恢复系统部分功能的目的。WatchDog 是一个递减计数定时器，若计数值递减至 0 将触发以下三种行为中的一种，使用时需在计数器递减至 0 前对计数值进行恢复，即“喂狗”。

- 1. SYS Reset：系统复位，复位整个 SOC 硬件。
- 2. CPU Reset：CPU 复位，复位 CPU 但 SOC 其他硬件不会被复位。
- 3. IRQ：产生中断。

如下图，使用时首先调用“HAL_WDG_Init”获取驱动资源，再调用“HAL_WDG_Open”配置 watchdog “喂狗”超时时间（单位 ms）和触发模式（SYS/CPU/IRQ），若是中断模式则需注册中断回调函数，至

此 watchdog 已经开启。在超时时间前调用“HAL_WDG_Feed”进行喂狗，若程序异常未及时调用“HAL_WDG_Feed”将会触发复位或中断。

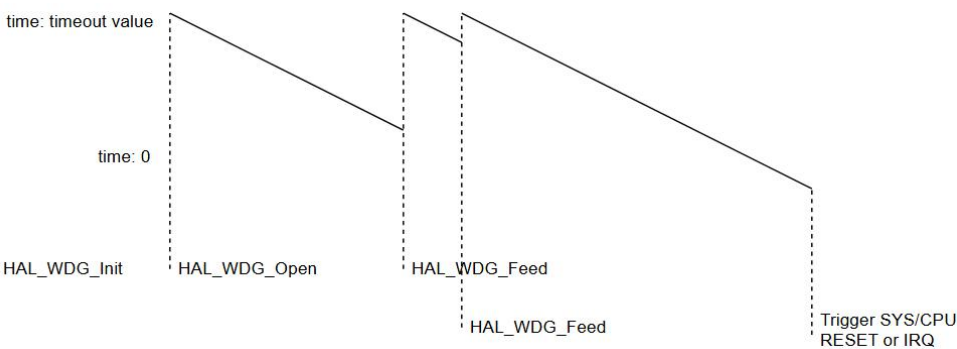


图 5- 24 watchdog 使用示意图

其他注意事项：

- 1. HAL_WDG_Close 可关闭 watchdog，关闭后调用 HAL_WDG_Feed 返回正确但不会执行任何硬件操作。
- 2. HAL_WDG_Close 是关闭硬件，HAL_WDG_DeInit 是释放驱动资源，先关闭硬件再释放资源。
- 3. HAL_WDG_Open 的配置参数可以传入 NULL 表示使用上次 Open 参数重新开启 watchdog。
- 4. Watchdog 在休眠时硬件会掉电，开启 PM 后 watchdog 驱动会自动恢复配置。

参考 sample：project/driver/wdg

5.7.2 接口

5.7.2.1 HAL_WDG_Init

表 5- 139 HAL_WDG_Init

接口	说明
函数	HAL_Dev_t * HAL_WDG_Init(WDG_ID_t id);
功能	初始化 WDG，申请系统资源但并不初始化任何硬件。
参数	WDG_ID_t id WDG 外设 ID，默认只有一个 WDG_DEFT_ID 即 0；
注意事项	
返回值	WDG 控制句柄，若初始化失败则返回 NULL。

5.7.2.2 HAL_WDG_DeInit

表 5- 140 HAL_WDG_Deinit

接口	说明
----	----

接口	说明
函数	HAL_Status_t HAL_WDG_DeInit(HAL_Dev_t *wdg);
功能	反初始化 watchdog, 释放系统资源。
参数	HAL_Dev_t * wdg WDG 句柄
注意事项	
返回值	返回值为 HAL_STATUS_OK, 表示函数执行成功; 返回值为负数表示函数执行失败, 具体参考 HAL_STATUS_*。

5.7.2.3 HAL_WDG_Open

表 5- 141 HAL_WDG_Open

接口	说明
函数	HAL_Status_t HAL_WDG_Open(HAL_Dev_t *wdg, WDG_InitParam_t * pInitParam);
功能	开启 watchdog。
参数	HAL_Dev_t * wdg WDG 句柄 WDG_InitParam_t * pInitParam 配置参数 typedef struct { WDG_Mode_t mode; //模式, 可选择复位 CPU 或复位系统或只产生中断 /* ms as unit */ uint32_t timeout; // 喂狗超时时间, 单位 ms /* take effect when mode is interrupt. */ HAL_CbFunc_t callback; //中断模式回调函数 } WDG_InitParam_t; typedef enum { WDG_MODE_RESET_CPU = 0, WDG_MODE_RESET_SYS = 1, WDG_MODE_INTERRUPT = 2, WDG_MODE_MAX, } WDG_Mode_t;
注意事项	若 pInitParam 为 NULL 表示使用上次 Open 时的参数。

接口	说明
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.7.2.4 HAL_WDG_Close

表 5- 142 HAL_WDG_Close

接口	说明
函数	HAL_Status_t HAL_WDG_Close(HAL_Dev_t *wdg);
功能	关闭 watchdog
参数	HAL_Dev_t * wdg WDG 句柄
注意事项	无
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

5.7.2.5 HAL_WDG_Feed

表 5- 143 HAL_WDG_Feed

接口	说明
函数	HAL_Status_t HAL_WDG_Feed(HAL_Dev_t *wdg);
功能	喂狗
参数	HAL_Dev_t * wdg WDG 句柄
注意事项	在 wdg 计数到之前喂狗。
返回值	返回值为 HAL_STATUS_OK，表示函数执行成功； 返回值为负数表示函数执行失败，具体参考 HAL_STATUS_*。

免责声明和著作权声明

本司在本文档中提供的信息力求准确，但并不保证内容完全无误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或不遵守本文档使用说明而进行的任何不当使用行为（包括但不限于如超压，超频，超温使用）所造成的损失，本司概不负责。

本文中的内容，包括但不限于供参考的 URL 地址，由于产品版本升级或其他原因，与实际内容存在差异，如有变更，恕不另行通知。文档“按现状”提供，不负任何担保责任，包括对适销性、适用于特定用途或非侵权性的任何担保，和任何提案、规格或样品在他处提到的任何担保。

本文档不负任何责任，包括使用本文档内信息产生的侵犯任何专利权行为的责任。本文档在此未以禁止反言或其他方式授予任何知识产权使用许可，不管是明示许可还是暗示许可。

版权所有©2026 珠海正和微芯科技有限公司。保留所有权利。